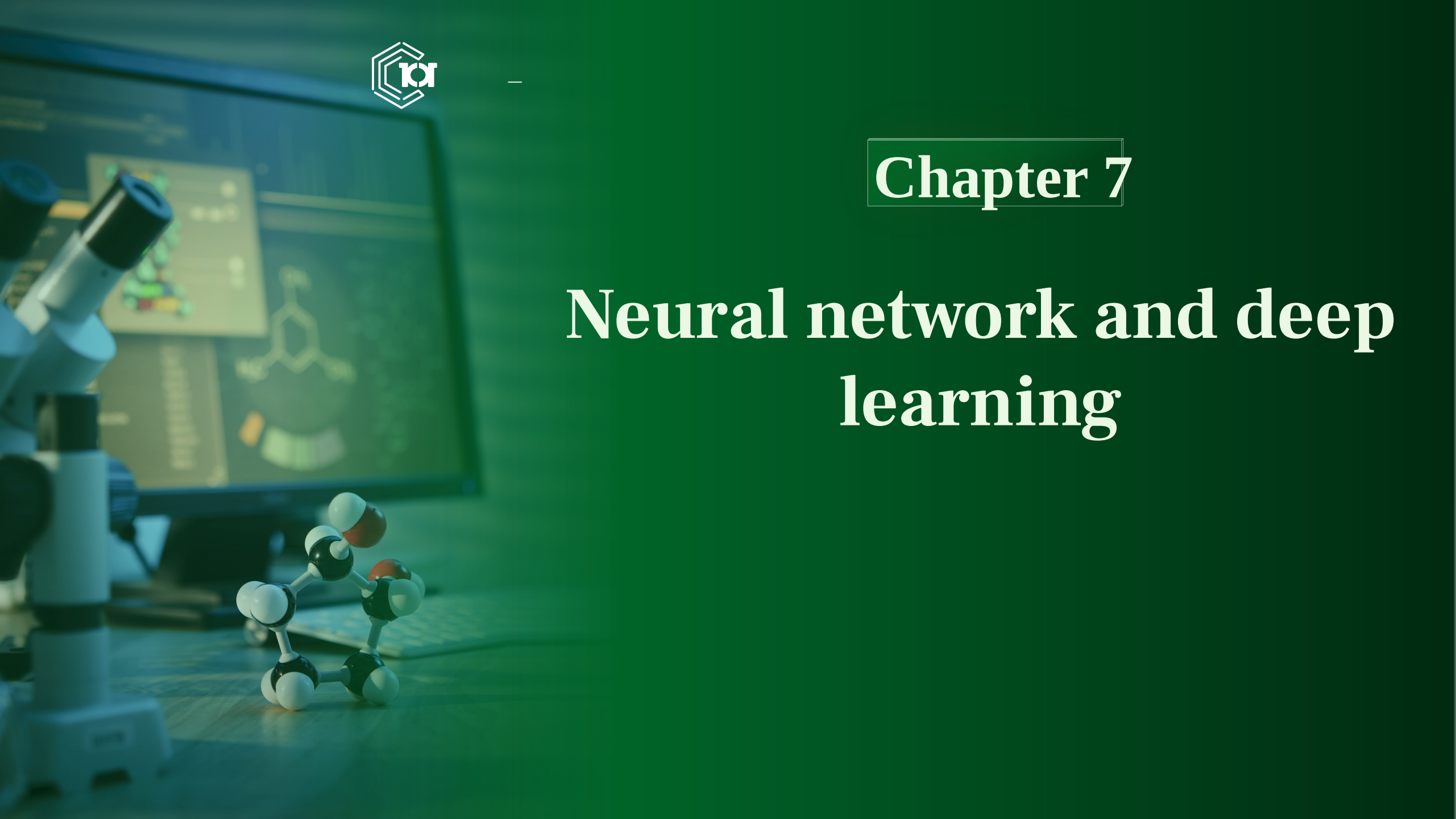




Chapter 7

Neural network and deep learning



Contents

1. Why neural network and deep learning?

- Learning map (where we are now)
- Tasks
- Basic knowledge and tips

2. Classic architectures

- CNN
- RNN
- GNN
- Transformer
- Reinforcement learning
- Diffusion

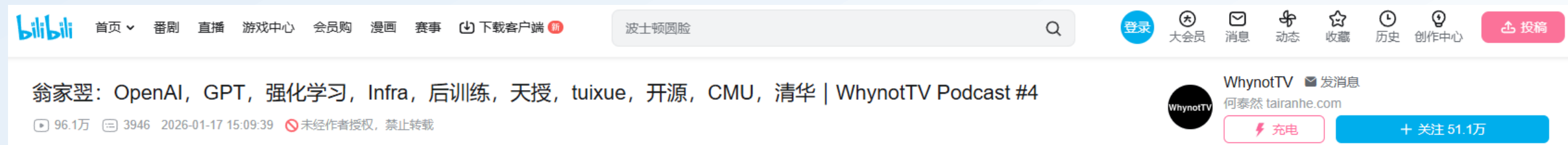
3. Replication code

Before we start (Take home message)

- A perspective on understanding the world

Optimization

Probability and Expectation

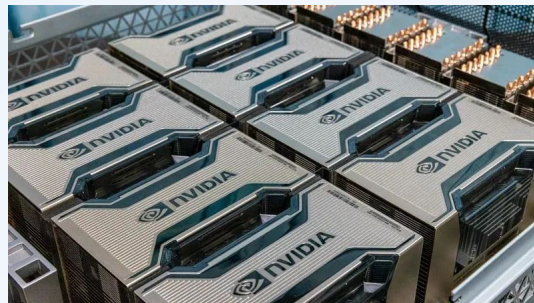
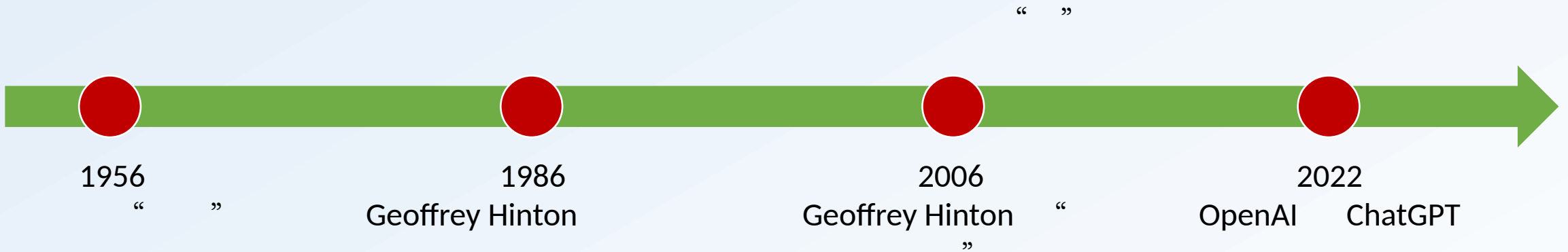


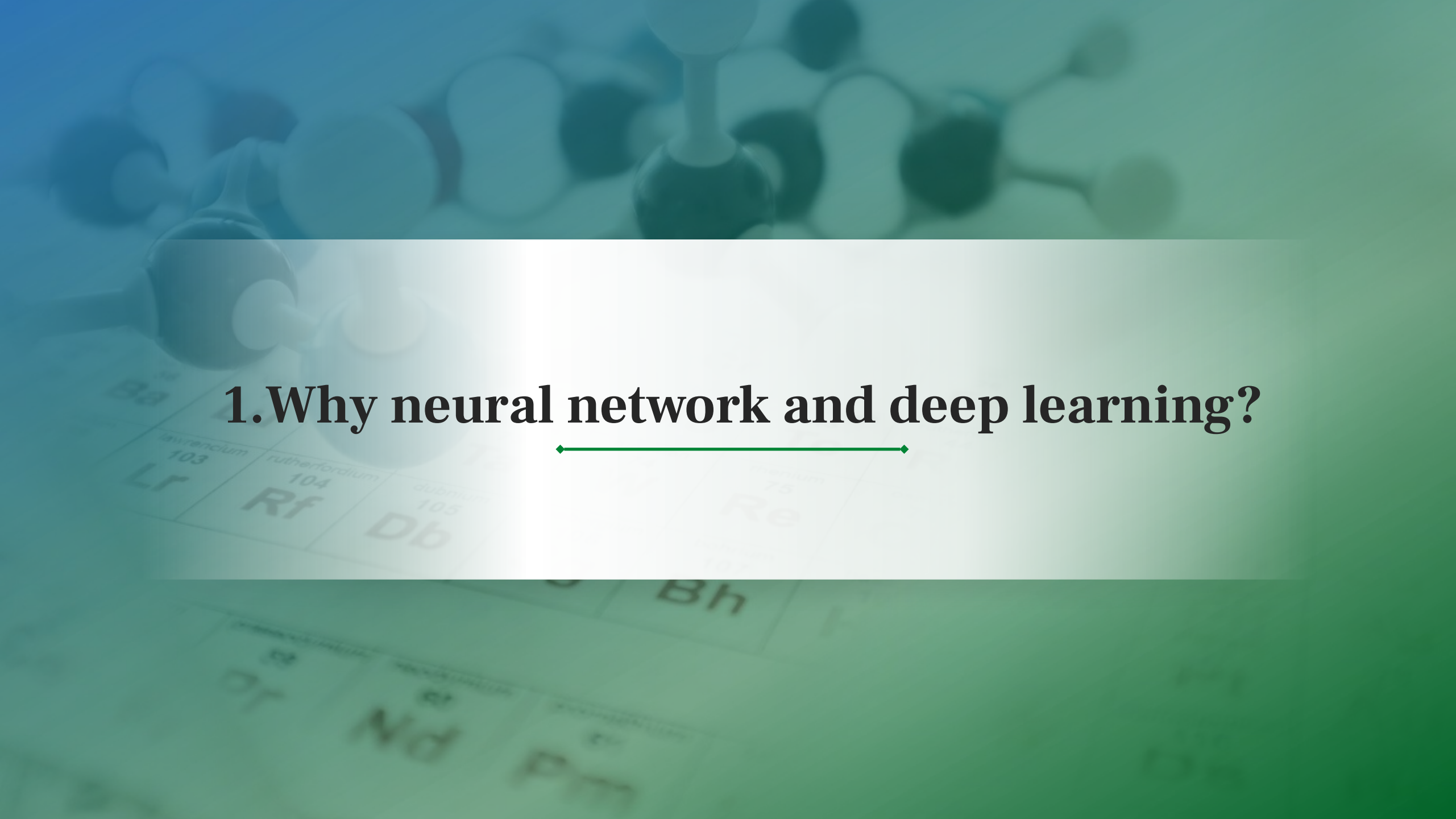
- Baseline for you to learn machine learning

Recognition

Math/Algebra

Before we start (Take home message)



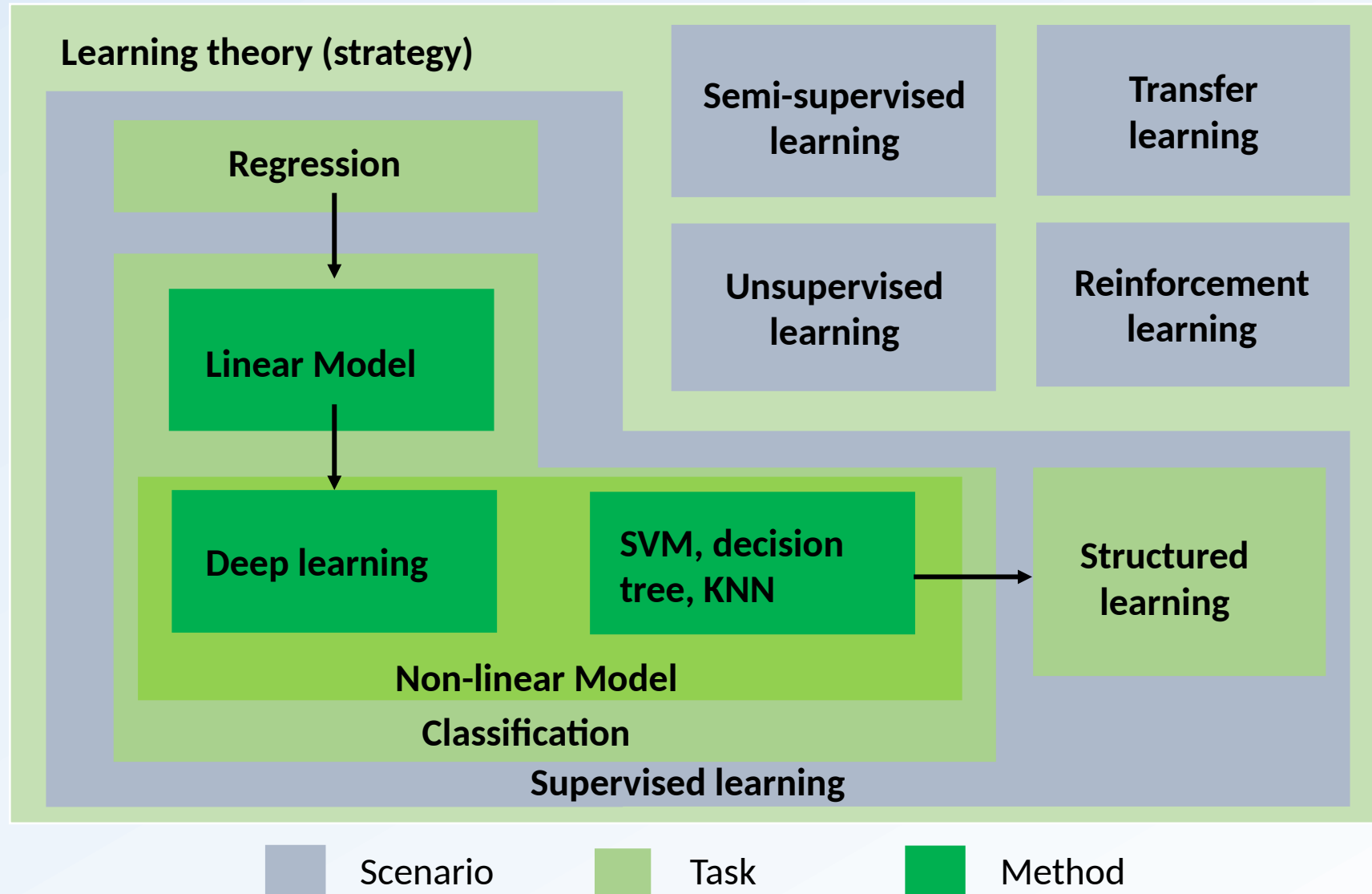


1. Why neural network and deep learning?

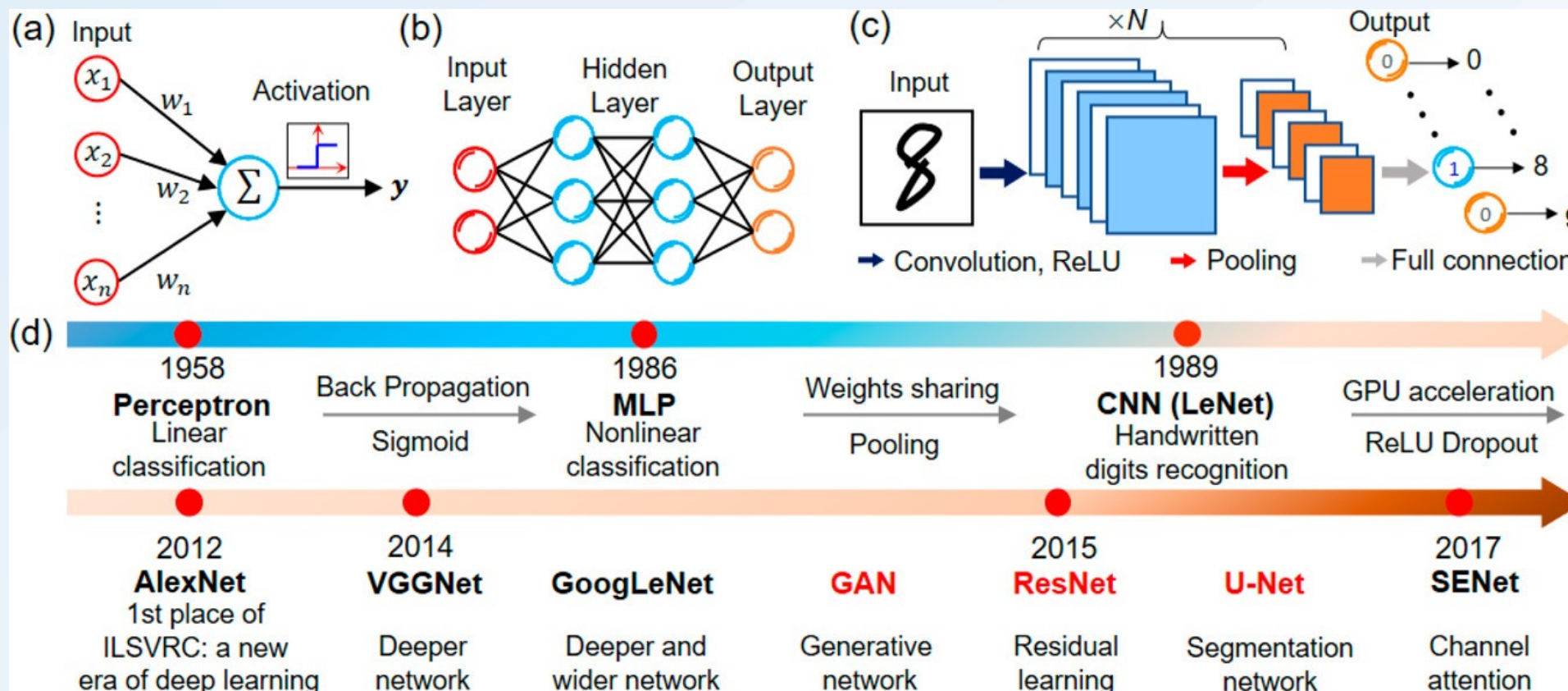
Why neural network and deep learning?



◆ Learning map (where we are now)



◆ History of neural network



Transformer

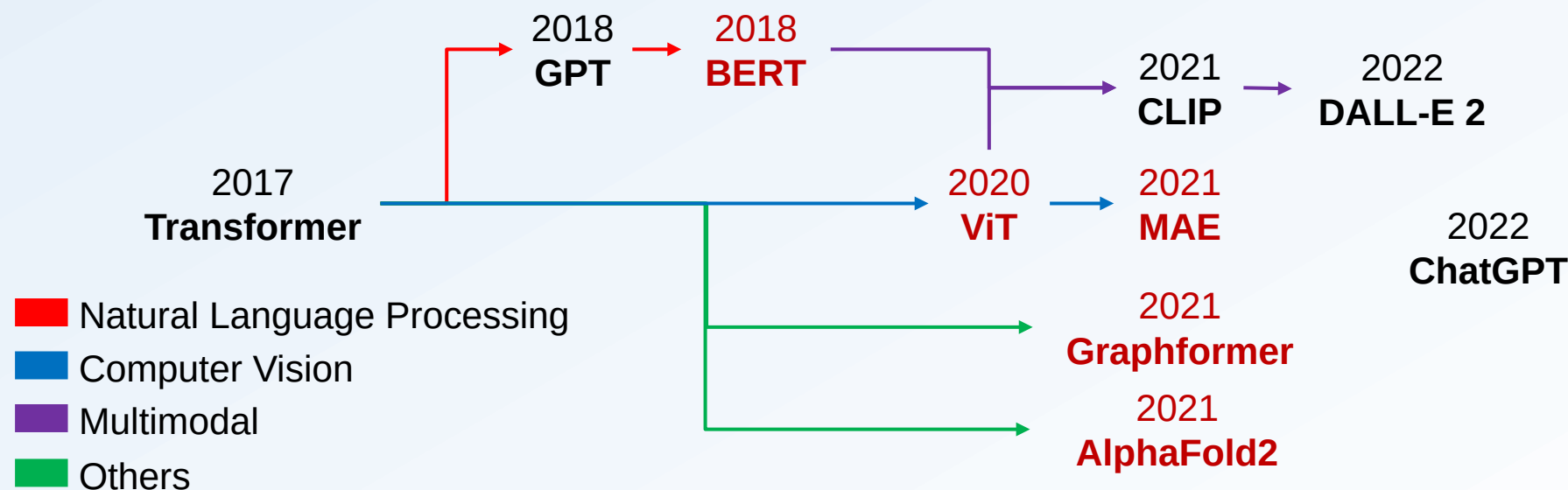
Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com



Transformer has been a fundamental structure of deep learning since 2017

◆ Perceptron()

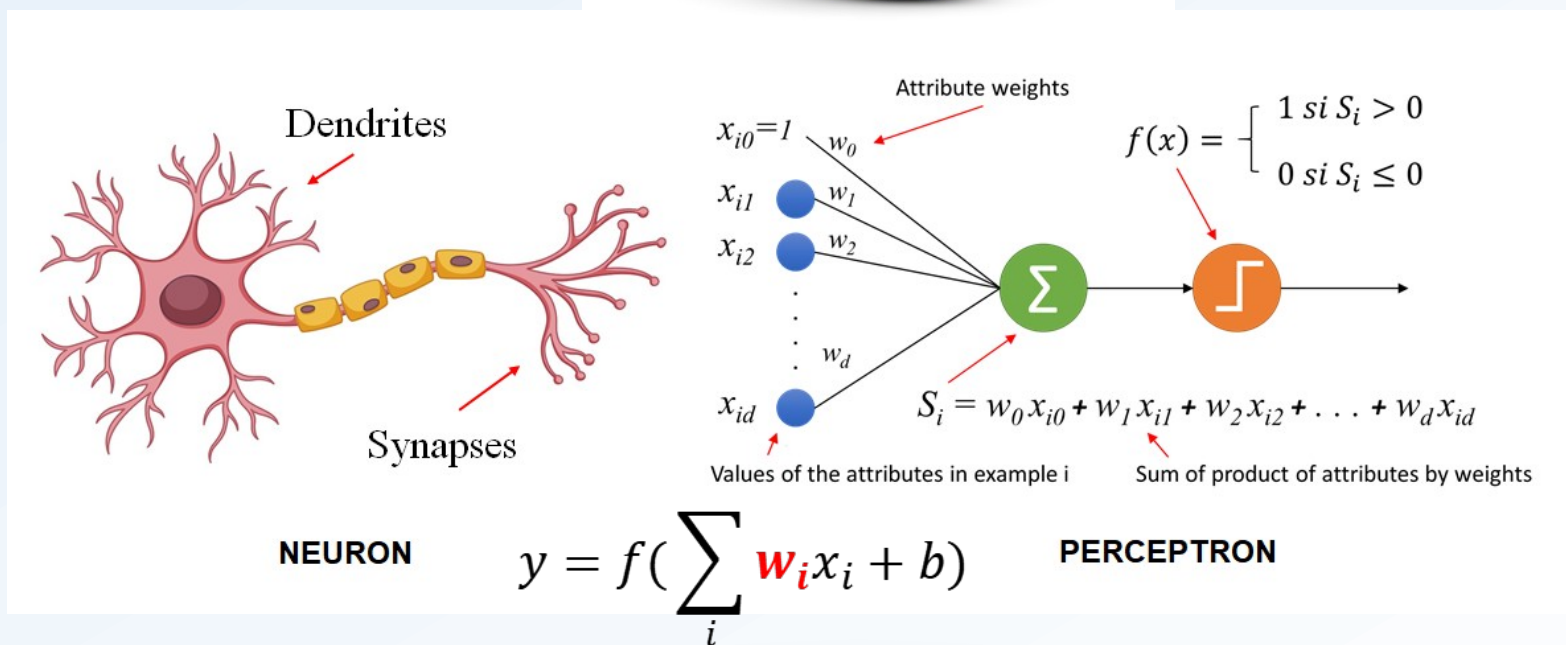
→ x_0

→ x_1

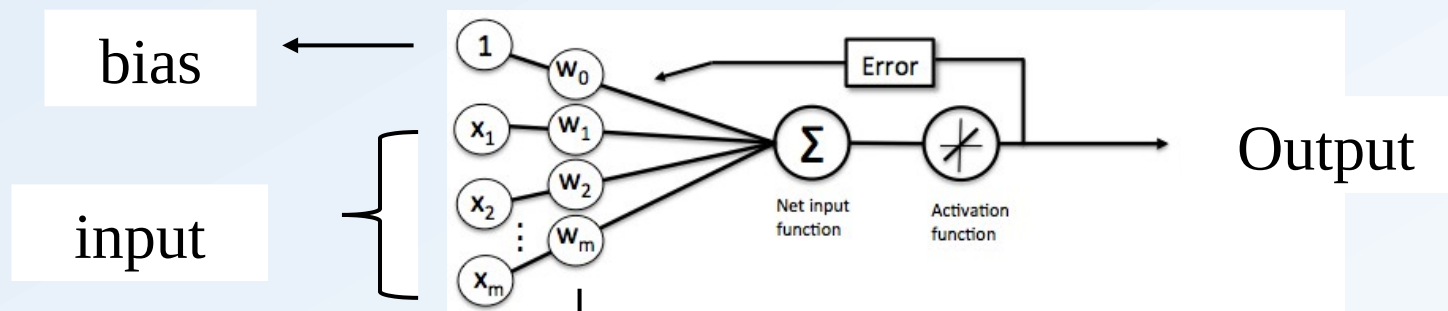
→ x_2



or ? → y



◆ Perceptron()



weight

$$y = h(x) = w_1 \times x_1 + w_2 \times x_2 + \dots + w_n \times x_n + b$$

$$= w_0 \times x_0 + w_1 \times x_1 + \dots + w_n \times x_n = w^T x$$

$$f(z) = \begin{cases} 1 \\ 0 \end{cases}$$

$$\begin{matrix} 1 & / \\ > z > 0.5 \\ 0.5 & > z > 0 \end{matrix}$$

“0-1”classification
to
linear regression

$$f(z) = z$$

How to train it?

0~1

◆ Gradient descent()

$$e = \frac{1}{2} (y - \bar{y})^2$$

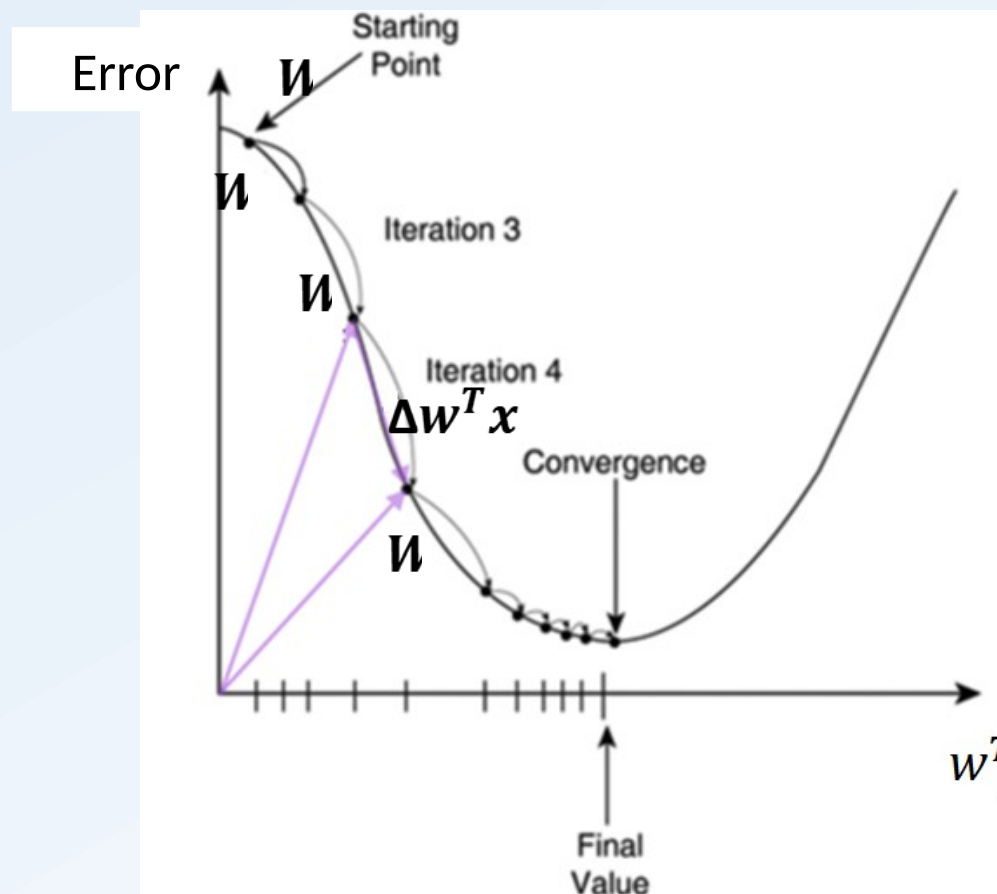
Where y is actual value
and $\bar{y}$ is predicted value

$$y = w^T x:$$

$$\begin{aligned} E &= e^{(1)} + e^{(2)} + e^{(3)} + \dots + e^{(n)} \\ &= \sum_{i=1}^n e^{(i)} \\ &= \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \bar{y}^{(i)})^2 \\ E(w) &= \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \bar{y}^{(i)})^2 \\ &= \frac{1}{2} \sum_{i=1}^n (y^{(i)} - w^T x^{(i)})^2 \end{aligned}$$

**How to get the
minimum value?**

◆ Gradient descent()



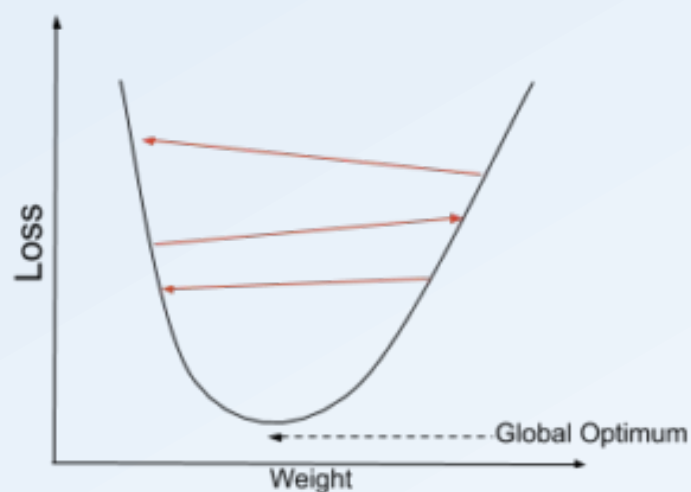
$$\begin{aligned}\nabla E(\mathbf{w}) &= \frac{\partial}{\partial \mathbf{w}} E(\mathbf{w}) \\ &= \frac{\partial}{\partial \mathbf{w}} \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \bar{y}^{(i)})^2 \\ &= - \sum_{i=1}^n (y^{(i)} - \bar{y}^{(i)}) \mathbf{x}\end{aligned}$$

$$\mathbf{w}_{new} = \mathbf{w}_{old} - \eta \nabla E(\mathbf{w})$$

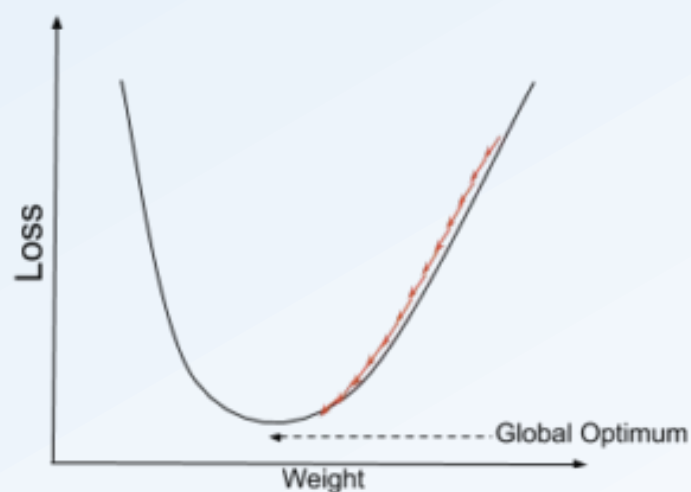
$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y - \bar{y}) \mathbf{x}$$

Where η is learning rate
(学习率, 学习步长)
and ∇ is gradient
(学习的方向)

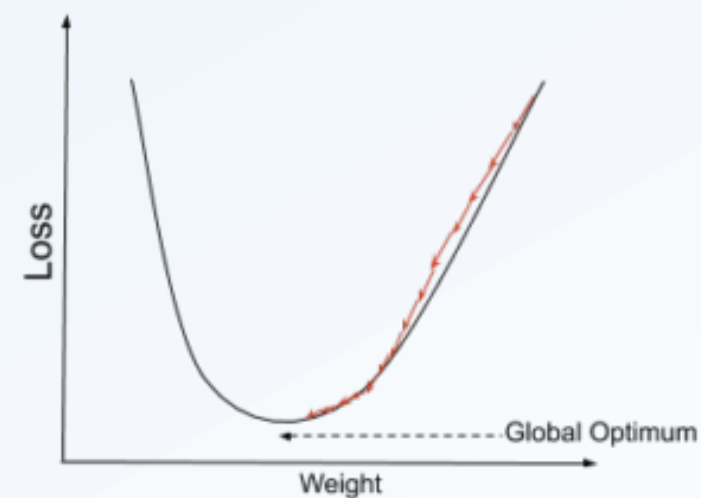
◆ The effect of learning rate



To High-Overshooting Problem

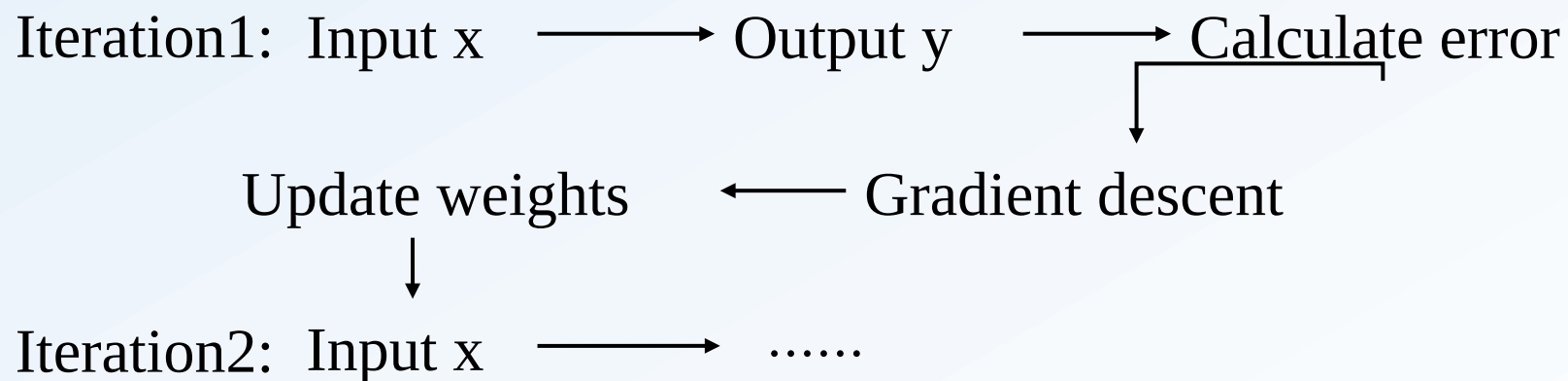
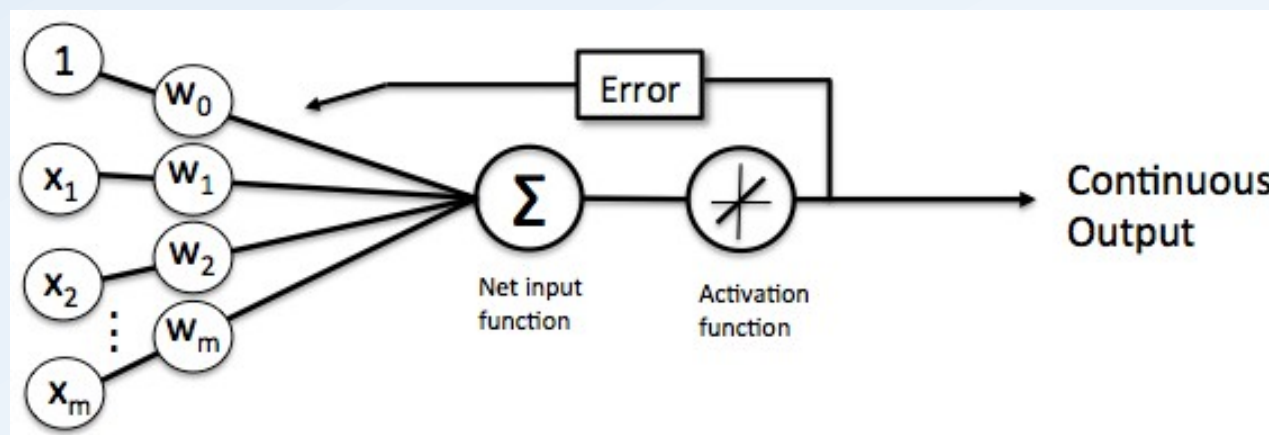


To-Low- Take Time to Train



Lower the learning rate as the training progresses

Summary



ONE CONSENSUS



Geoffrey Hinton

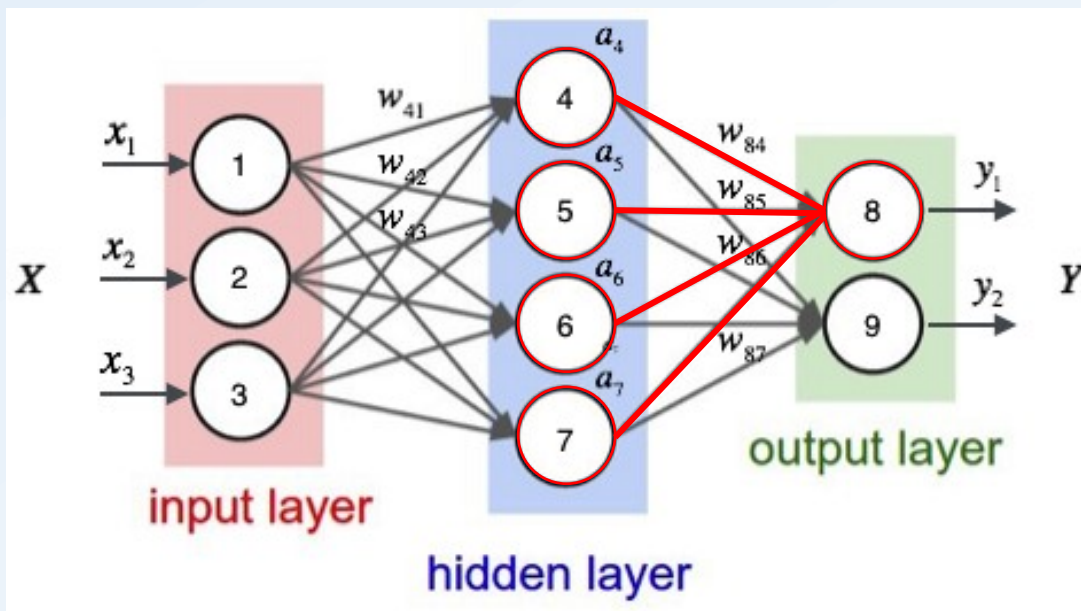


Yann LeCun



Yoshua Bengio

◆ Neural networks



Parameters: weights

Hyper-parameters():

connection mode
the number of layers
the number of nodes

(parameters that are set manually)

full connected neural network

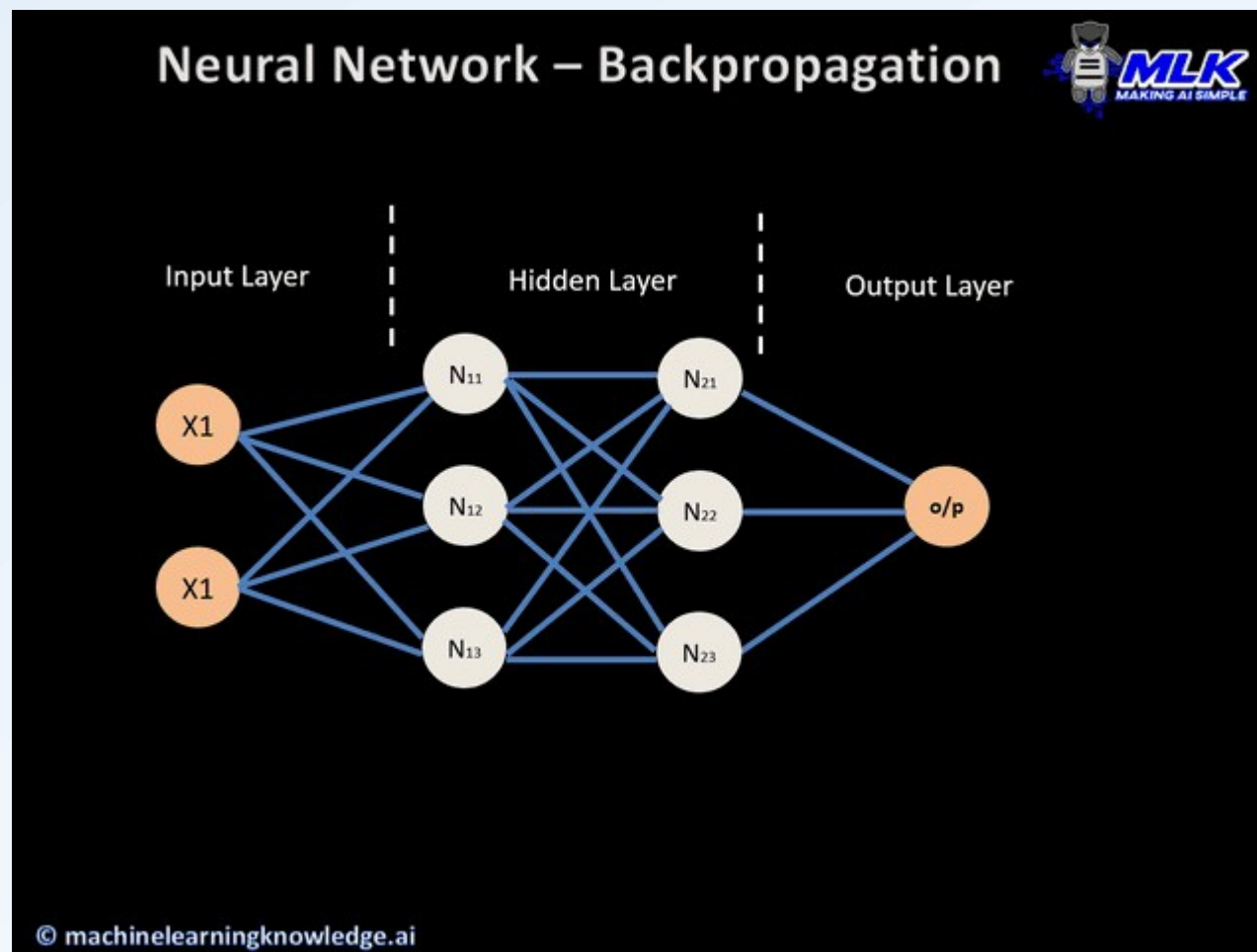
$$a_4 = \text{sigmoid}(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + w_{4b})$$

$$a_5 = \text{sigmoid}(w_{51}x_1 + w_{52}x_2 + w_{53}x_3 + w_{5b})$$

$$a_6 = \text{sigmoid}(w_{61}x_1 + w_{62}x_2 + w_{63}x_3 + w_{6b})$$

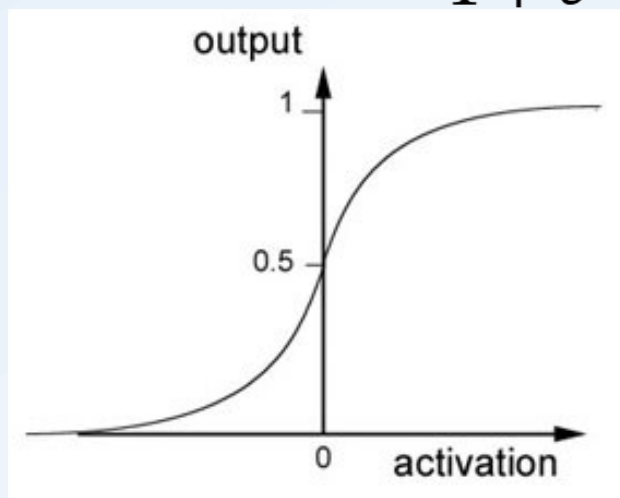
$$a_7 = \text{sigmoid}(w_{71}x_1 + w_{72}x_2 + w_{73}x_3 + w_{7b})$$

◆ Back propagation

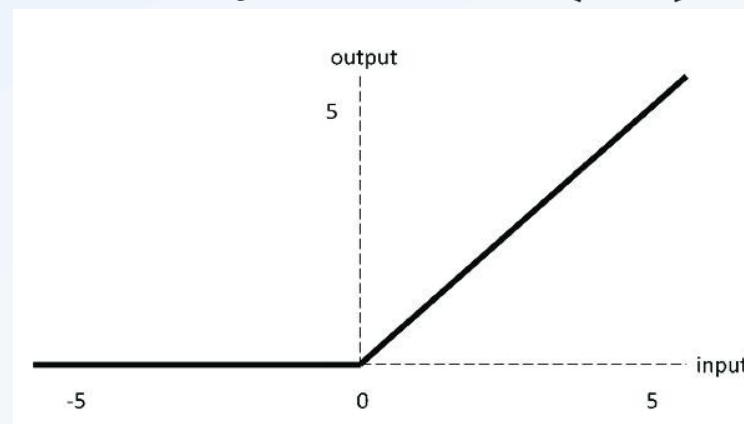


◆ Non-linear function

$$\text{sigmoid } f(z) = \frac{1}{1 + e^{-z}}$$



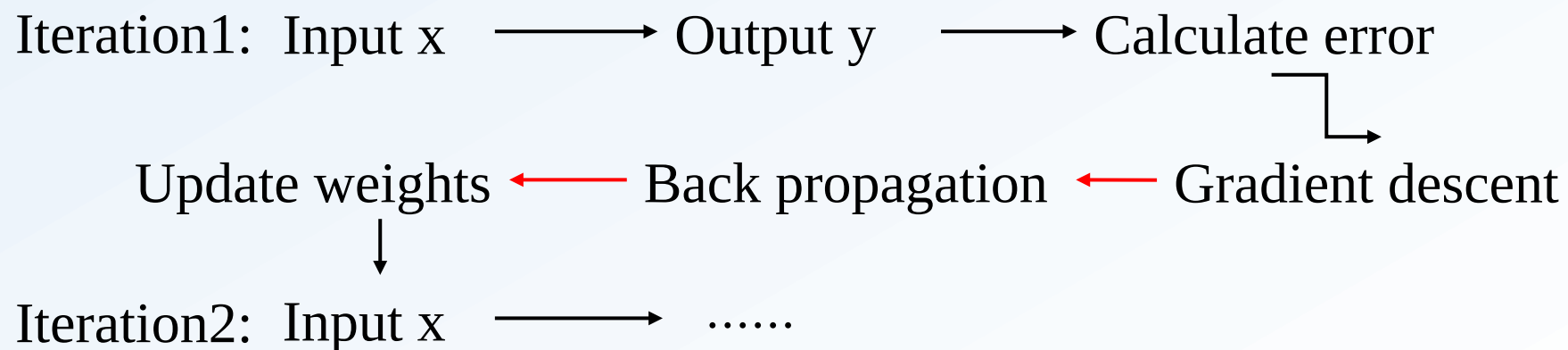
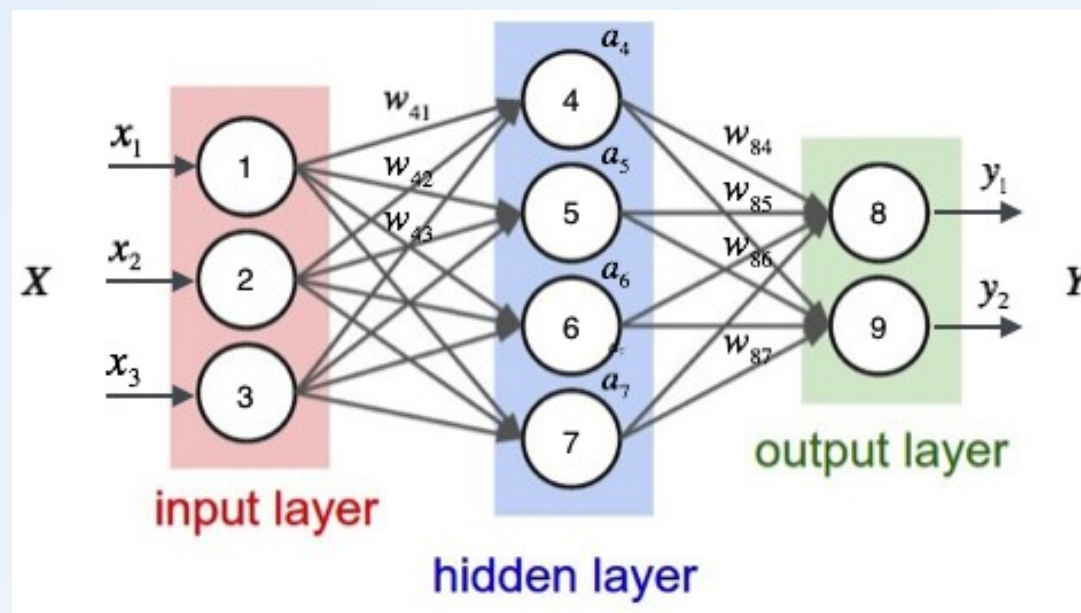
$$\text{ReLU } f(z) = \max(0, z)$$



$$f(z) = z \xrightarrow{\text{Linear to non-linear}} f(z) = \text{activate}(z)$$

Summary

<https://playground.tensorflow.org/>



Contents

1. Why neural network and deep learning?

- Learning map (where we are now)
- Tasks
- Basic knowledge and tips

2. Classic architectures

- CNN
- RNN
- GNN
- Transformer
- Reinforcement learning
- Diffusion

3. Replication code

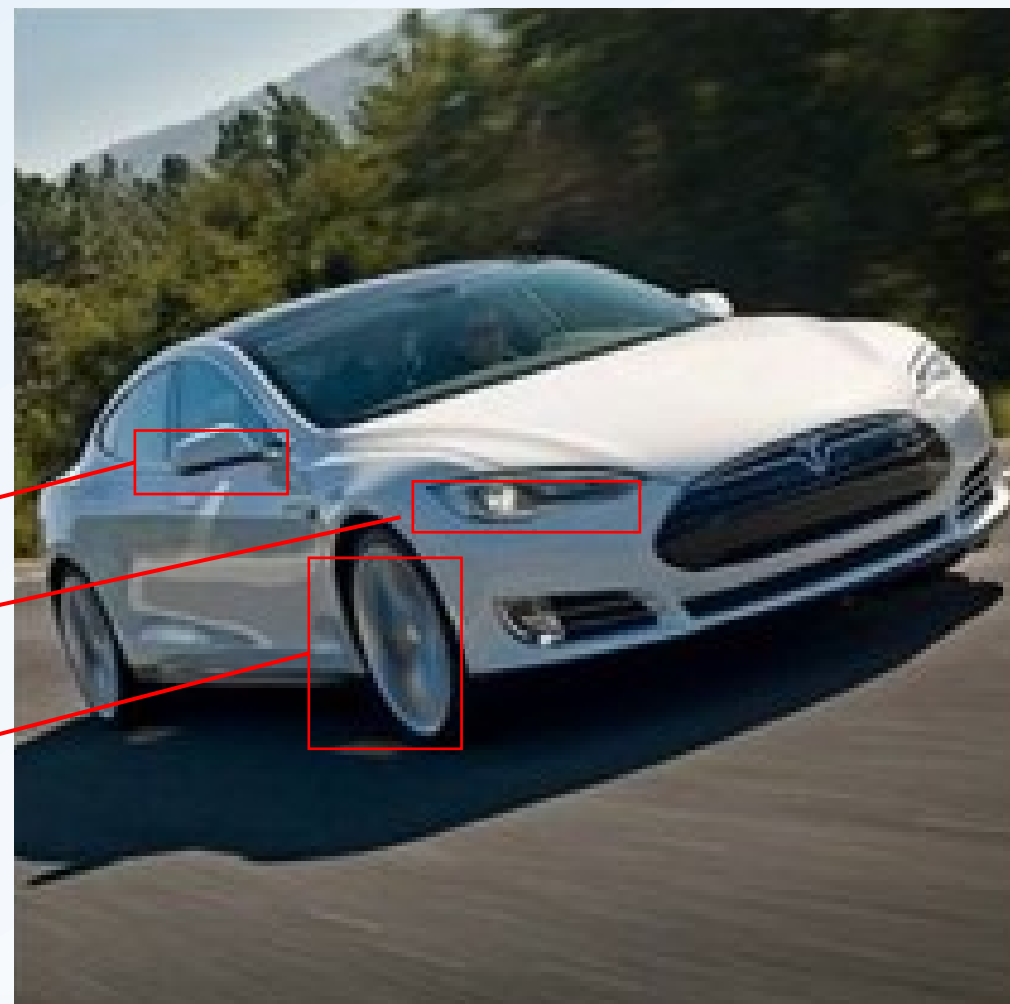
◆ CNN :Convolutional neural networks

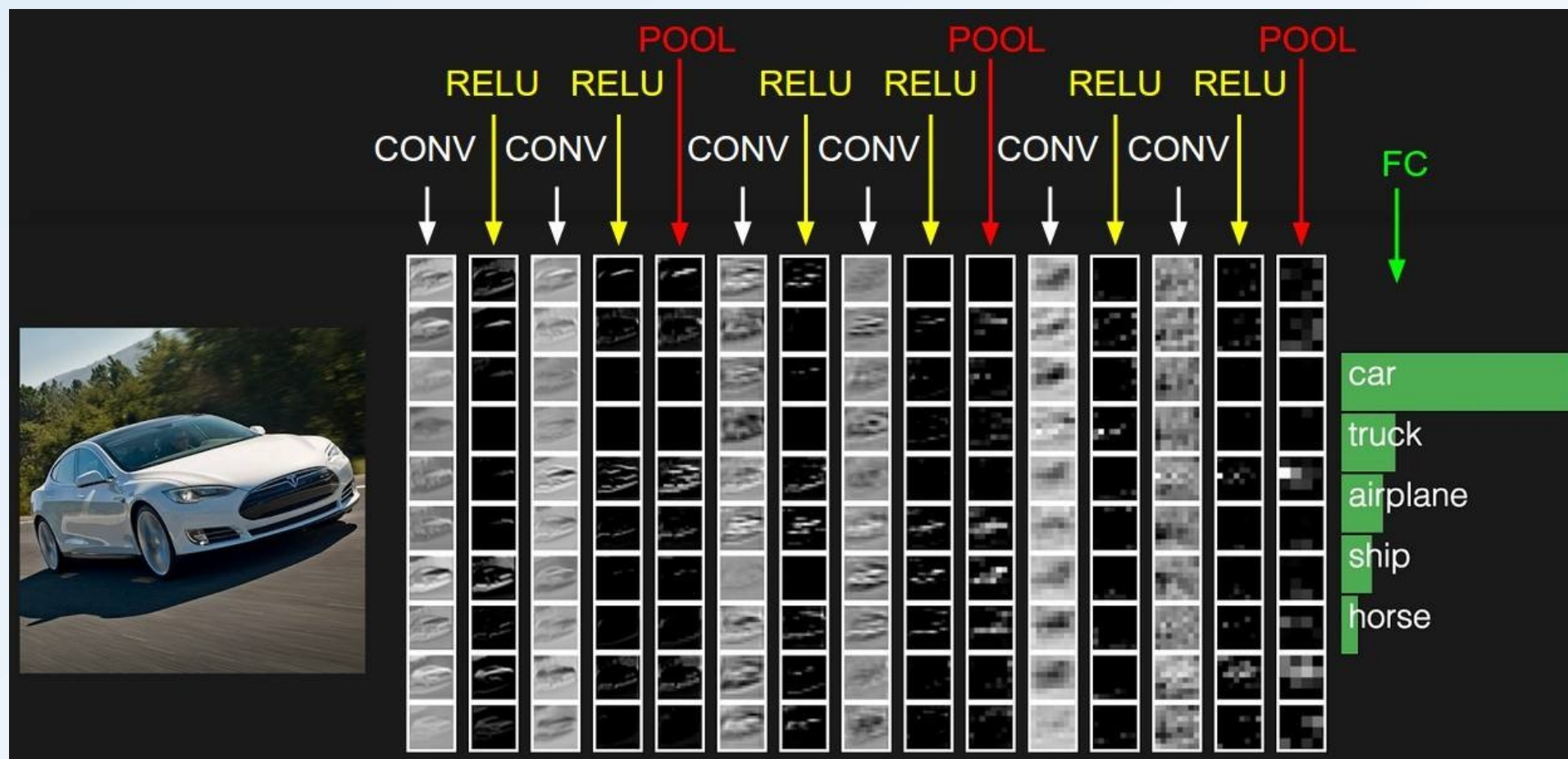
How do we know if this
is a car?

rearview mirror

lighting

Wheels





Extraction

Amplification

Prediction

◆ Convolutional layer

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

image 5*5

1	0	1
0	1	0
1	0	1

bias=0

filter 3*3

feature map 3*3

X

W

Y

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

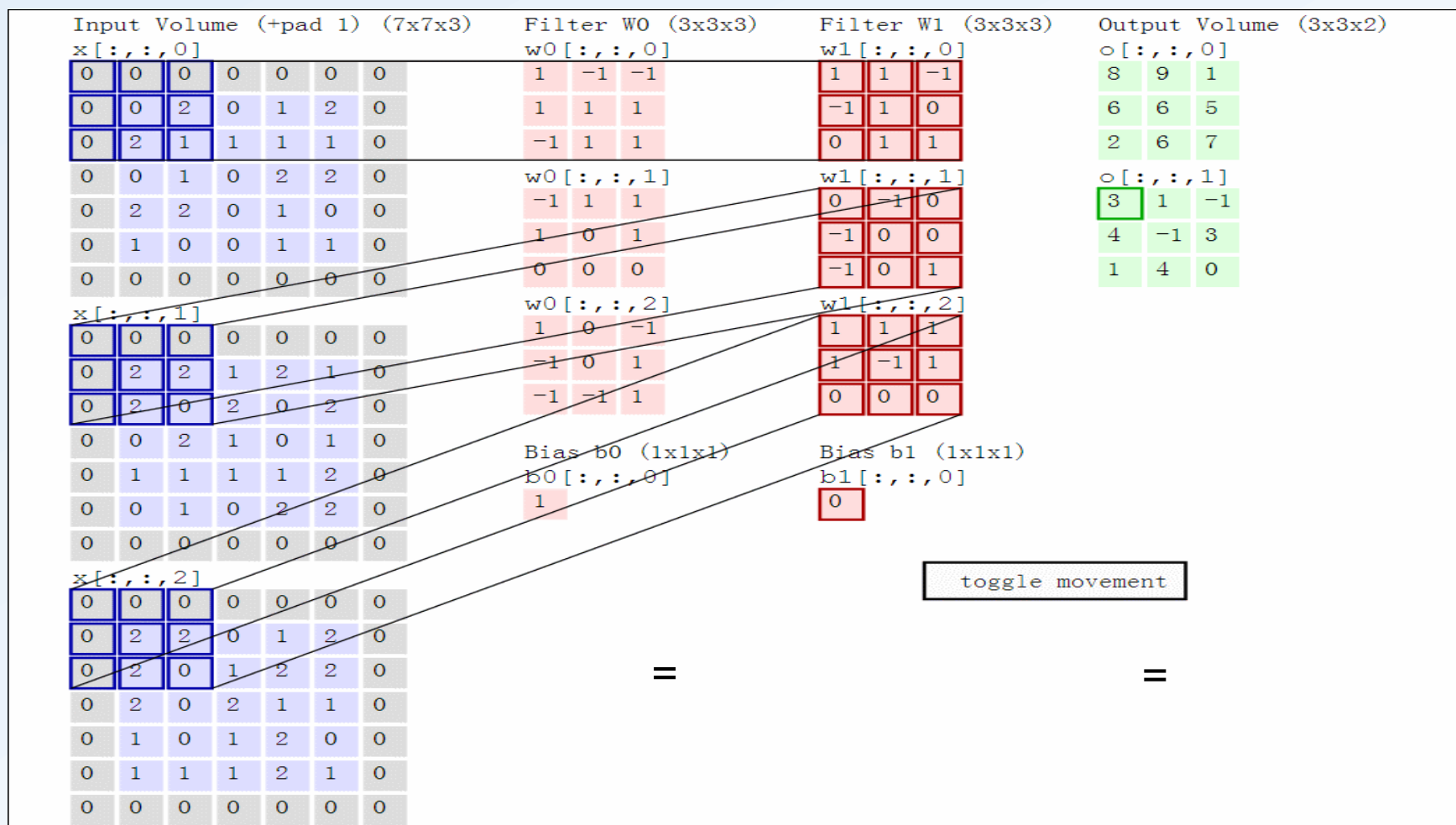
Image

4		

Convolved
Feature

$$a_{i,j} = f(\sum_{m=0}^2 \sum_{n=0}^2 w_{m,n} x_{i+m,j+n} + w_b)$$

◆ Multiple convolution kernels



◆ Pooling layer

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

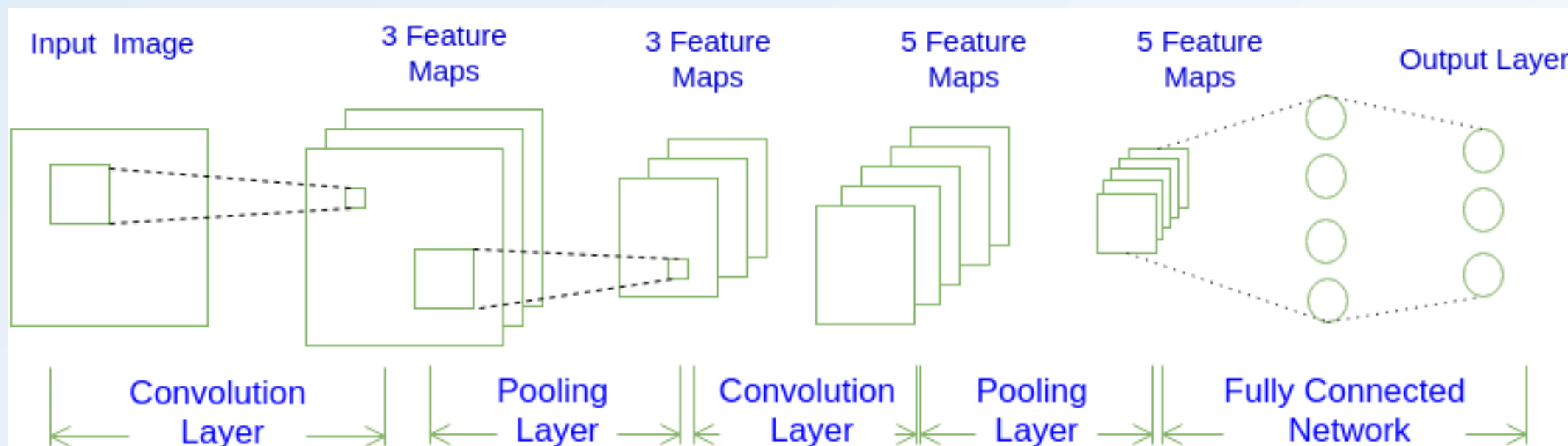
Max
Pooling

Average
Pooling

Down-sampling

Remove the unimportant samples in the feature maps

◆ Convolutional neural networks



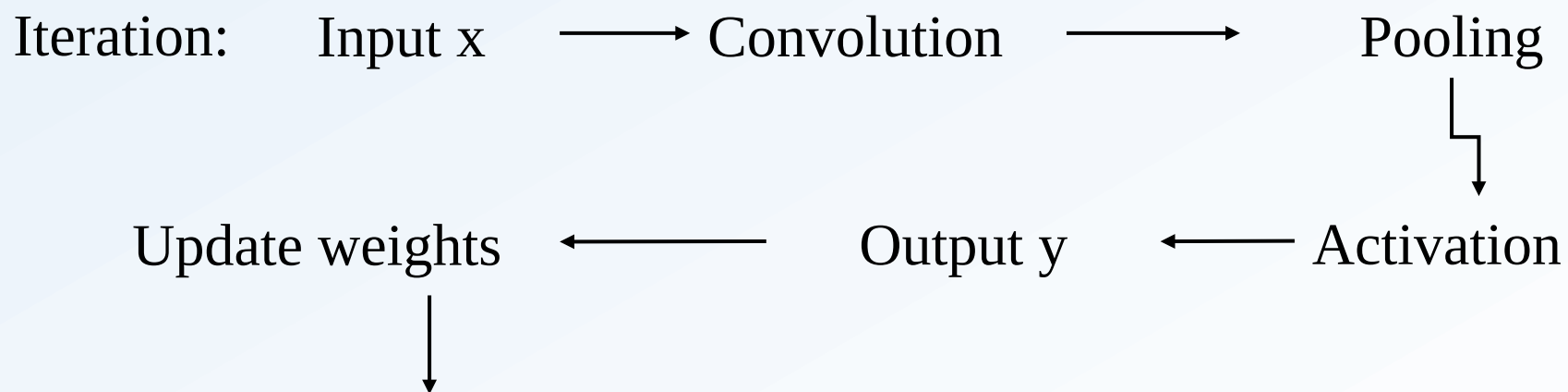
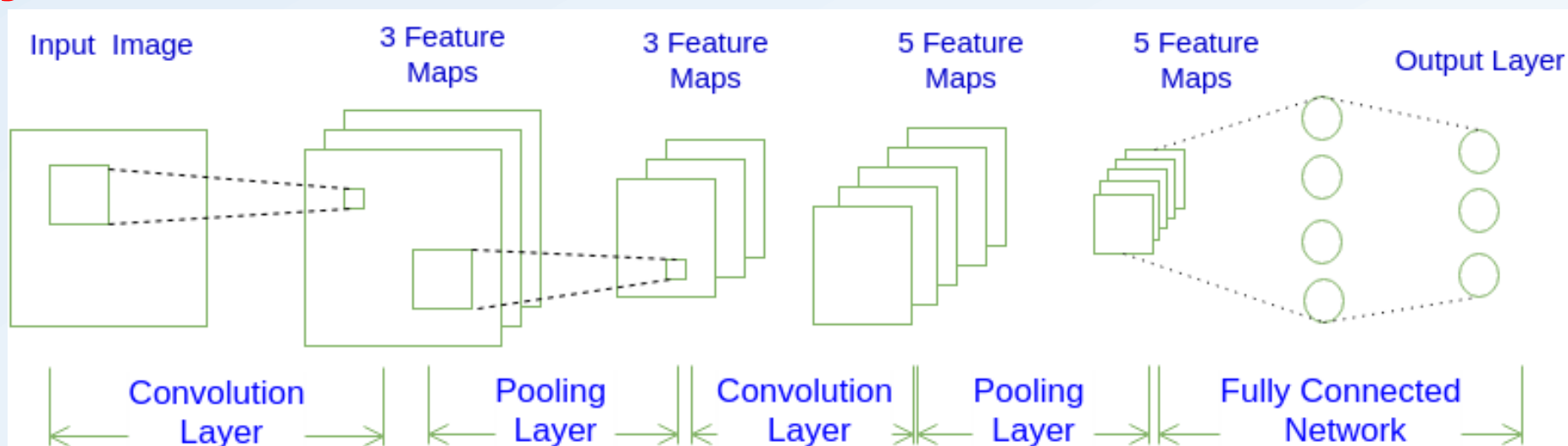
Extraction

Amplification

Prediction

Local connection Sharing Weights Down-sampling

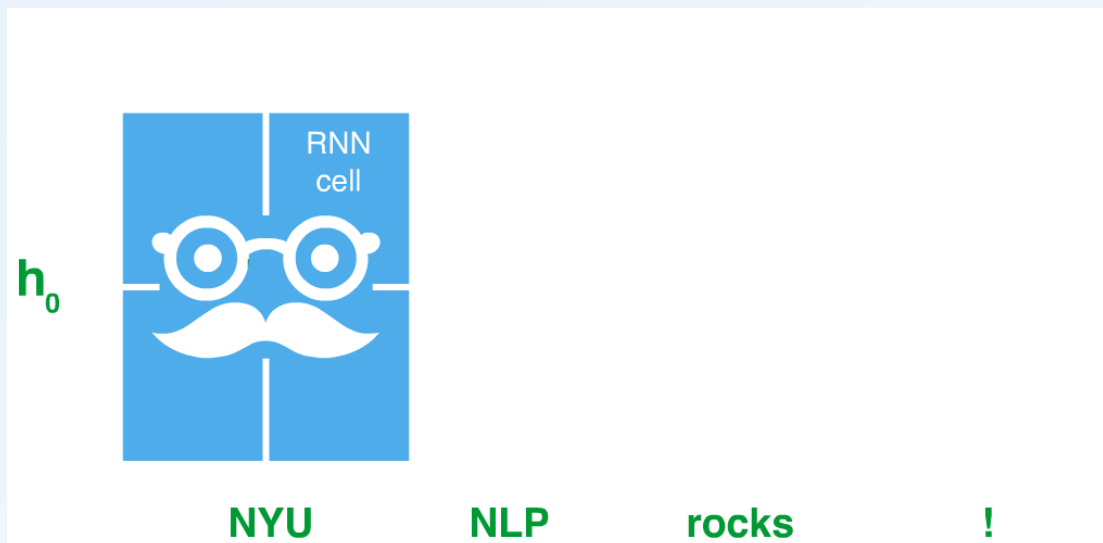
Summary



◆ RNN: Recurrent Neural Network

"I voted for Nader because he was most aligned with my values," she said.

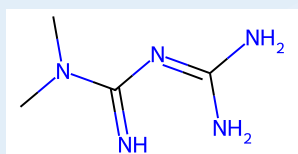
What is the gender of Nader?



$$h_t = f(w_h h_{t-1} + w_x x_t)$$

Prior knowledge: seriality

◆ RNN: Recurrent Neural Network



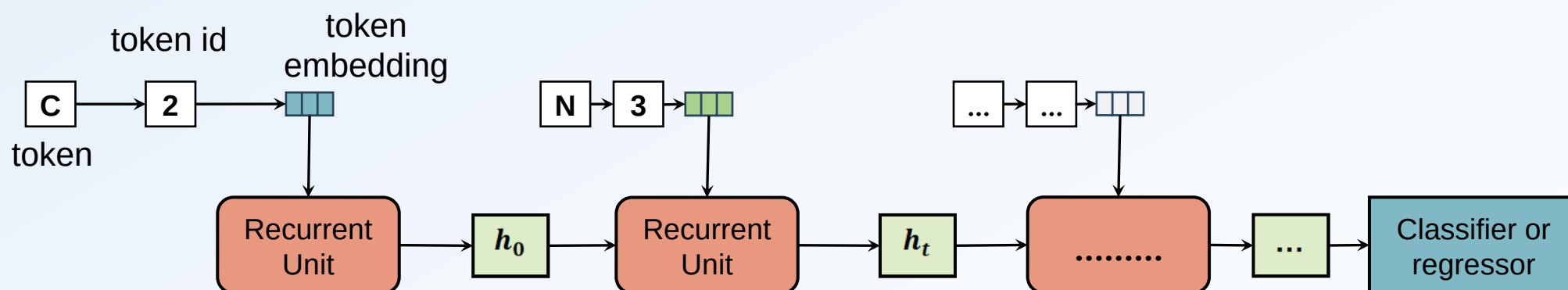
Input molecule

CN(C)C(=N)N=C(N)N $\rightarrow$ C $\rightarrow$ N $\rightarrow$ ($\rightarrow$ C $\rightarrow$) $\rightarrow$ C $\rightarrow$ ($\rightarrow$ = $\rightarrow$ N $\rightarrow$) $\rightarrow$ N $\rightarrow$ = $\rightarrow$ C $\rightarrow$ ($\rightarrow$ N $\rightarrow$) $\rightarrow$ N ① **split**

SMILES string

$\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 2 $\rightarrow$ 5 $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 6 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 3 $\rightarrow$ 6 $\rightarrow$ 2 $\rightarrow$ 4 $\rightarrow$ 3 $\rightarrow$ 5 $\rightarrow$ 3 ② **convert**

Token	ID	Token embedding
<BOS>	0	
<EOS>	1	
C	2	
N	3	
(	4	
)	5	
=	6	

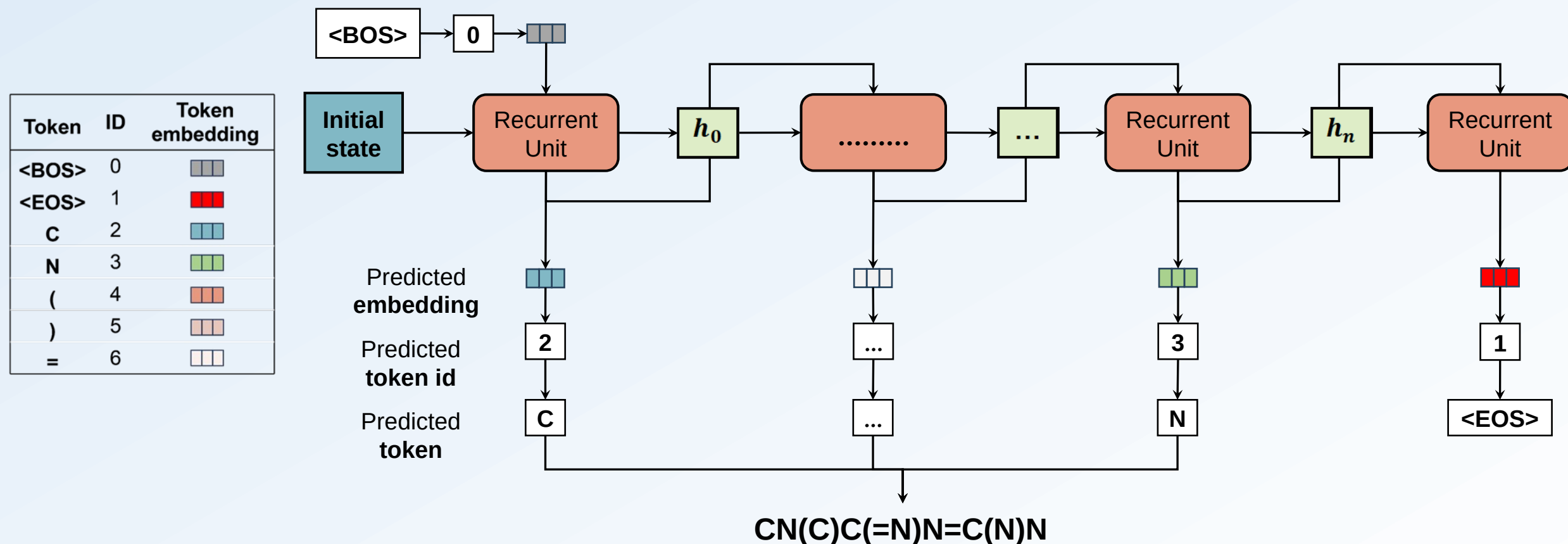


$$h_t = \sigma(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + b_h)$$

RNN can maintain a hidden memory and capture the **sequential features**



RNN as decoder

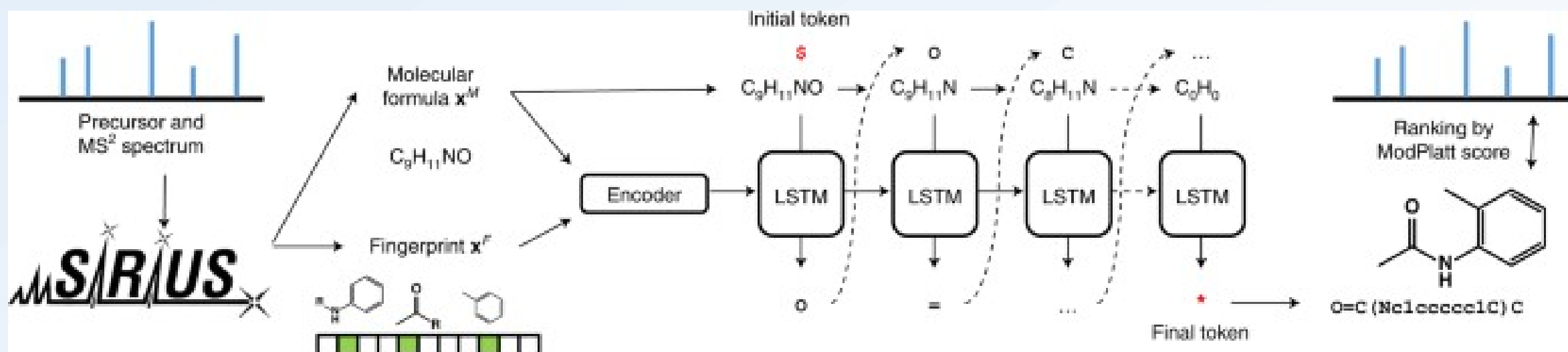


$$h_t = \sigma(W_x x_t + W_h h_{t-1} + b_h)$$

$$\hat{y}_t = \text{softmax}(W_y h_t + b_y)$$

RNN is used to **predict the token** by
captured sequential features

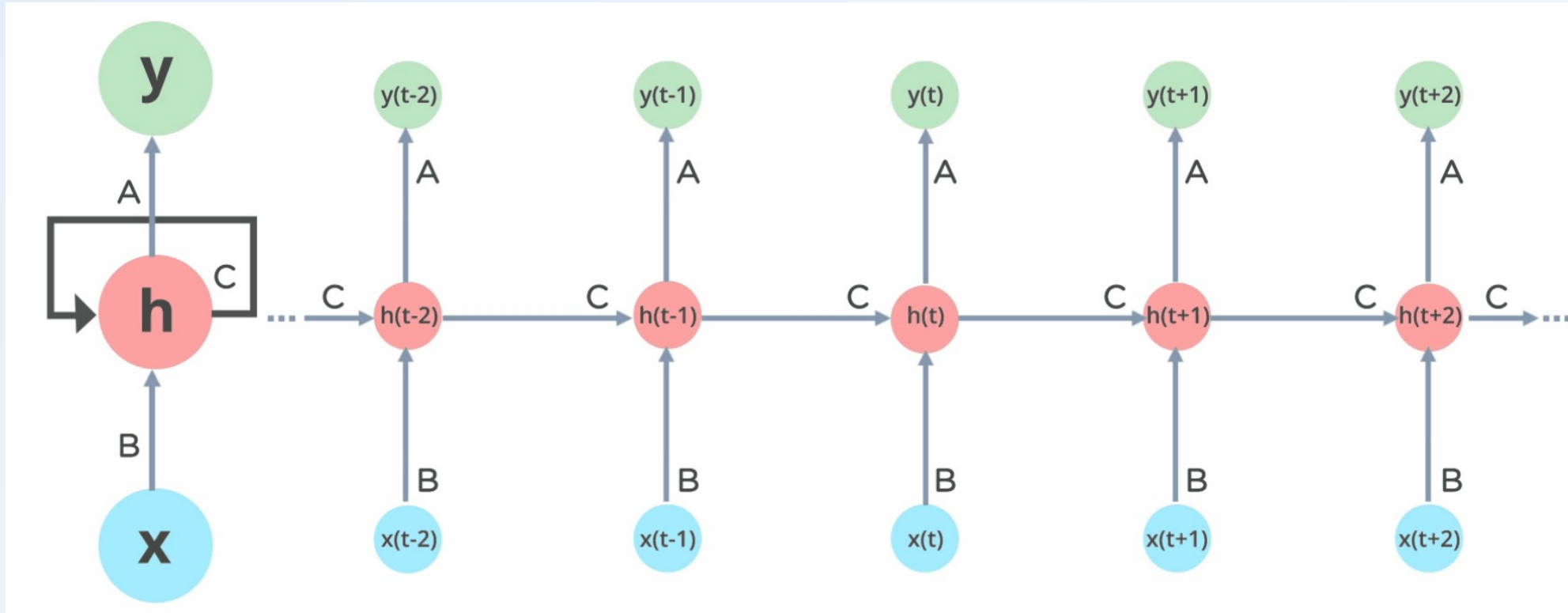
◆ RNN: Recurrent Neural Network



Nat Methods 2022, 19, 865–870

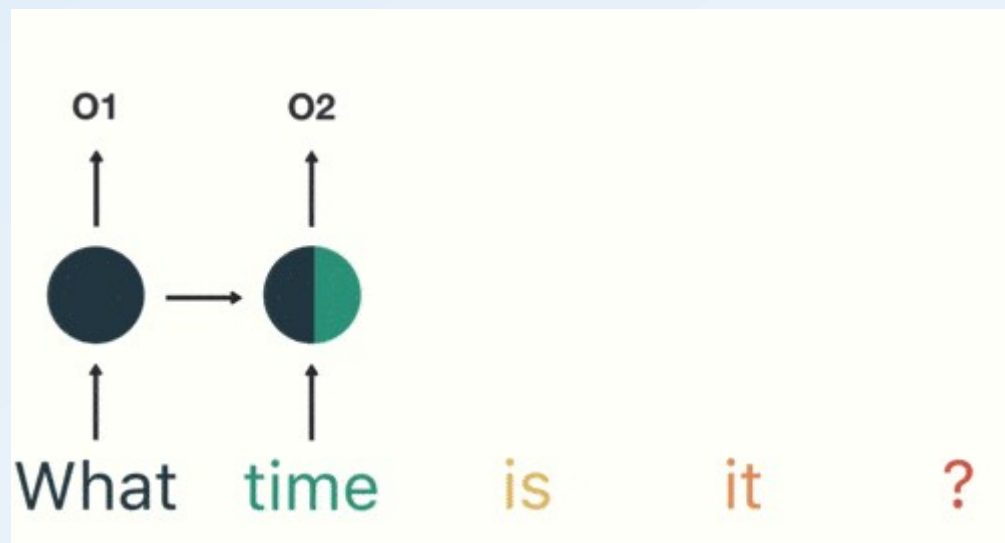
Limitation

◆ RNN: Recurrent Neural Network



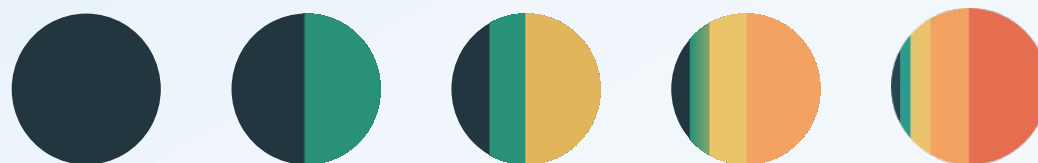
RNN is a sequential network —
 Inefficiency computation
 Memories may be forgotten

◆ Limitation of RNN



Problems:

The longer the sentences,
The worse the performance

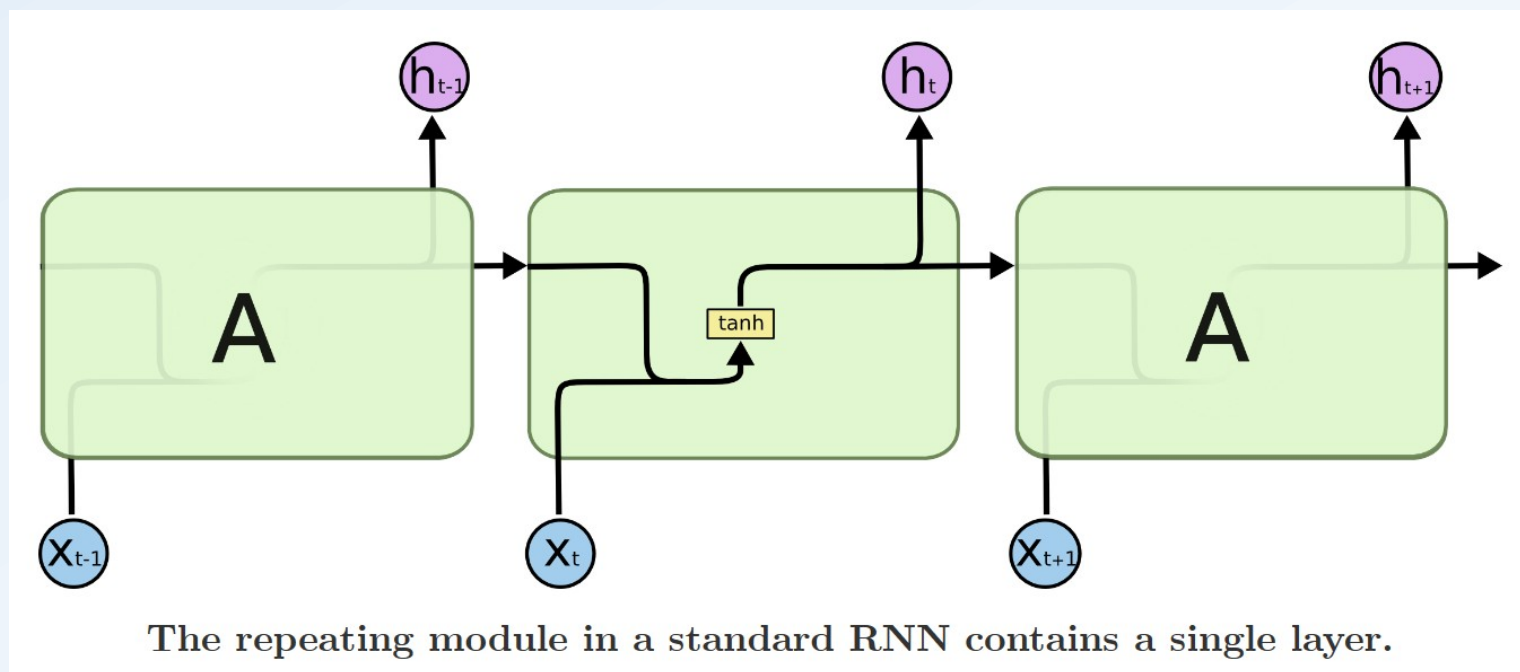


Memory is short

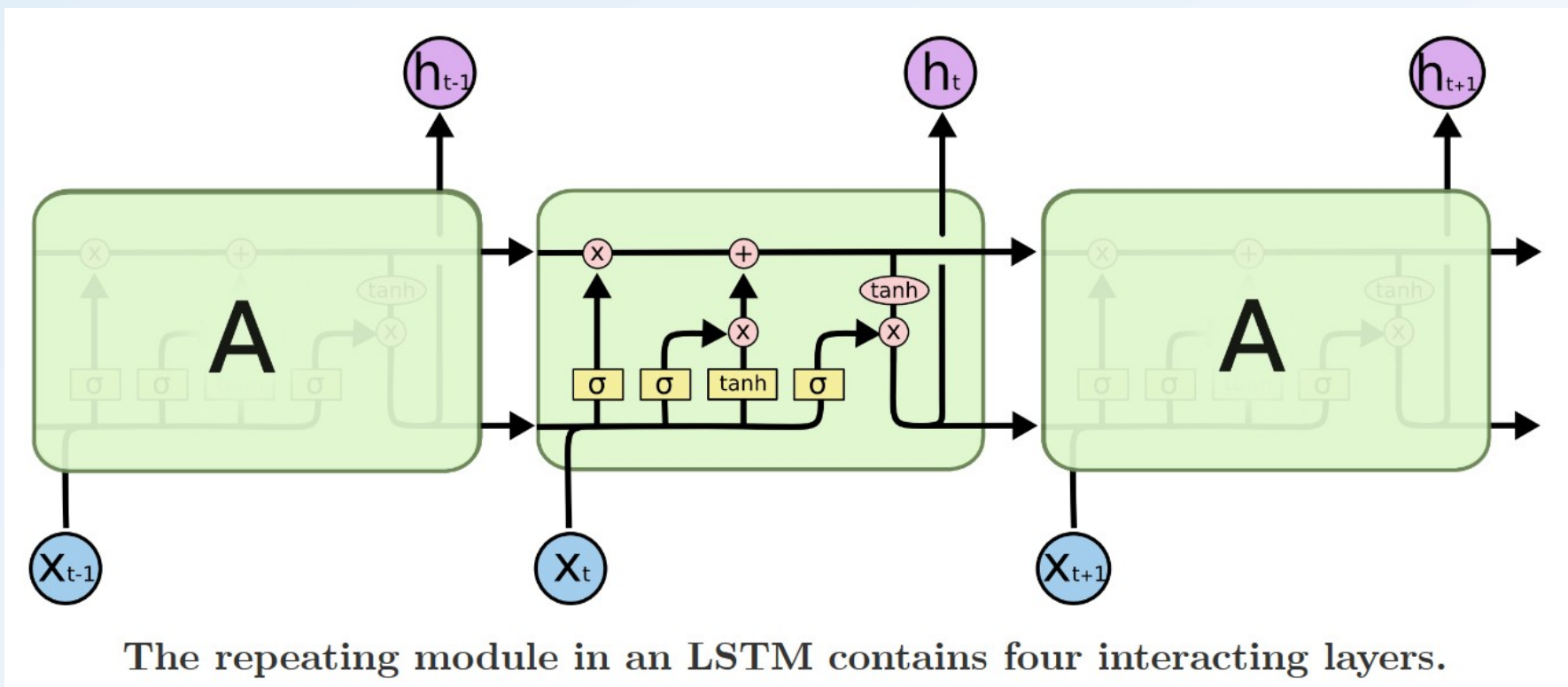
$$h_t = f(w_h h_{t-1} + w_x x_t)$$

As sentence get longer, there
will be less information about
previous words in h_{t-1}

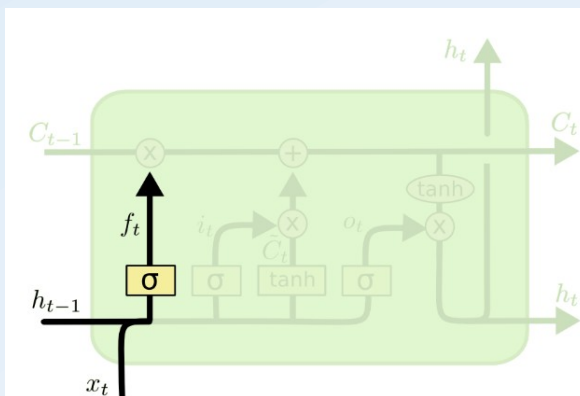
◆ RNN: Recurrent Neural Network



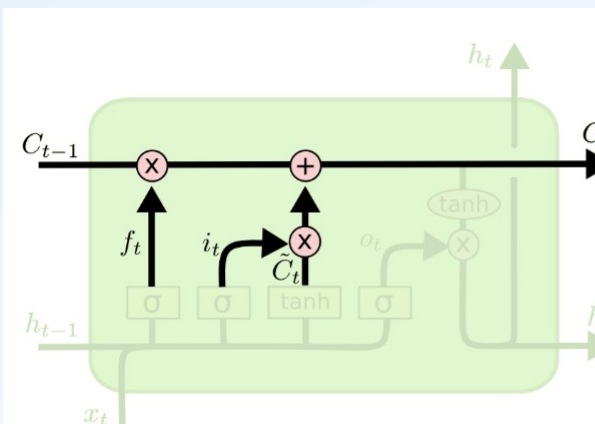
◆ RNN: Recurrent Neural Network



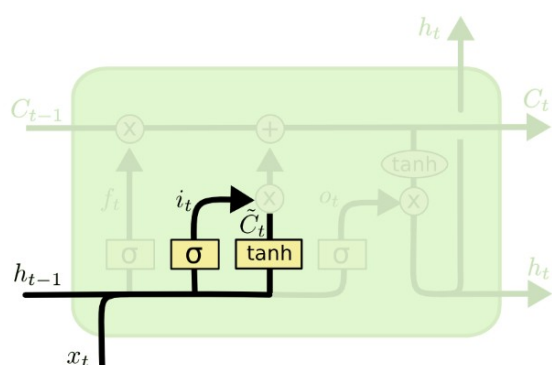
◆ RNN: Recurrent Neural Network



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

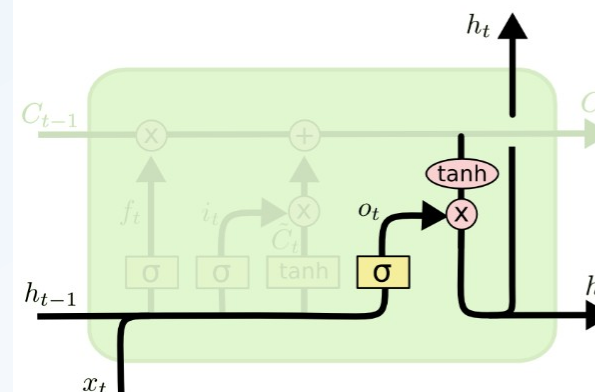


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

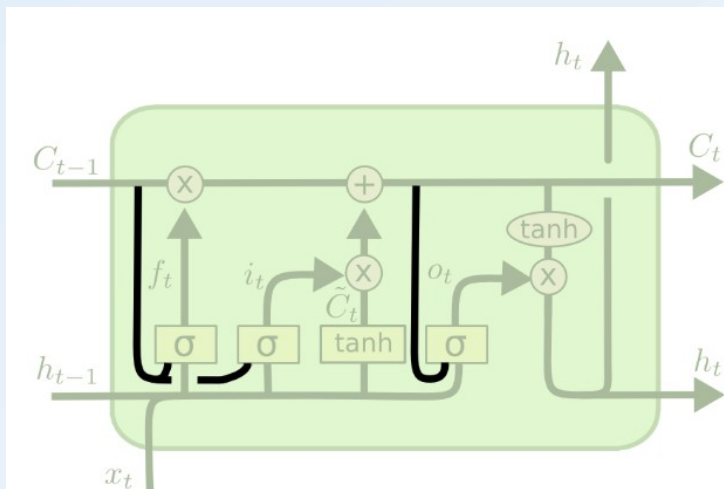
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

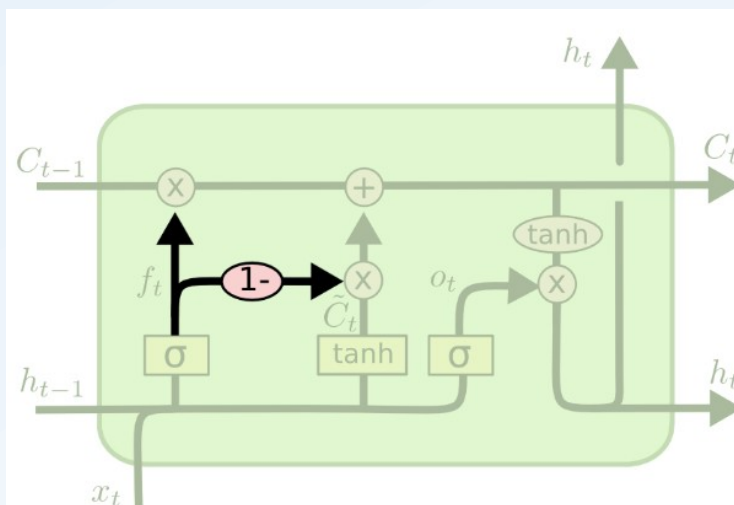
◆ RNN: Recurrent Neural Network



$$f_t = \sigma (W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

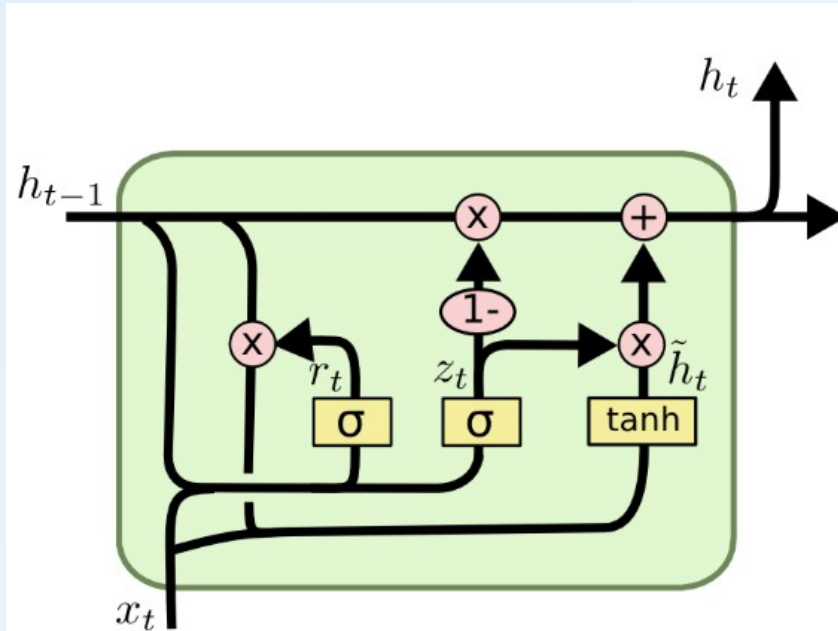
$$i_t = \sigma (W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma (W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

◆ RNN: Recurrent Neural Network



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

◆ Limitation of RNN

The woman brought two glasses of beer and two felt pads. She put the felt pads and the beer glasses on the table and looked at the man and the girl. The girl was looking off at the line of hills. They were white in the sun and the **country** was brown and dry.

RNN

Are there any models without this inductive bias?

Hemingway, *Hills Like White Elephants*

Attention is all you need

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Łukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* ‡ illia.polosukhin@gmail.com			

Microsoft
Research
微软亚洲研究院

Deep residual learning for image recognition

K He, X Zhang, S Ren, J Sun - Proceedings of the IEEE ..., 2016 - openaccess.thecvf.com

... Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. ...

☆ 保存 引用 被引用次数: 211374 相关文章 所有 73 个版本

Attention is all you need

A Vaswani, N Shazeer, N Parmar... - Advances in neural ..., 2017 - proceedings.neurips.cc

... to attend to all positions in the decoder up to and including that position. We need to prevent ... We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) ...

☆ 保存 引用 被引用次数: 115010 相关文章 所有 87 个版本

Google

科学智能

微软研究院团队获得首届AI药物研发
算法大赛总冠军

AI2BMD & ViSNet

Distributional Graphormer: 从分子
结构预测到平衡分布预测



科学智能 (AI4Science) 赋能科学发
现的第五范式



Academia

0→1 insights

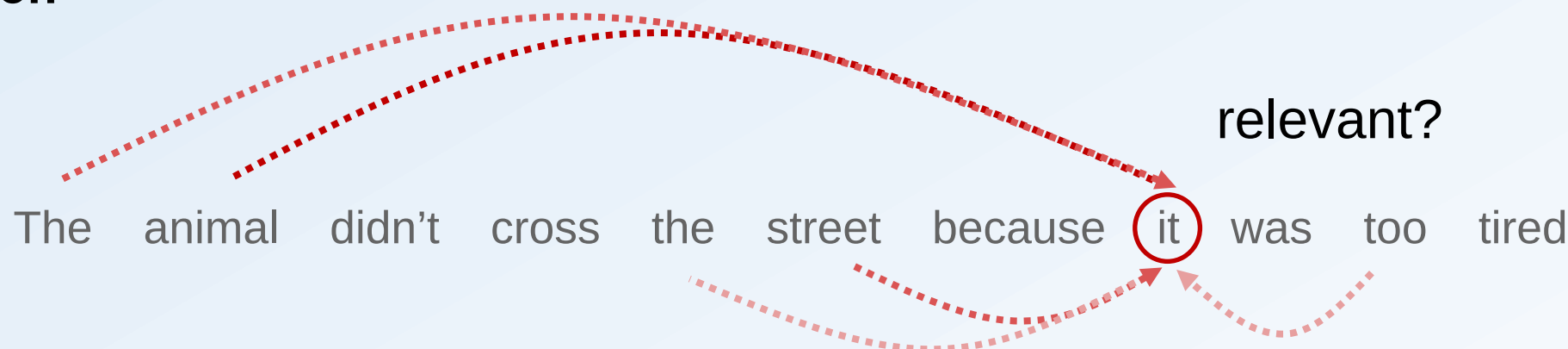
1→100 techniques



Industry

◆ Attention

Machine Translation



Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

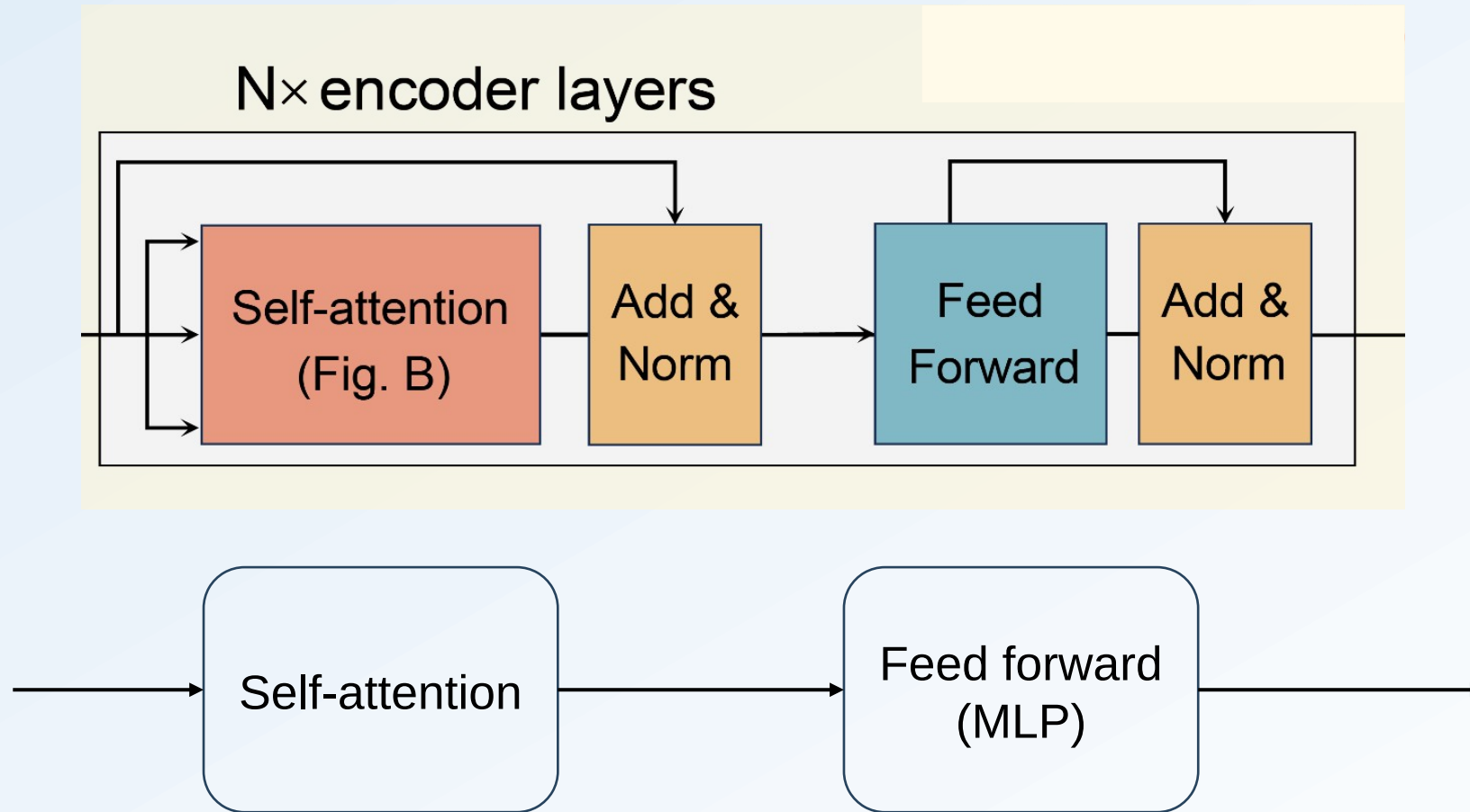
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

No matter **how long** the sentence is
attention mechanism can focus on **global**
information and find the **most critical**
information.

Transformer in computer vision

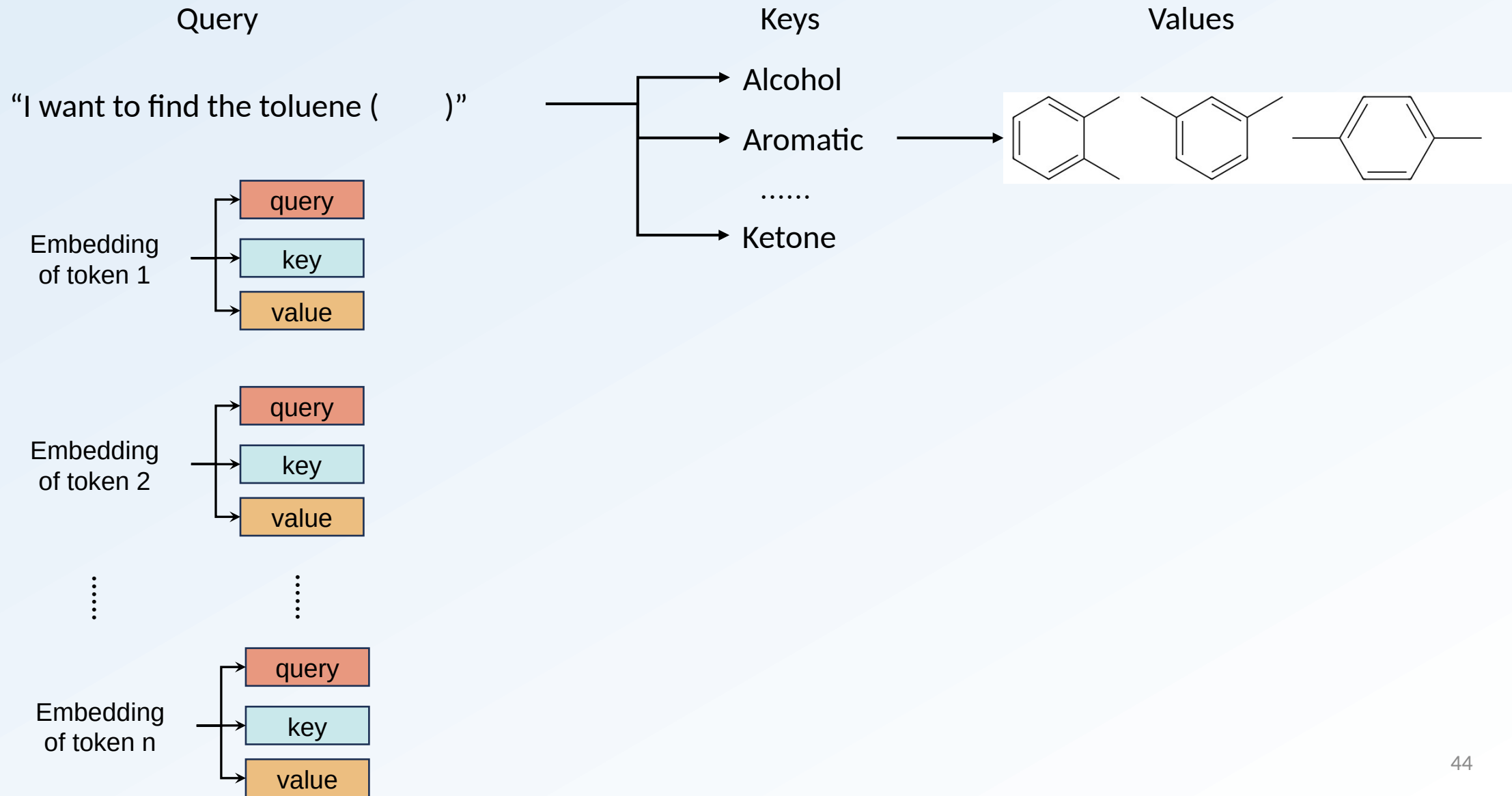


Feature extraction



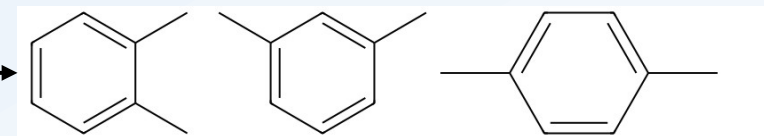
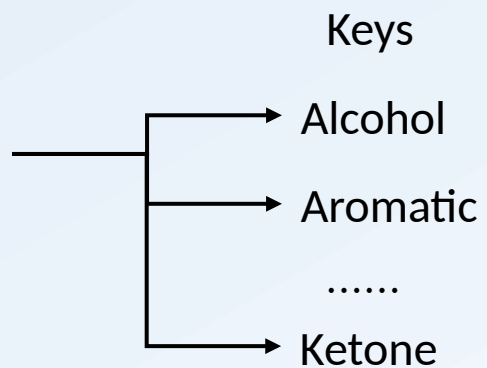
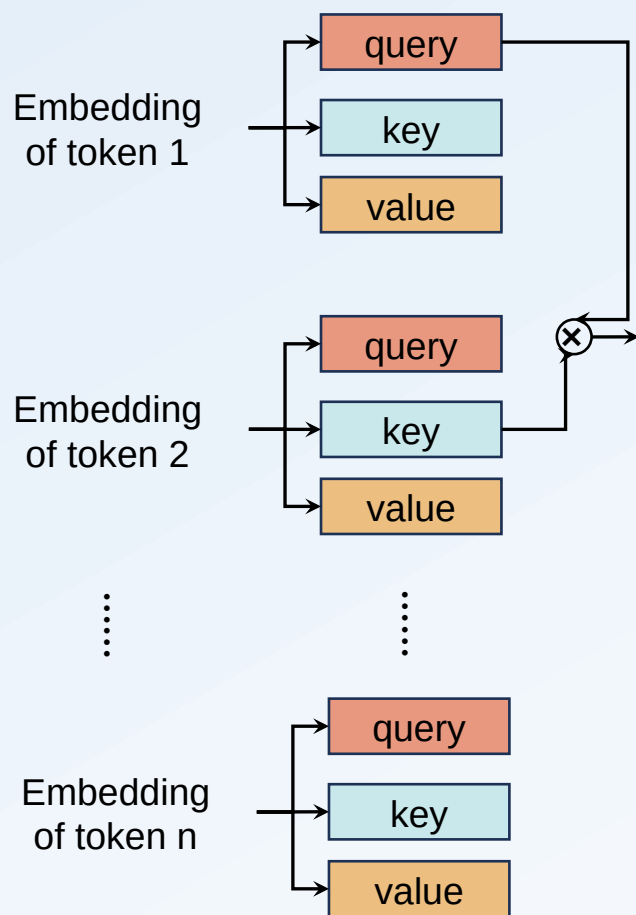
Self-attention plays the important role in encoding

Self-attention



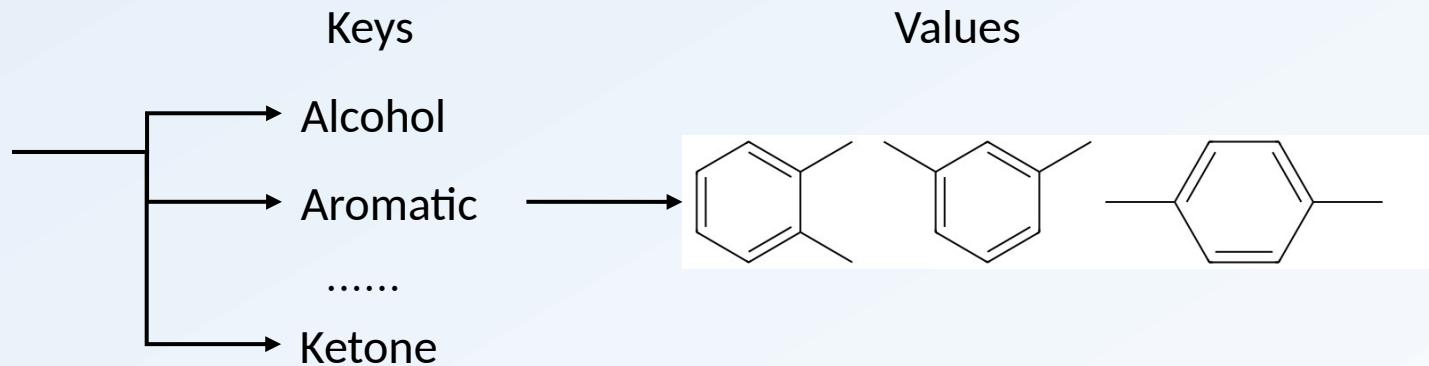
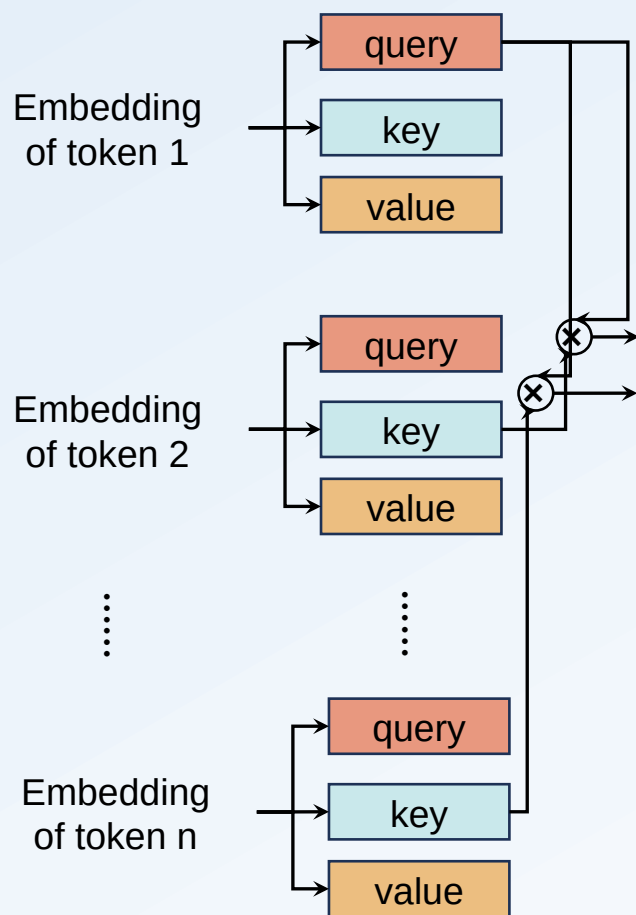
Self-attention

Query
“I want to find the toluene ()”

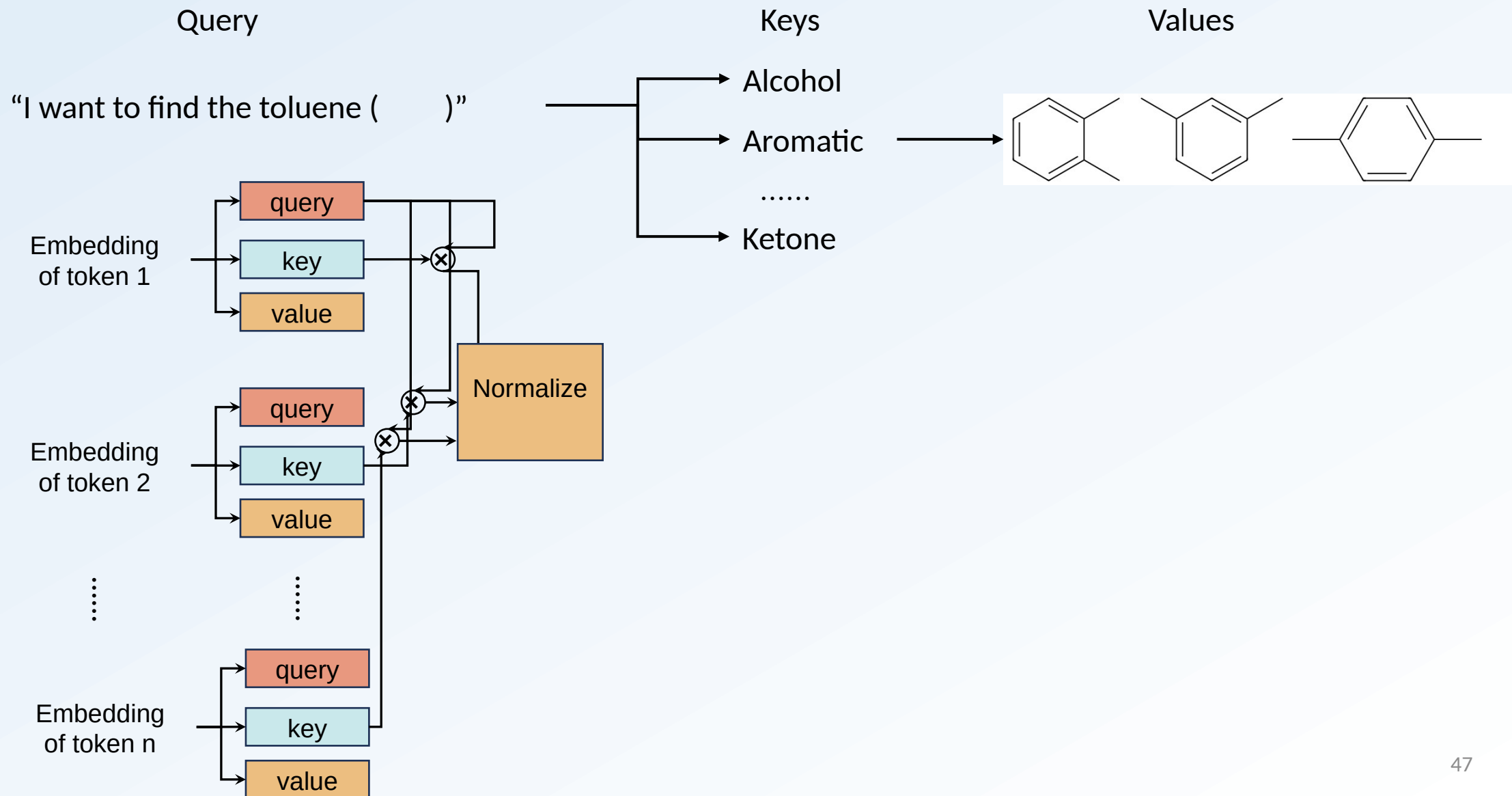


Self-attention

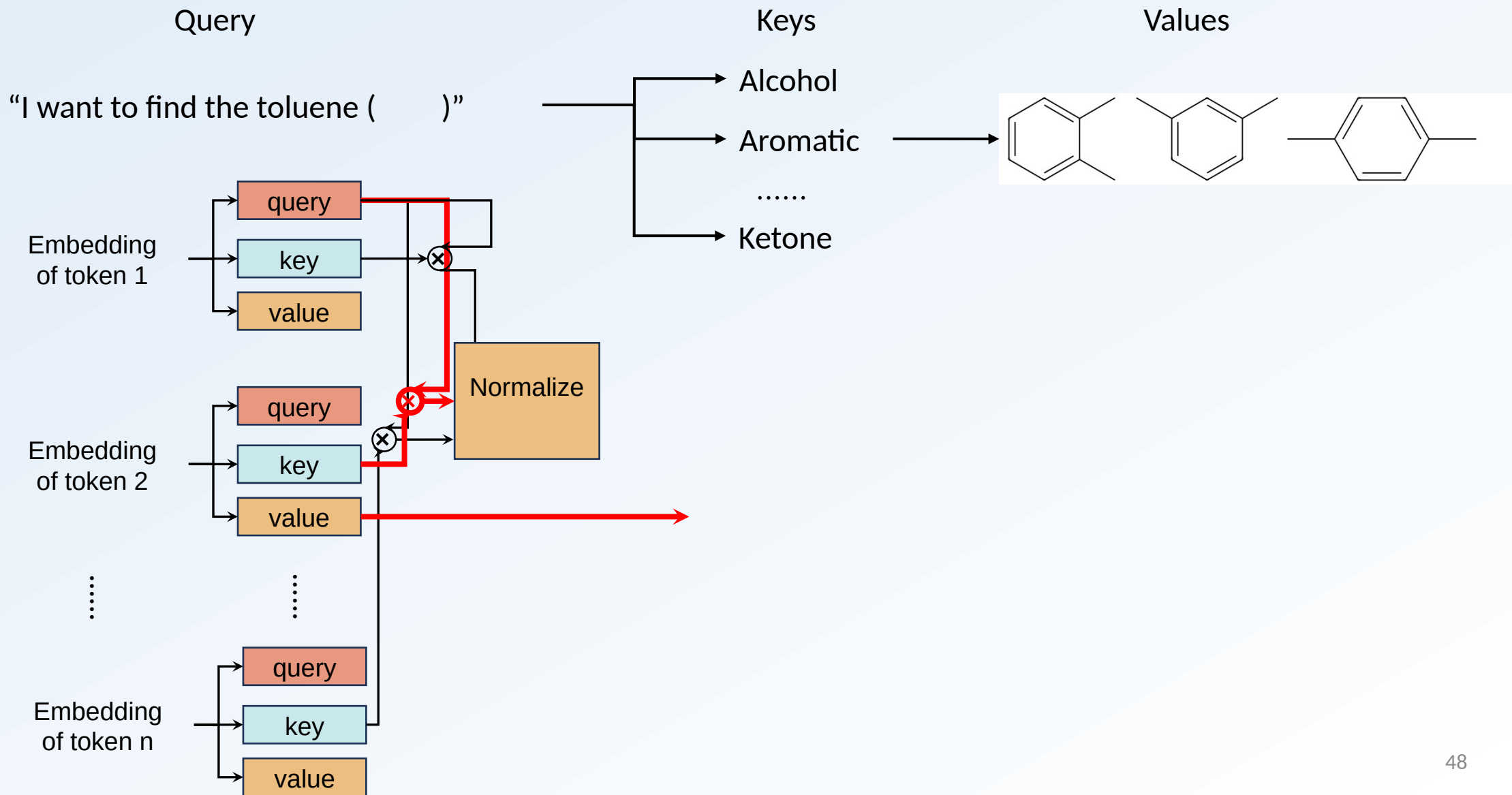
Query
“I want to find the toluene ()”



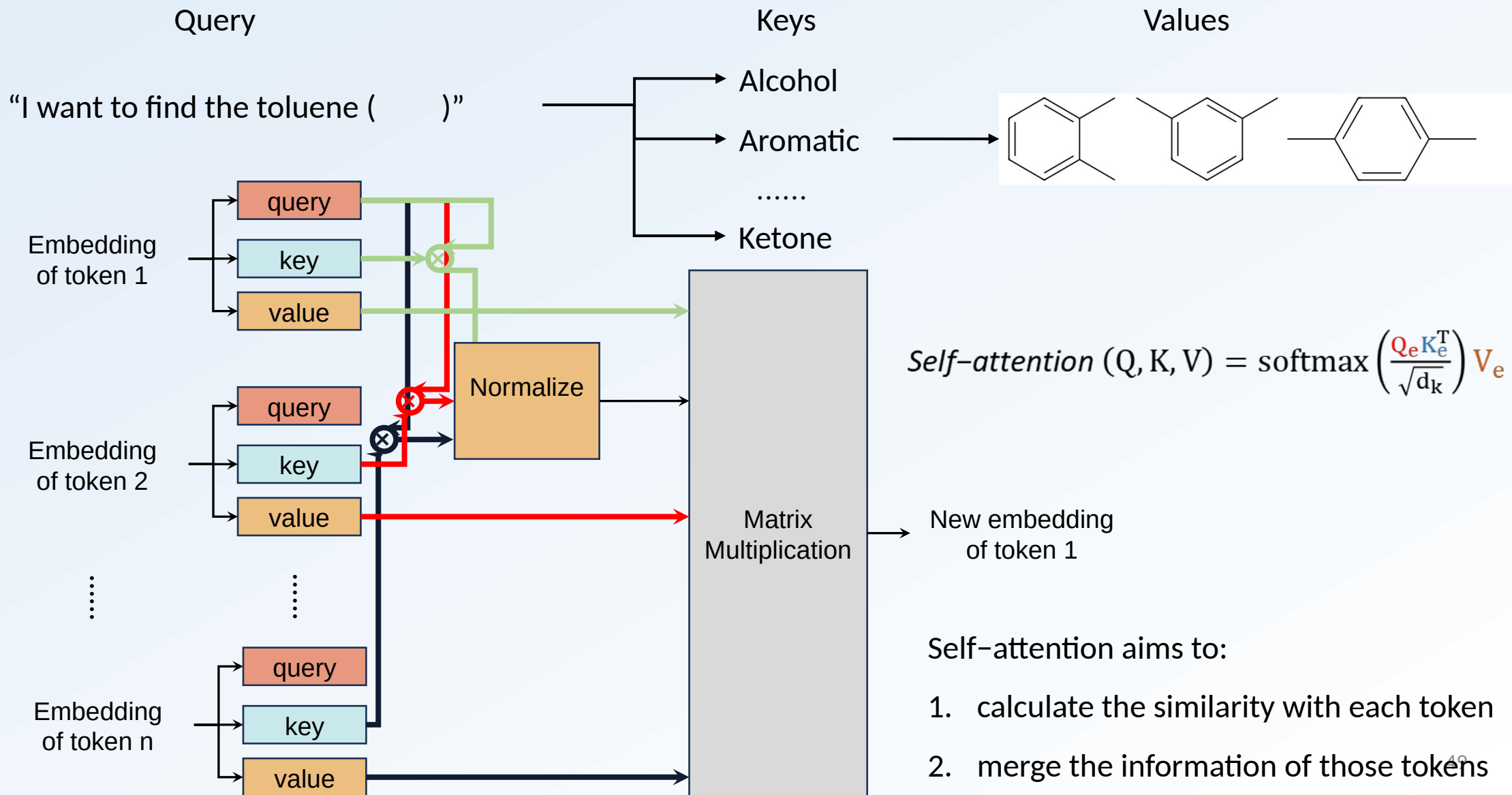
Self-attention

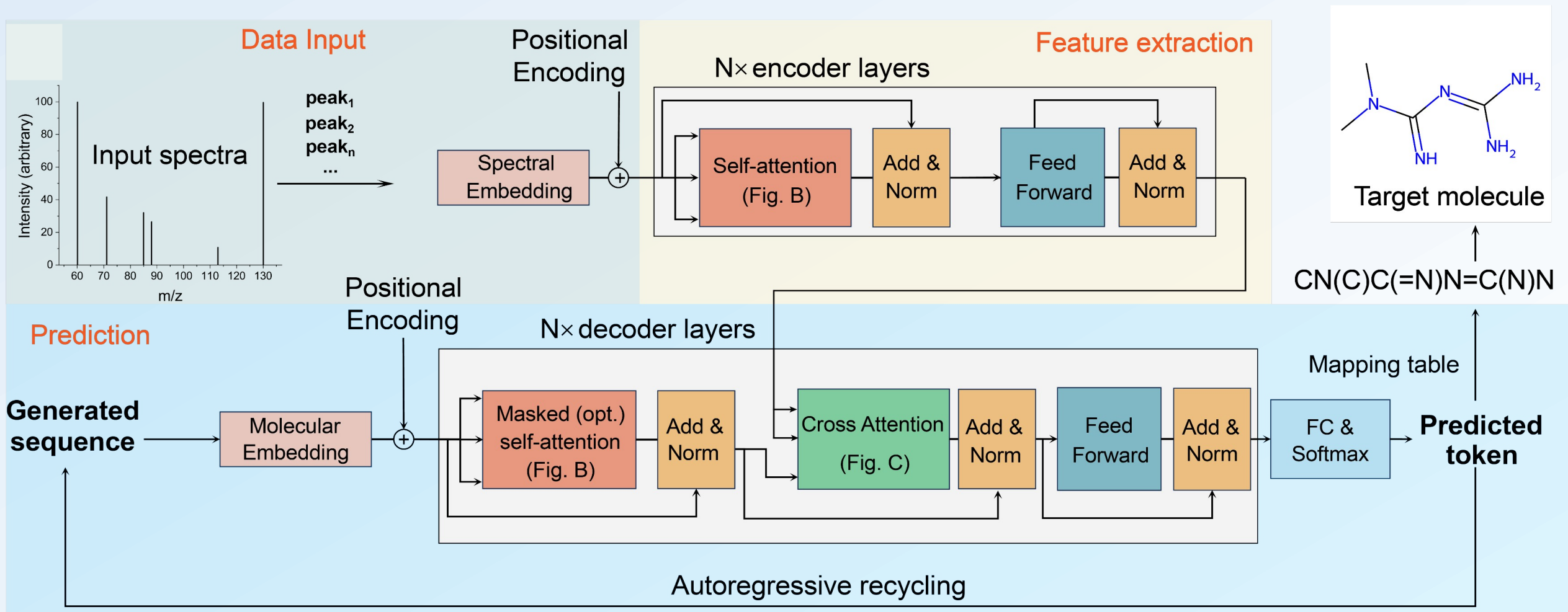


Self-attention

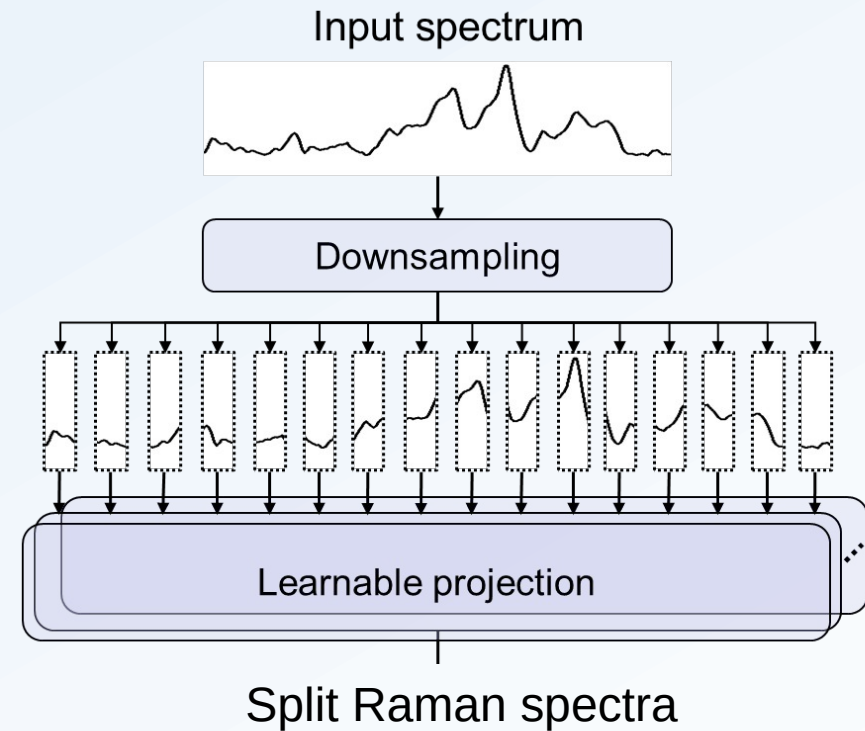
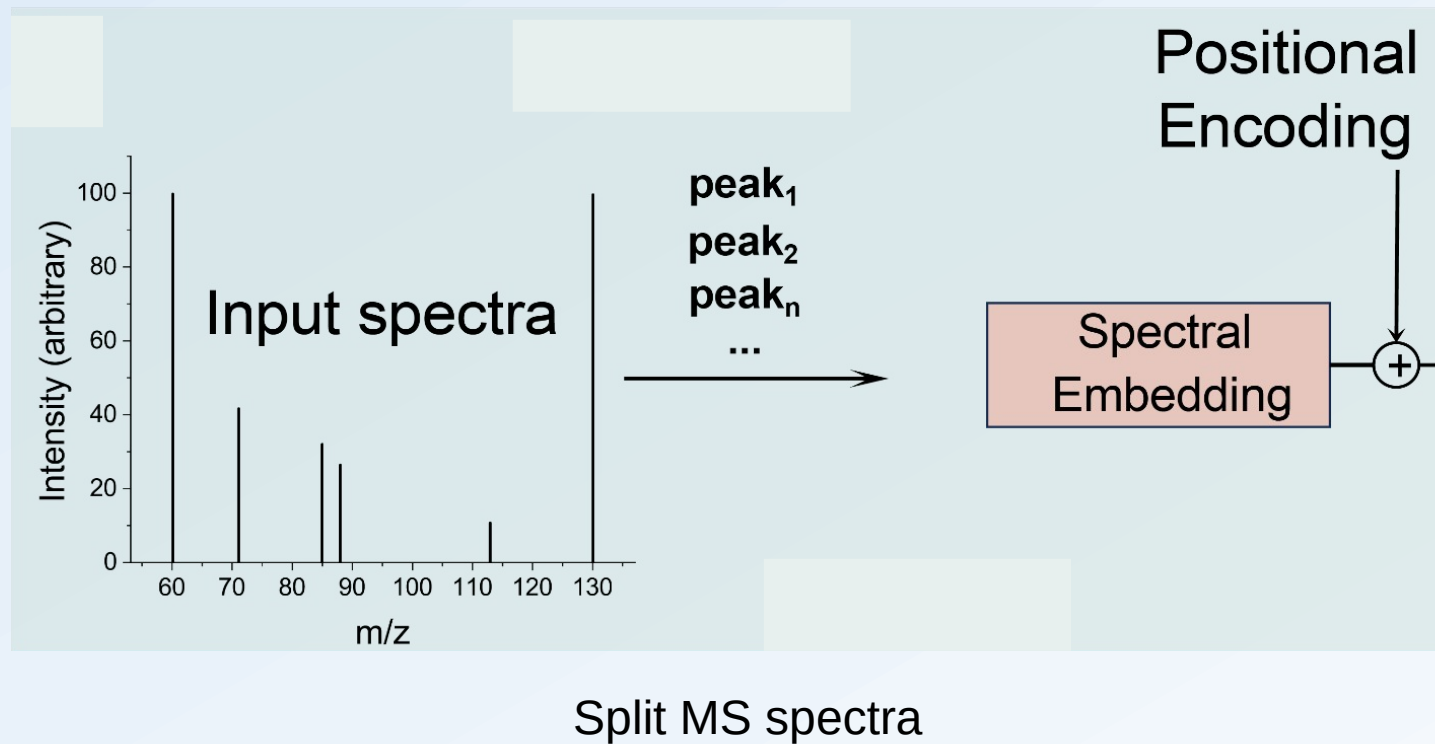


Self-attention





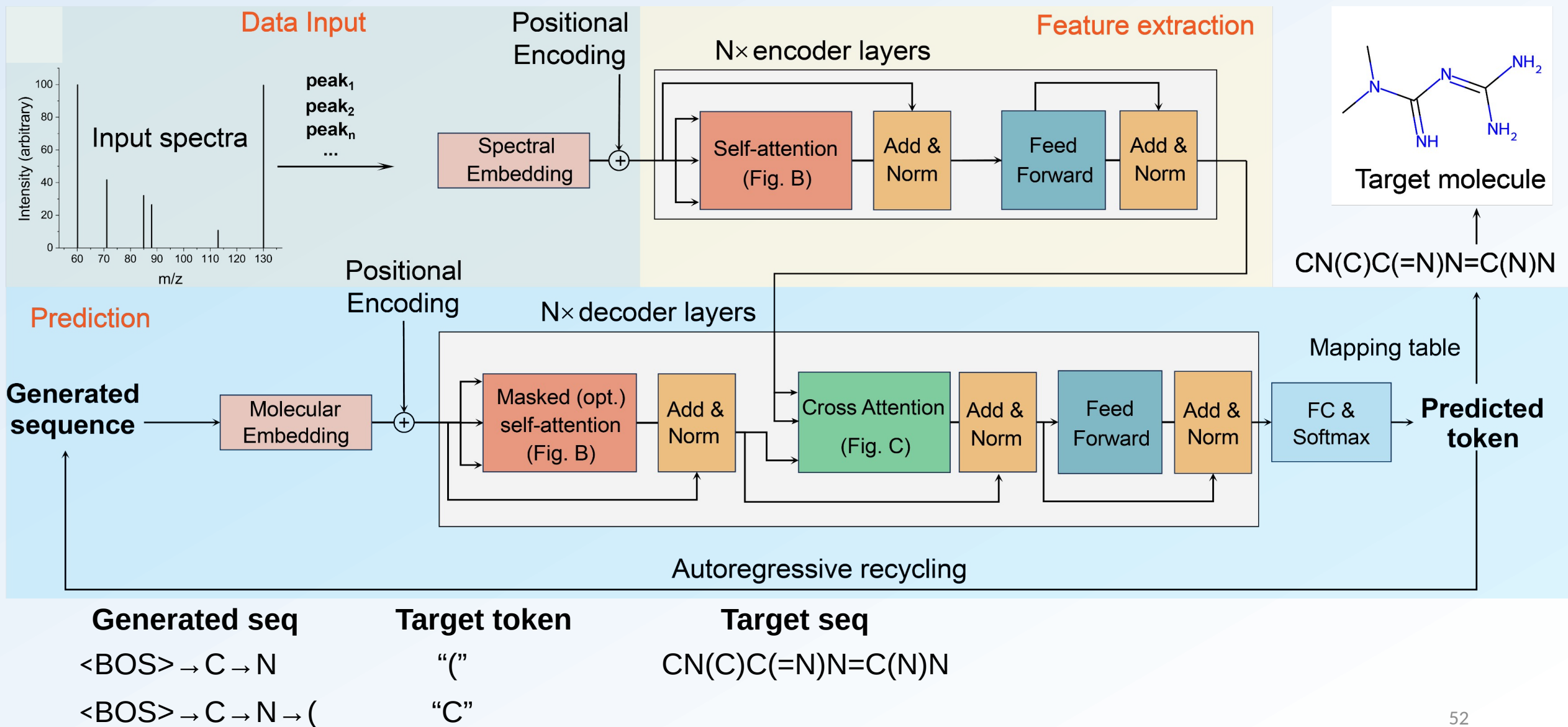
Input data



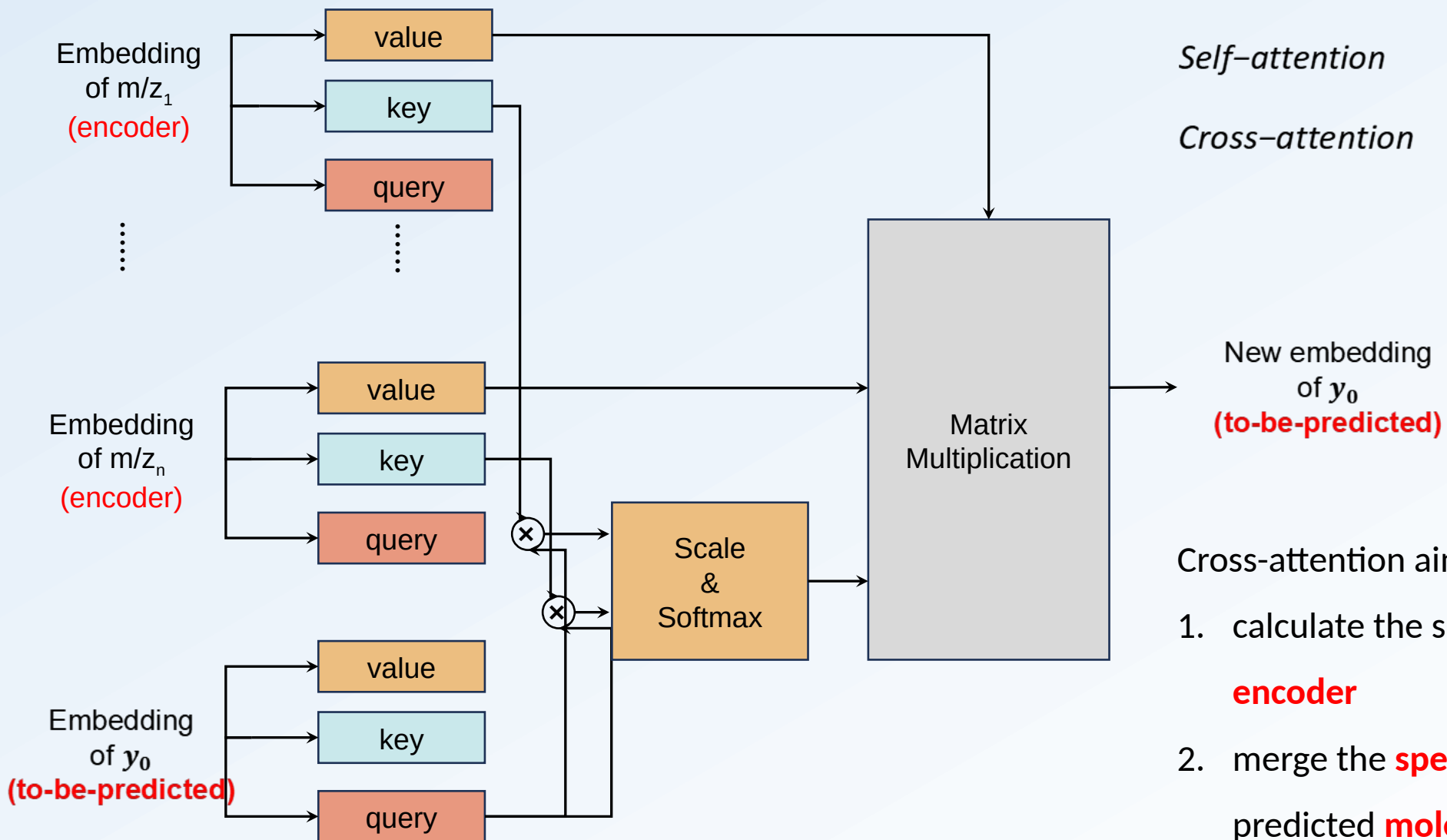
CN(C)C(=N)N=C(N)N $\rightarrow$ $C \rightarrow N \rightarrow \dots \rightarrow N$ $\rightarrow$ RNN encoder

Spectral vector $\rightarrow$ peak₁, peak₂, ..., peak_n $\xrightarrow{\text{position}}$ peak₁ $\rightarrow$ peak₂ $\rightarrow$... $\rightarrow$ peak_n $\rightarrow$ Transformer encoder

Prediction



Cross-attention



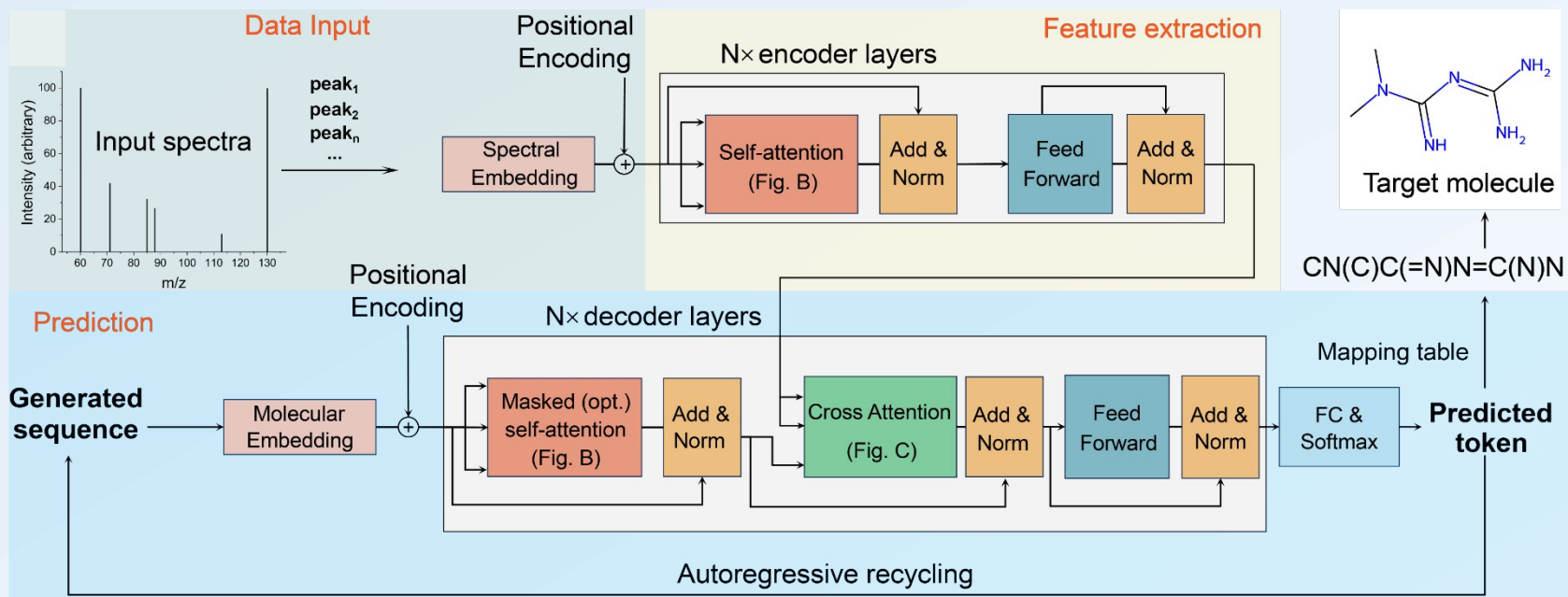
Self-attention $(Q, K, V) = f(Q_e, K_e, V_e)$

Cross-attention $(Q, K, V) = f(Q_d, K_e, V_d)$

Cross-attention aims to:

1. calculate the similarity with each token **from encoder**
2. merge the **spectral information** with to-be-predicted **molecular token**

Overview of transformer



1. A brand-new operation was developed to process sequential and non-sequential (without position) data
2. Its performance can be improved with more data

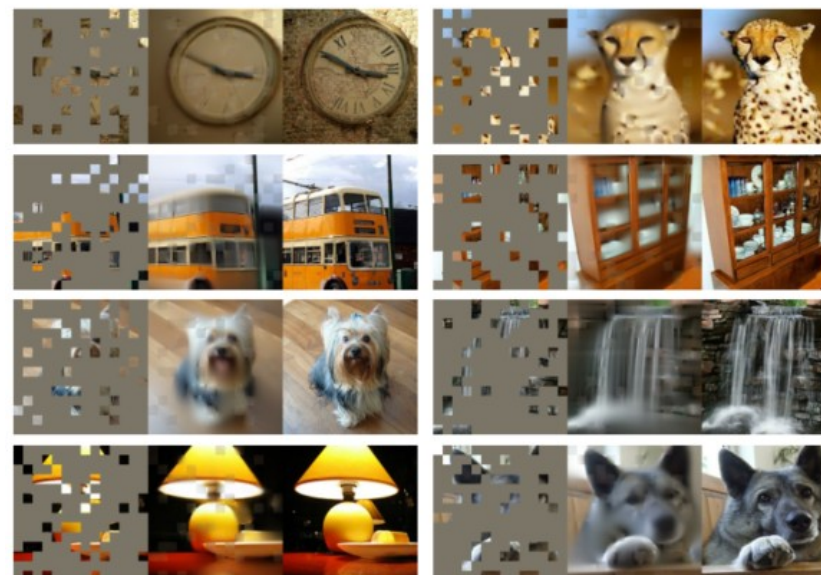
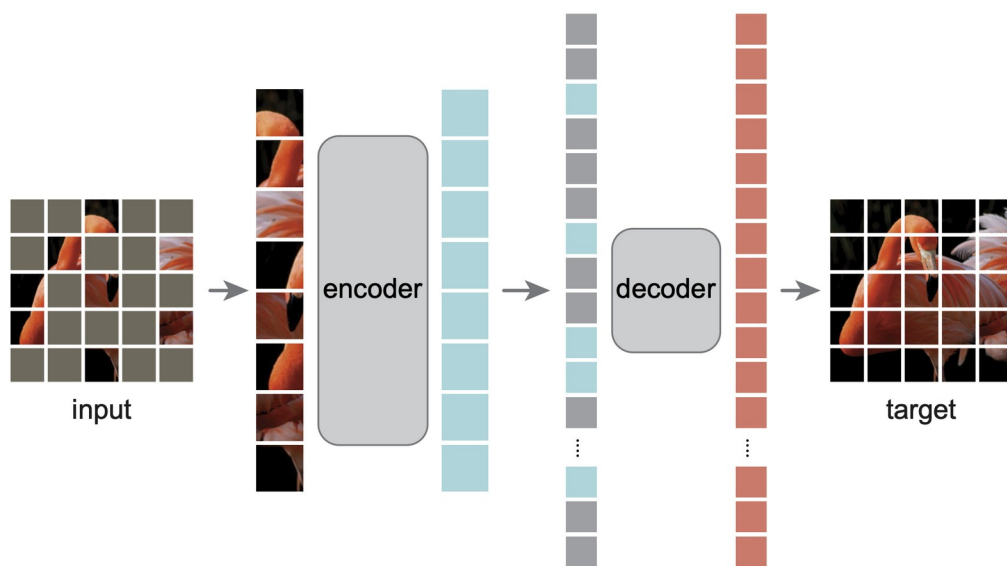


Masked Autoencoders Are Scalable Vision Learners

Kaiming He^{*,†} Xinlei Chen^{*} Saining Xie Yanghao Li Piotr Dollár Ross Girshick

^{*}equal technical contribution [†]project lead

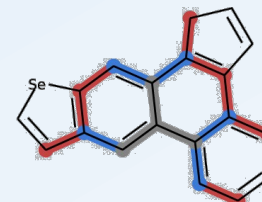
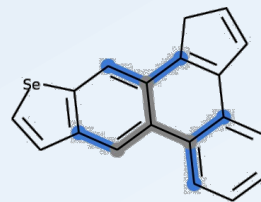
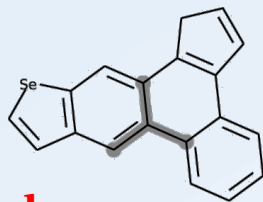
Facebook AI Research (FAIR)



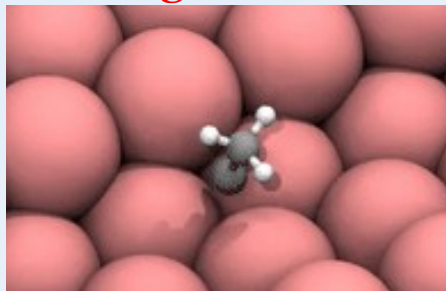
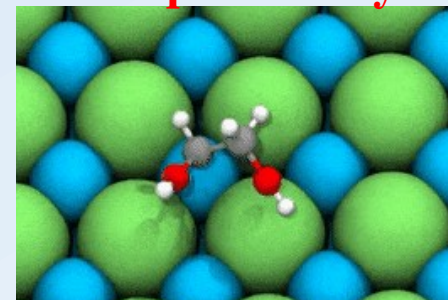
No label
Cheap

Reconstruction
More information

Graphormer



2021 Open Catalyst Challenge Winner Model



Transformer + Graph

Molecular property prediction

Material science

Biopharmaceutics

- Model the catalyst-absorbate reaction system
- Simulate more than 0.66 million catalyst-absorbate relaxation systems (144 million structure-energy frames) by density functional theory (DFT) software

Do Transformers Really Perform Bad for Graph Representation?

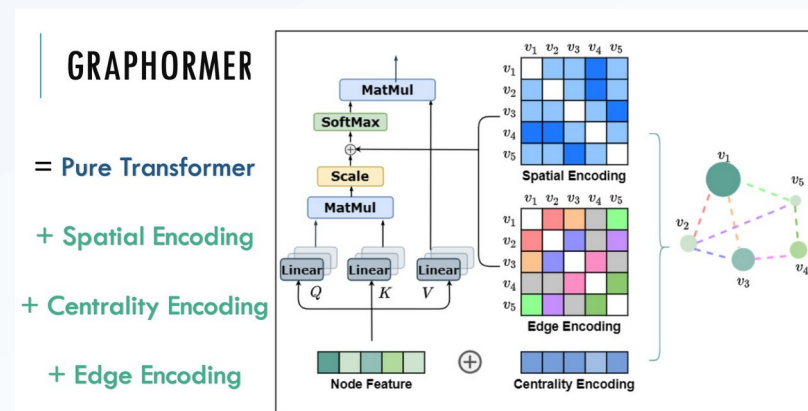
Chengxuan Ying^{1*}, Tianle Cai², Shengjie Luo^{3*},
Shuxin Zheng^{4†}, Guolin Ke⁴, Di He^{4†}, Yanming Shen¹, Tie-Yan Liu⁴

¹Dalian University of Technology ²Princeton University

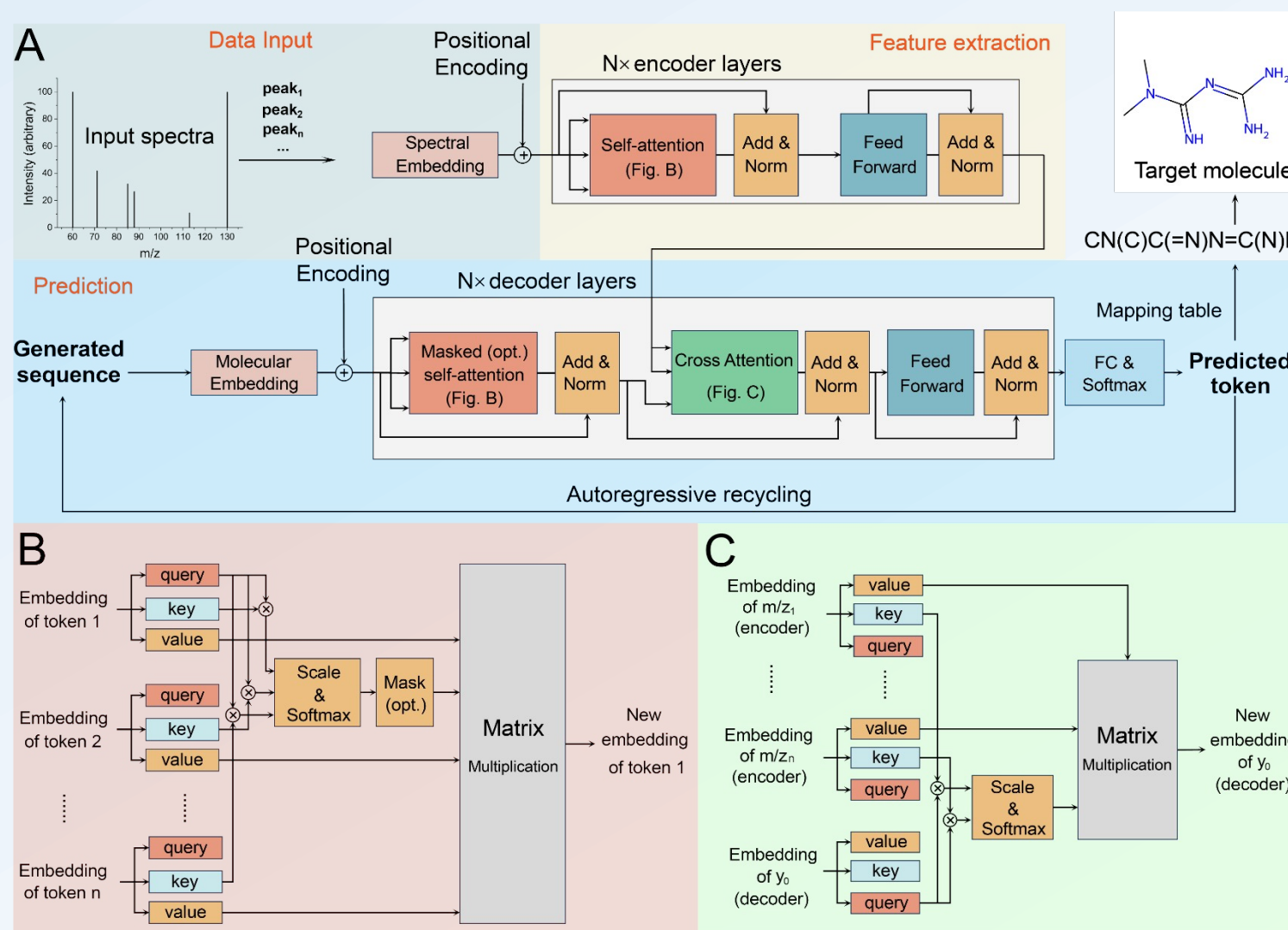
³Peking University ⁴Microsoft Research Asia

yingchengsyuan@gmail.com, tianle.cai@princeton.edu, luosj@stu.pku.edu.cn

{shuz†, guoke, dihe†, tyliu}@microsoft.com, shen@dlut.edu.cn



◆ Transformer

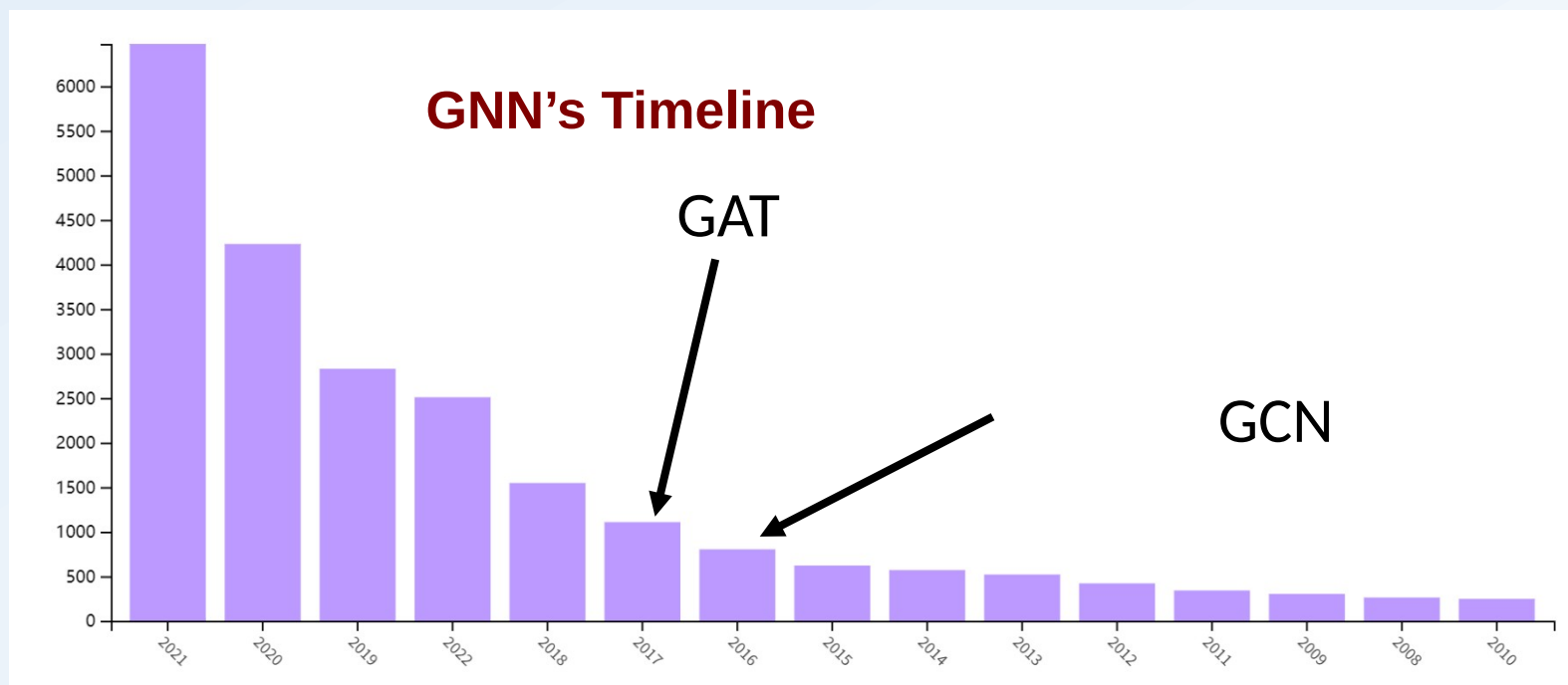


Attention mechanism is the key architecture of Transformer.

Summary

- Transformer was first proposed to solve machine translation problems
- Attention is the key architecture of Transformer
- Attention only uses relevance between two tokens, which results in no inductive bias
- Attention is a superset of both CNN and RNN units

◆ GNN :



SEMI-SUPERVISED CLASSIFICATION WITH GRAPH CONVOLUTIONAL NETWORKS

Thomas N. Kipf
University of Amsterdam
T.N.Kipf@uva.nl

Max Welling
University of Amsterdam
Canadian Institute for Advanced Research (CIFAR)
M.Welling@uva.nl

arXiv:1609.02907, 2016
Citaions: **16855**

GRAPH ATTENTION NETWORKS

Petar Veličković*
Department of Computer Science and Technology
University of Cambridge
petar.velickovic@cst.cam.ac.uk

Guillem Cucurull*
Centre de Visió per Computador, UAB
gcucurull@gmail.com

arXiv:1710.10903,
2017.
Citaions: **10024**

◆ Graph

– Introduction

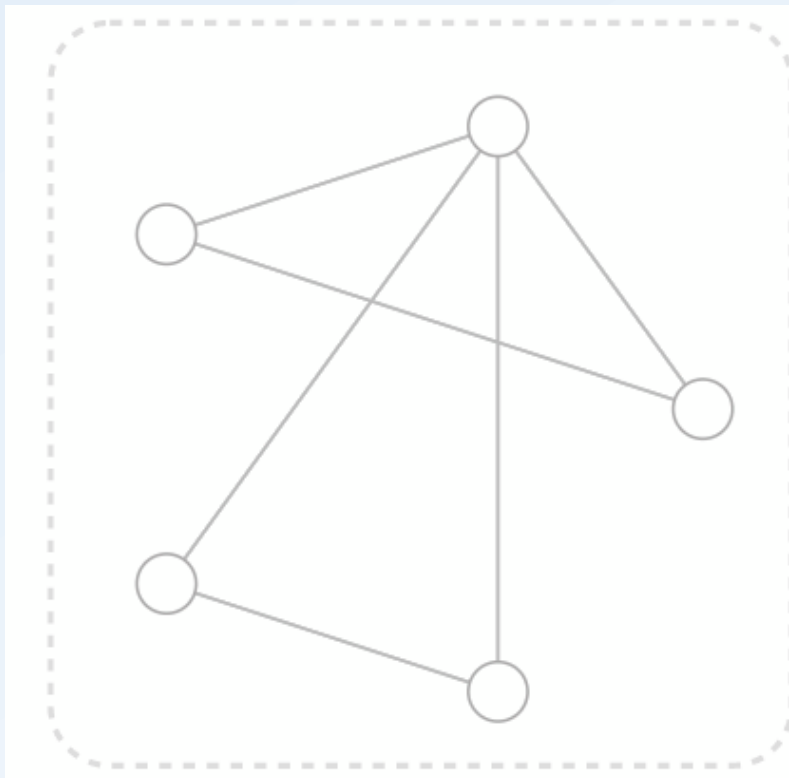
- ✓ What is graph?
- ✓ Where can we find graphs?
- ✓ What problems do graphs solve?

– Graph Neural Networks

– Application

◆ What can we do?

◆ What is graph?



- V** Vertex (or node) attributes
e.g., node identity, number of neighbors
- E** Edge (or link) attributes and directions
e.g., edge identity, edge weight
- U** Global (or master node) attributes
e.g., number of nodes, longest path

A data structure used to represent relationships

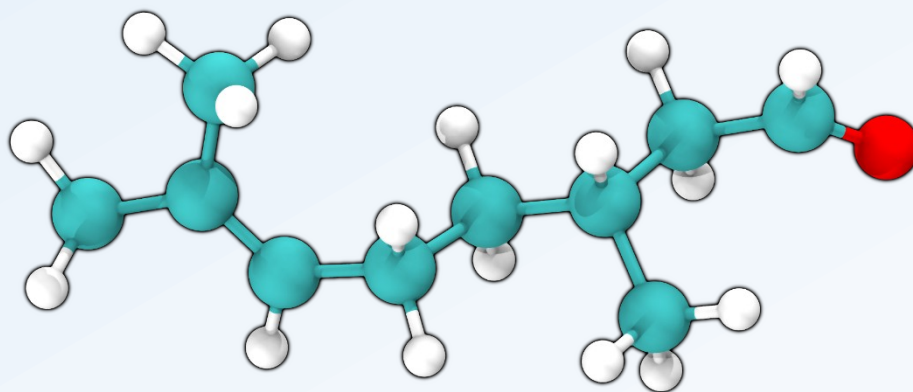
◆ Where can we find graphs?



Images



Texts



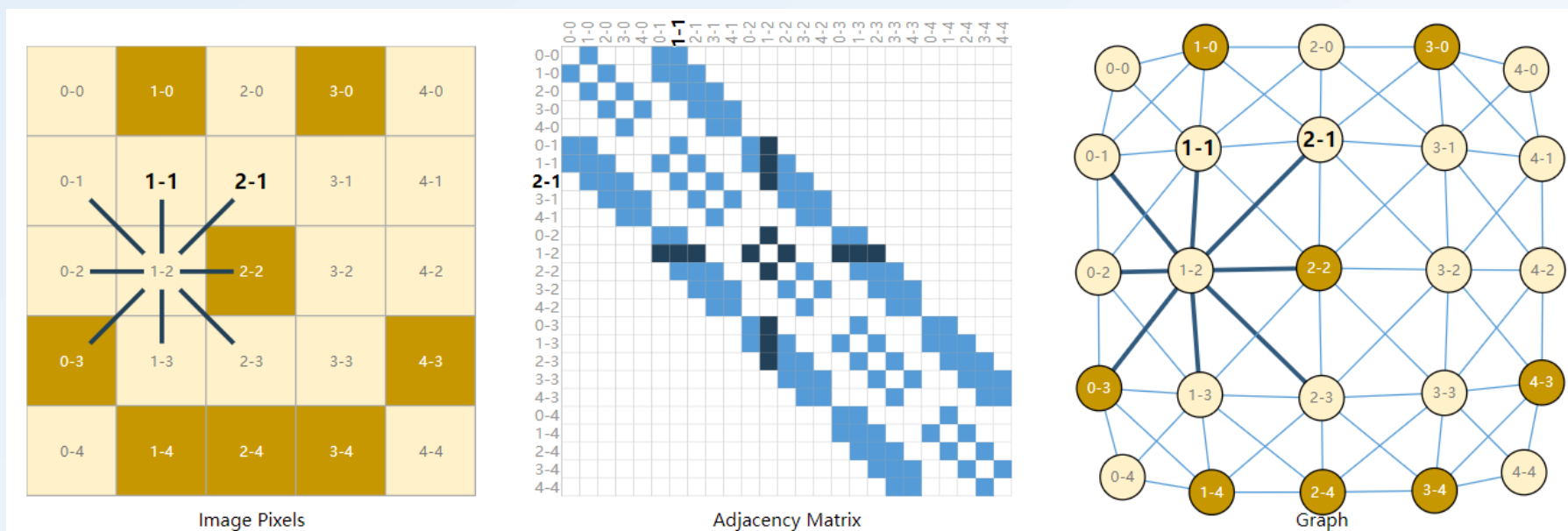
Molecules



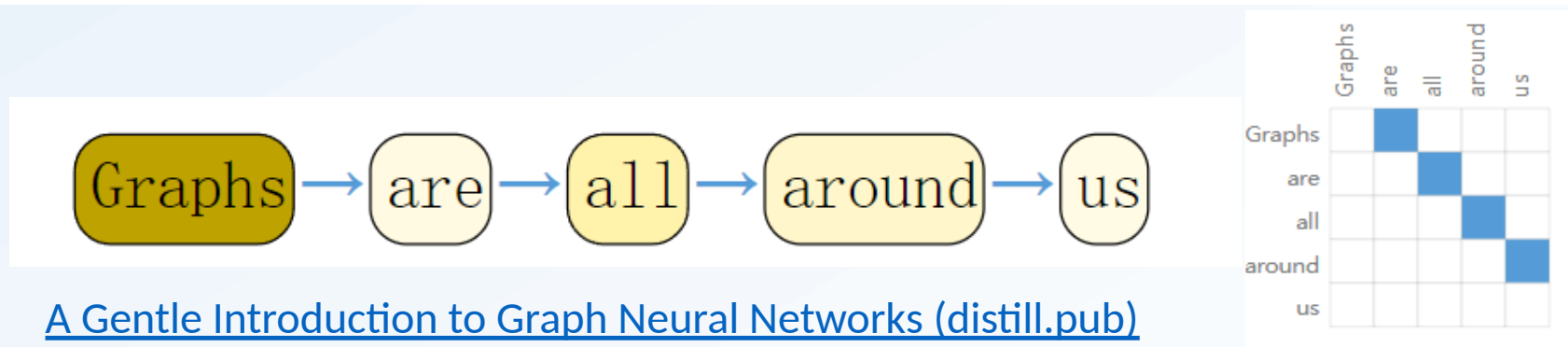
Social networks

◆ Where can we find graphs?

➤ Images



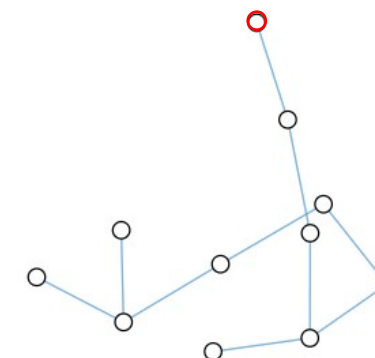
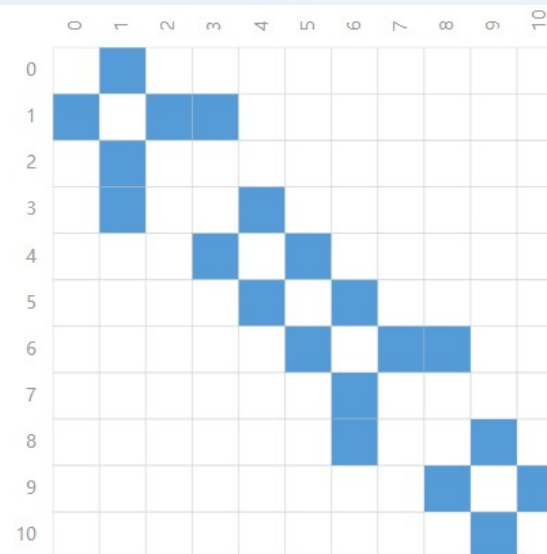
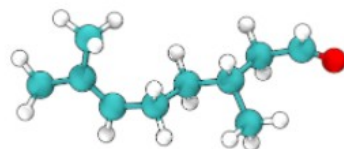
➤ Texts



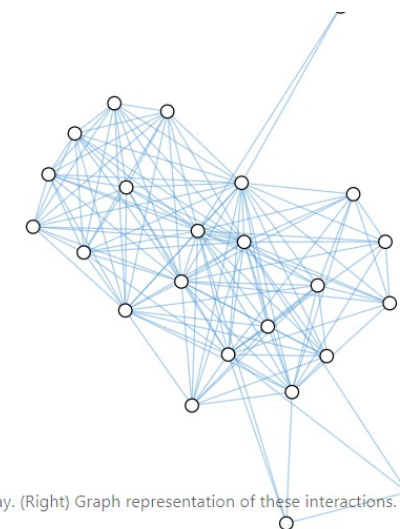
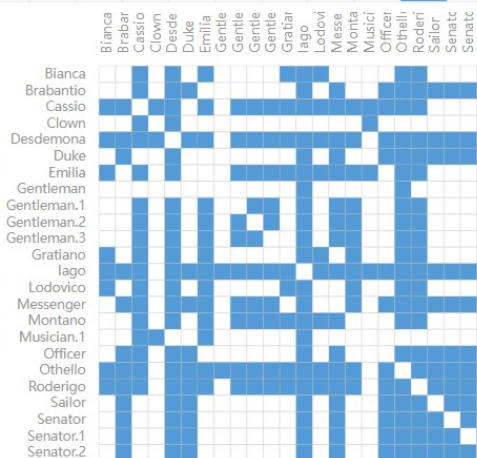
[A Gentle Introduction to Graph Neural Networks \(distill.pub\)](https://distill.pub/)

◆ Where can we find graphs?

➤ Molecules

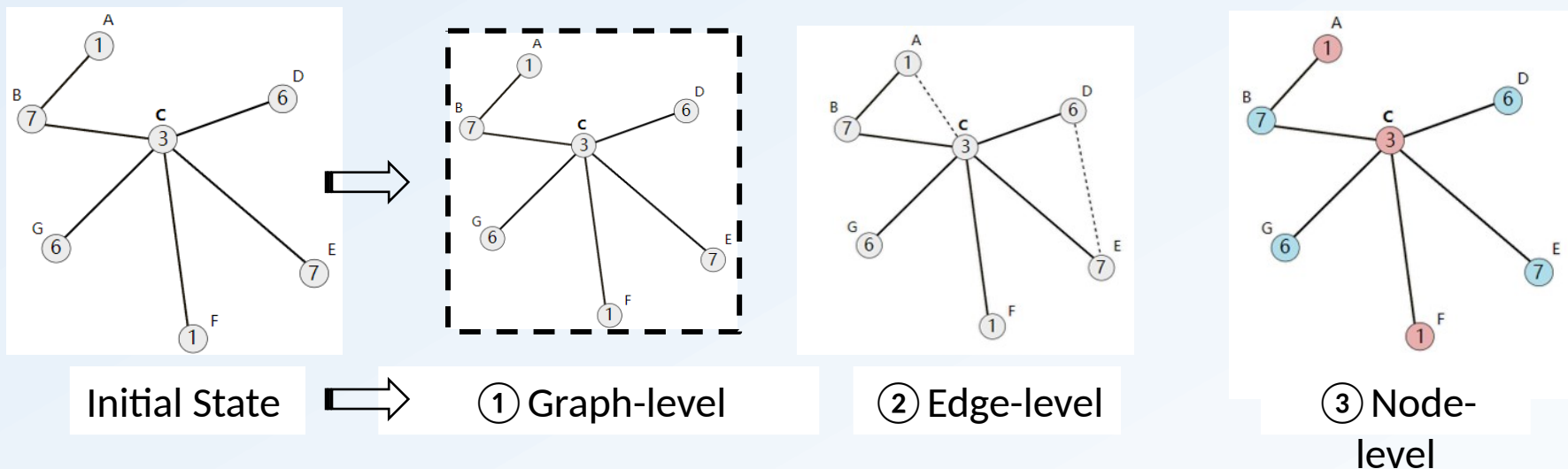


➤ Social Network

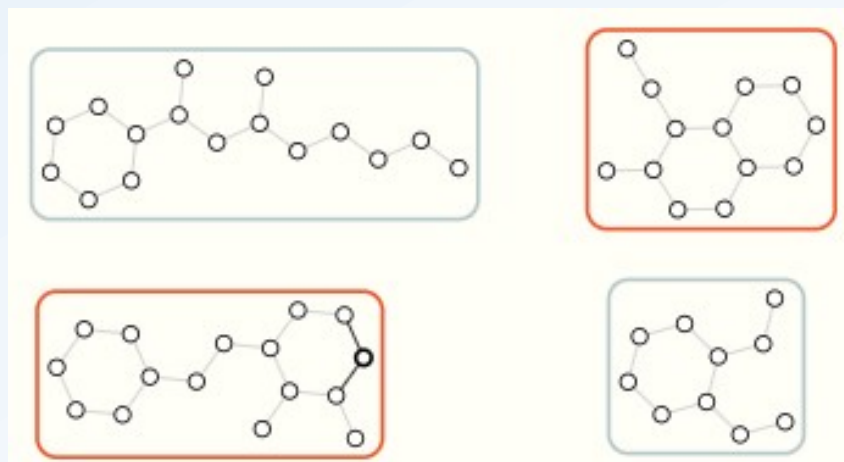


(Left) Image of a scene from the play "Othello". (Center) Adjacency matrix of the interaction between characters in the play. (Right) Graph representation of these interactions.

◆ What problems do graphs solve?



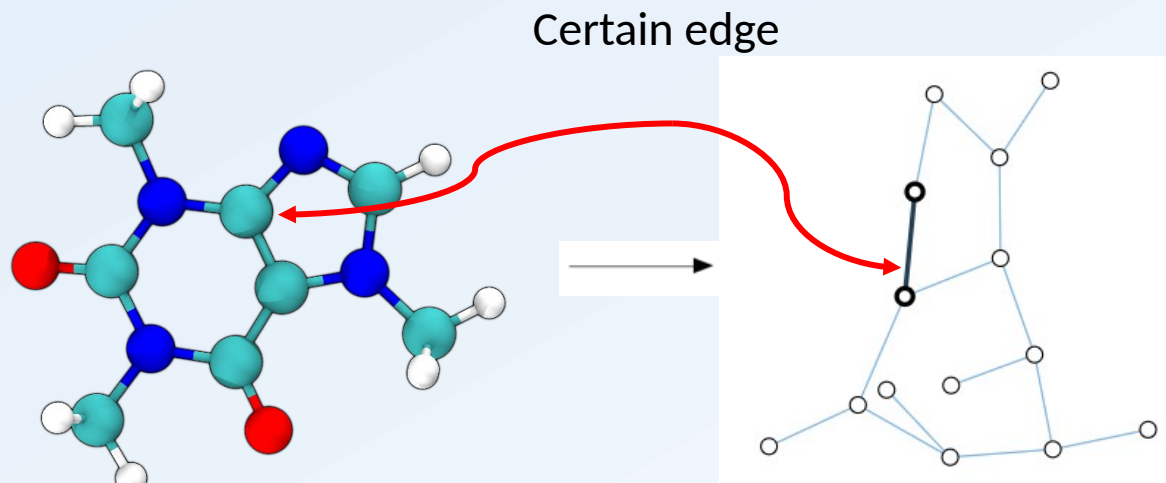
➤ Graph-level task



- What is this molecular?
- How active is the catalyst?

◆ What problems do graphs solve?

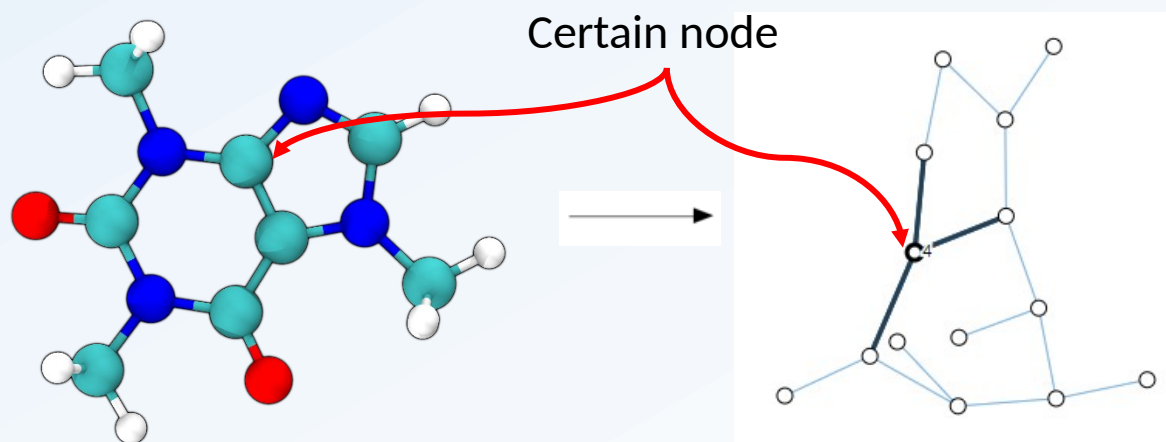
➤ Edge-level task



What is this bond?

How active is this bond?

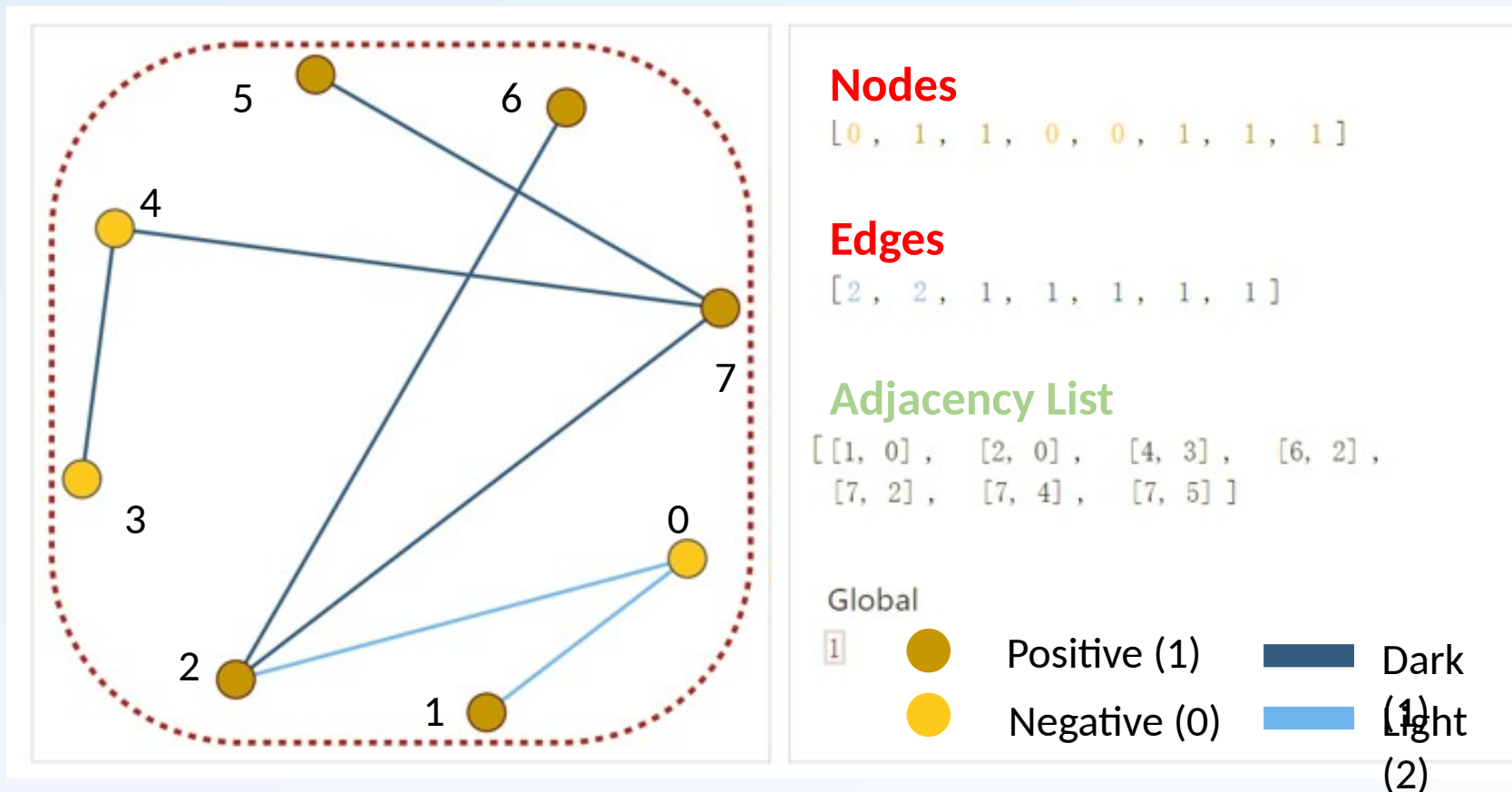
➤ Node-level task



What is this atom?

How active is this atom?

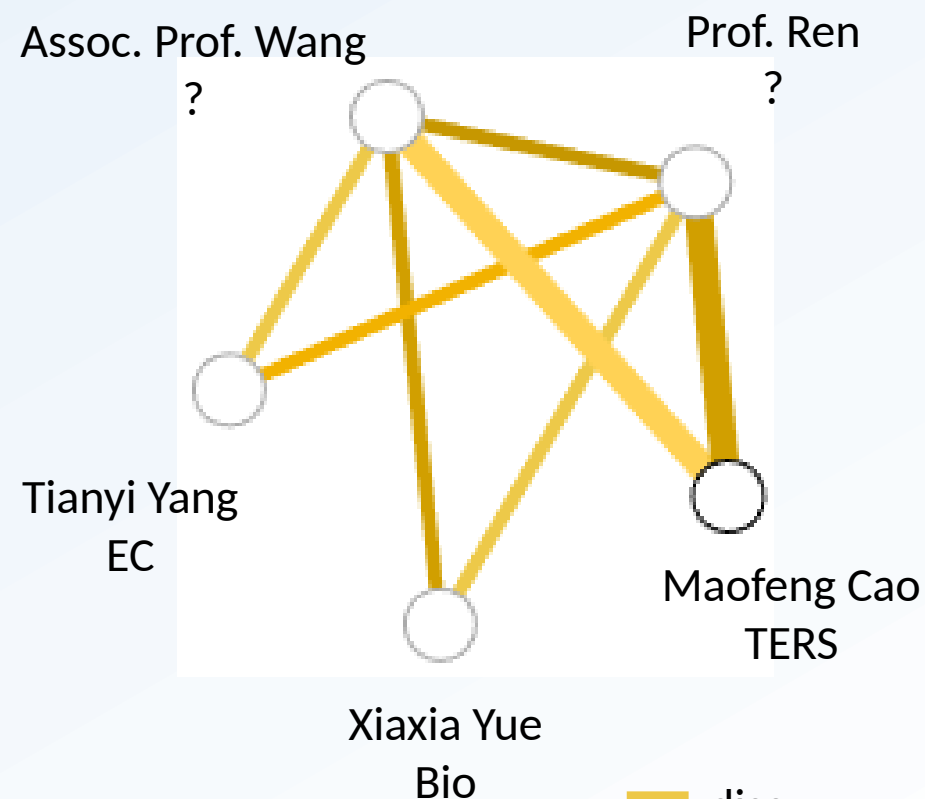
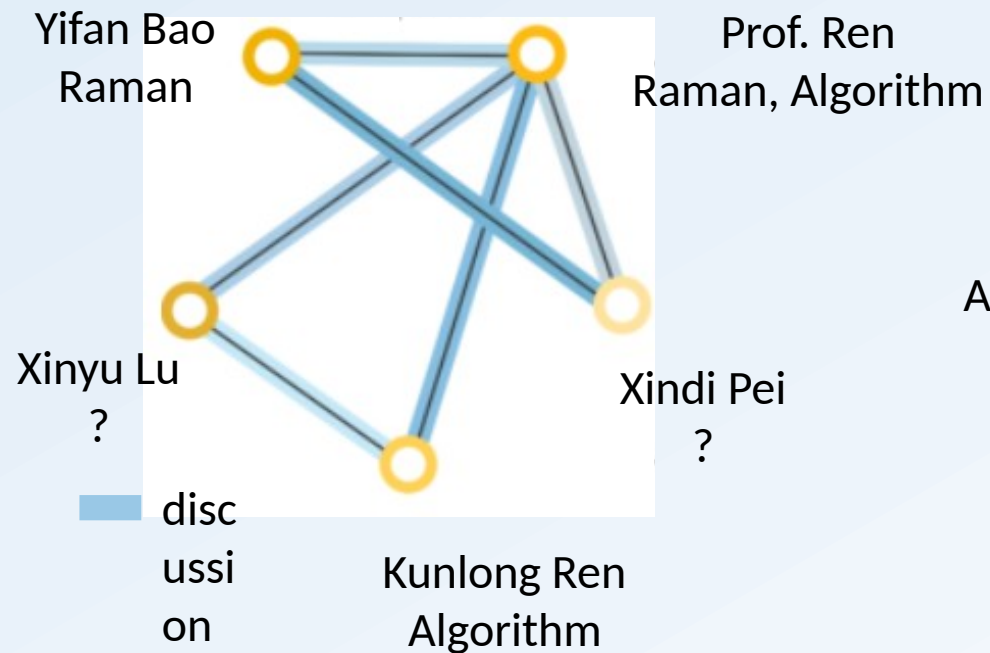
◆ How to represent a graph?



$$\text{output} = F(\text{features}, \text{structure})$$

$$= F(X, A)$$

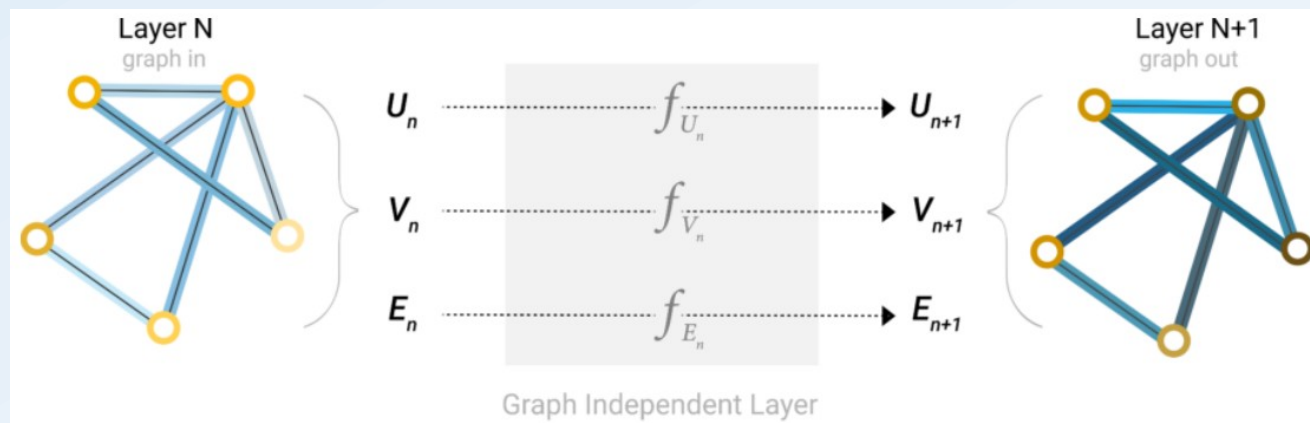
■ **Features/Embedding**
■ **Structure/Adjacency matrix**



What is the research
interest of him?

- **Graph in, graph out**

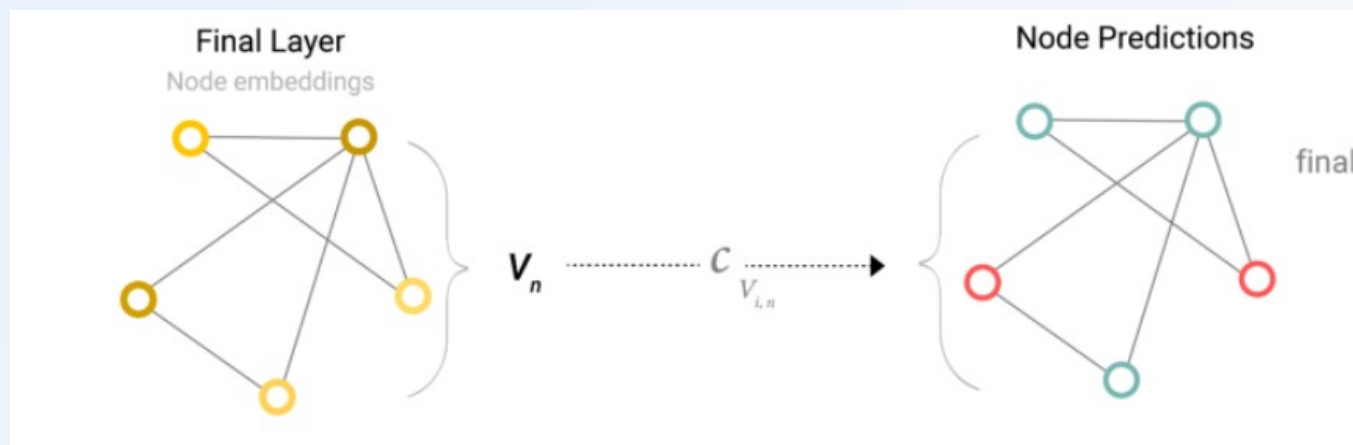
- **Permutation invariance**



update function $f = \text{[neural network icon]}, \dots$

Feature domain

Label domain



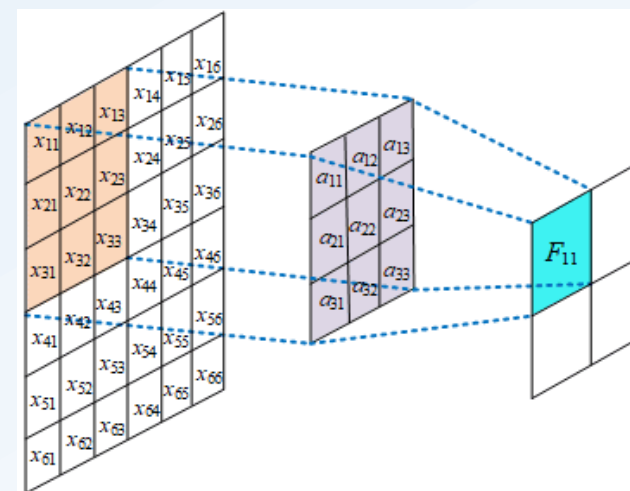
final classification $\mathcal{C} = \text{[neural network icon]}, \dots$

Nodes, edges and global attributes were updated independently

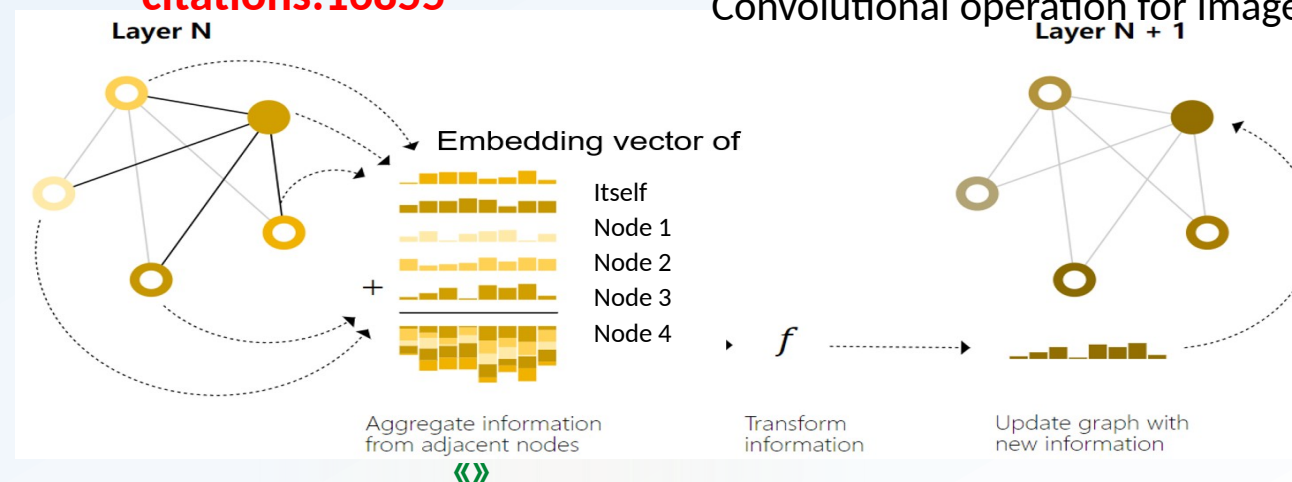
How to realize information aggregation?

◆ Graph Convolutional Network(GCN)

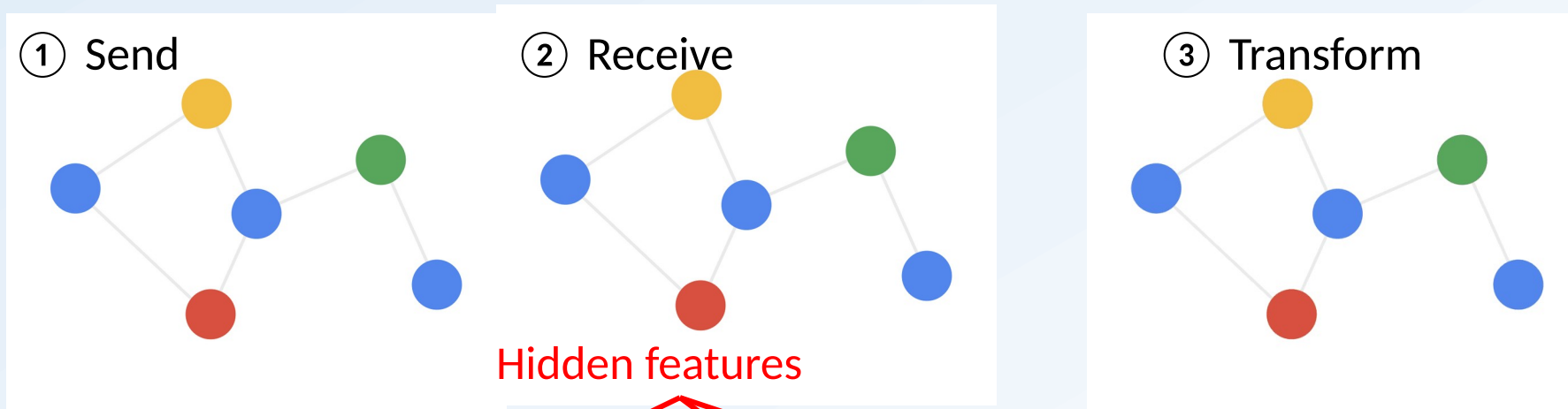
- 1998  Convolutional Neural Network
citations:46449
- 2014  Spatial Convolution on graphs
citations:3513
- 2015  Optimized by Chebyshev polynomial
citations:5075
- 2016  Stacked one-hop GCN
citations:16855



Convolutional operation for Images
Layer N + 1



◆ Graph Convolutional Network(GCN)



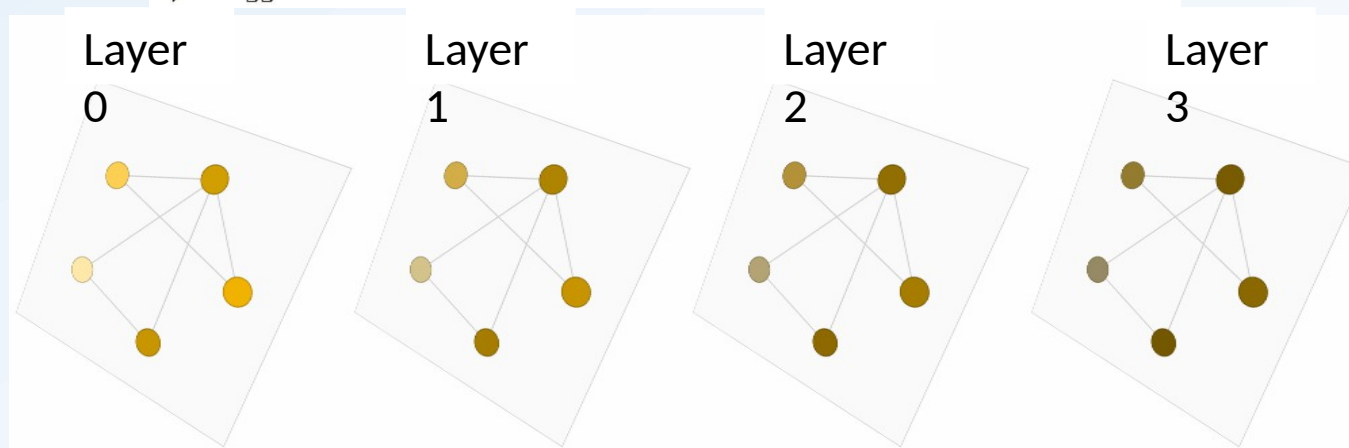
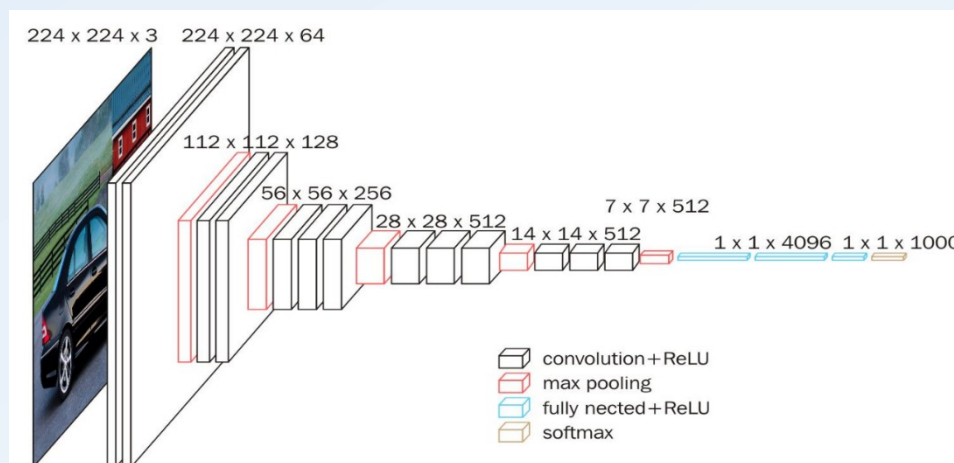
$$h_i^{l+1} = f\left(W^l \cdot \frac{\sum_{j \in N_i} h_j^l}{|N_i|} + B^l \cdot h_i^l\right)$$

Transform function

Neighbor of node i

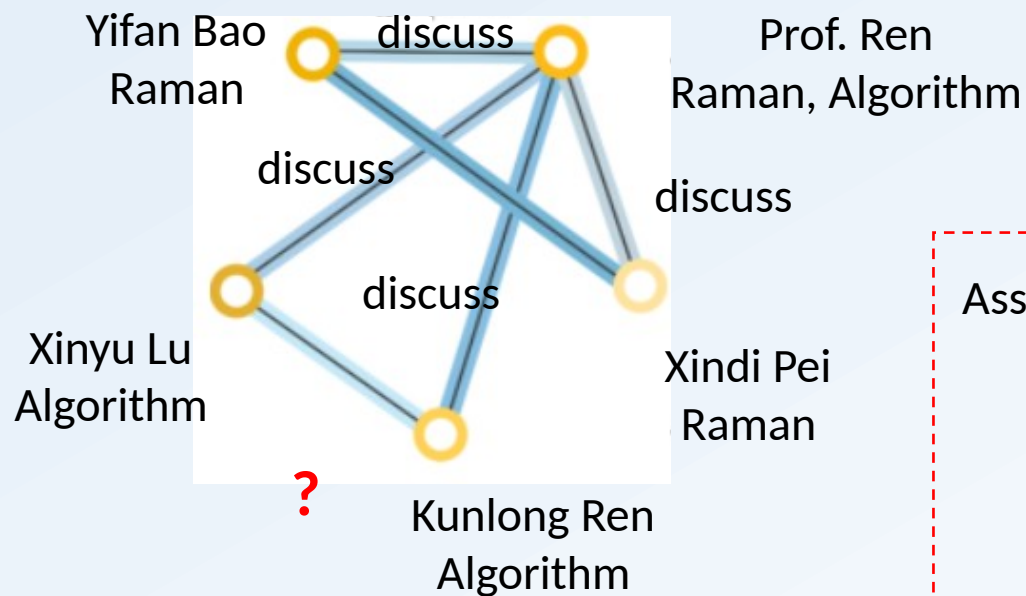
Learnable weights

◆ Graph Convolutional Network(GCN)



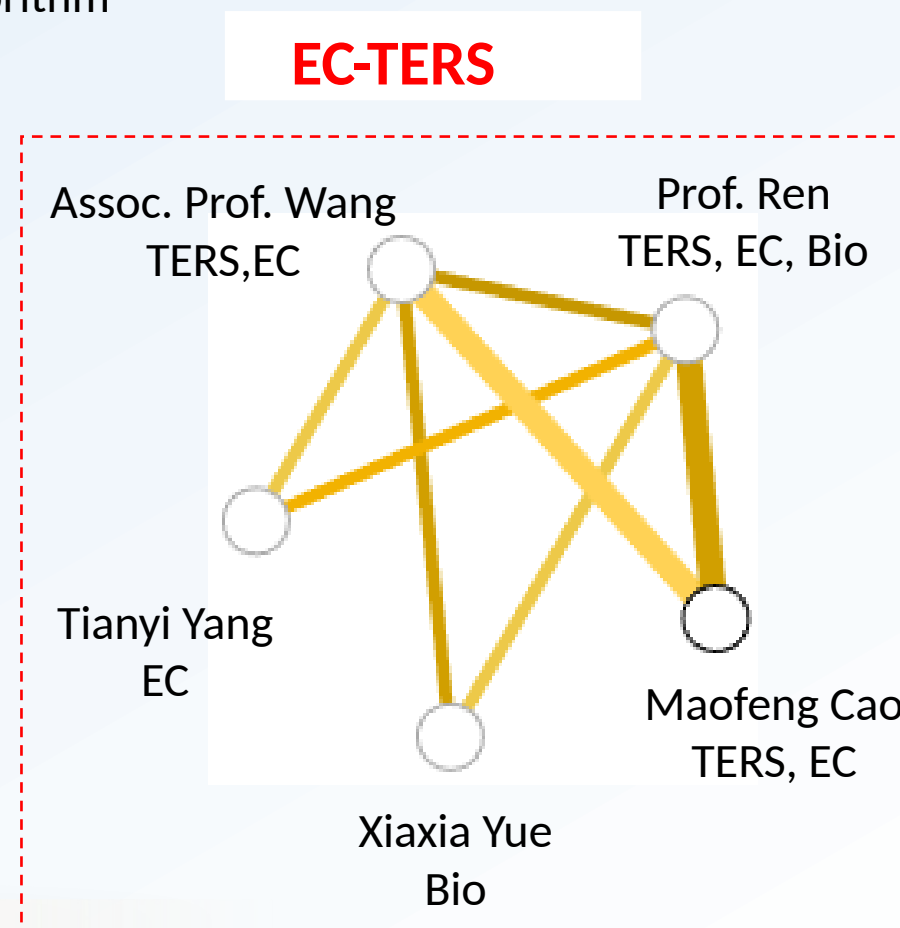
Information of each node is broadcast step by step through edges

◆ Graph Neural Network

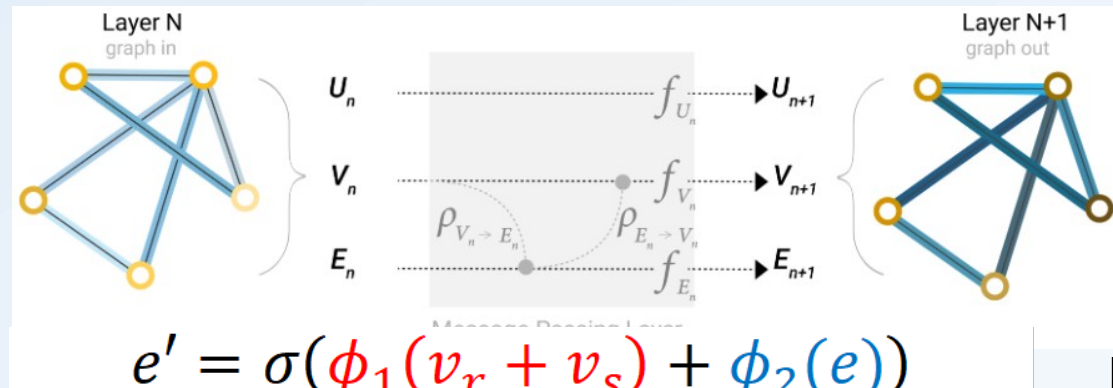


What are they doing?

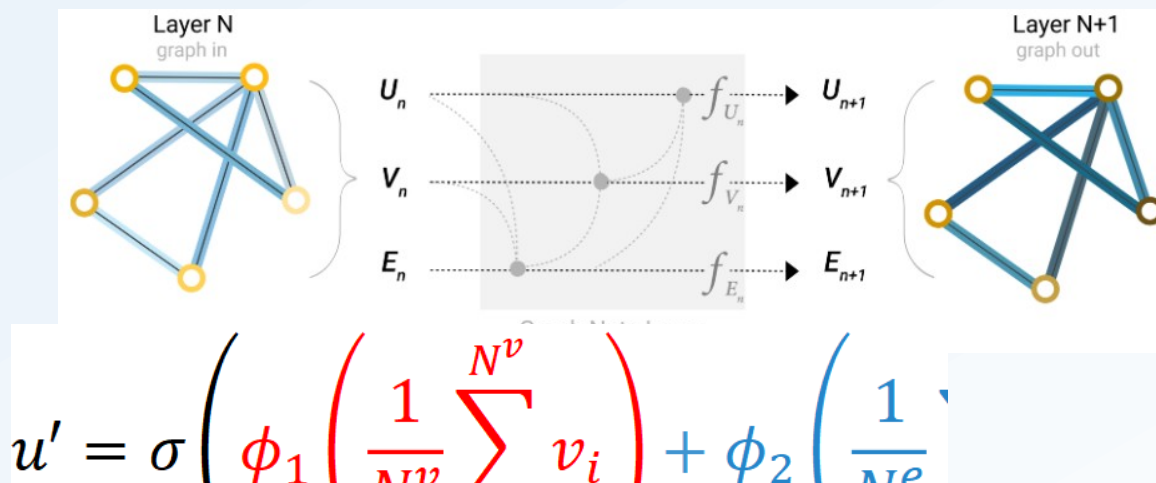
What the main research interest of this group?



◆ Different transmission methods

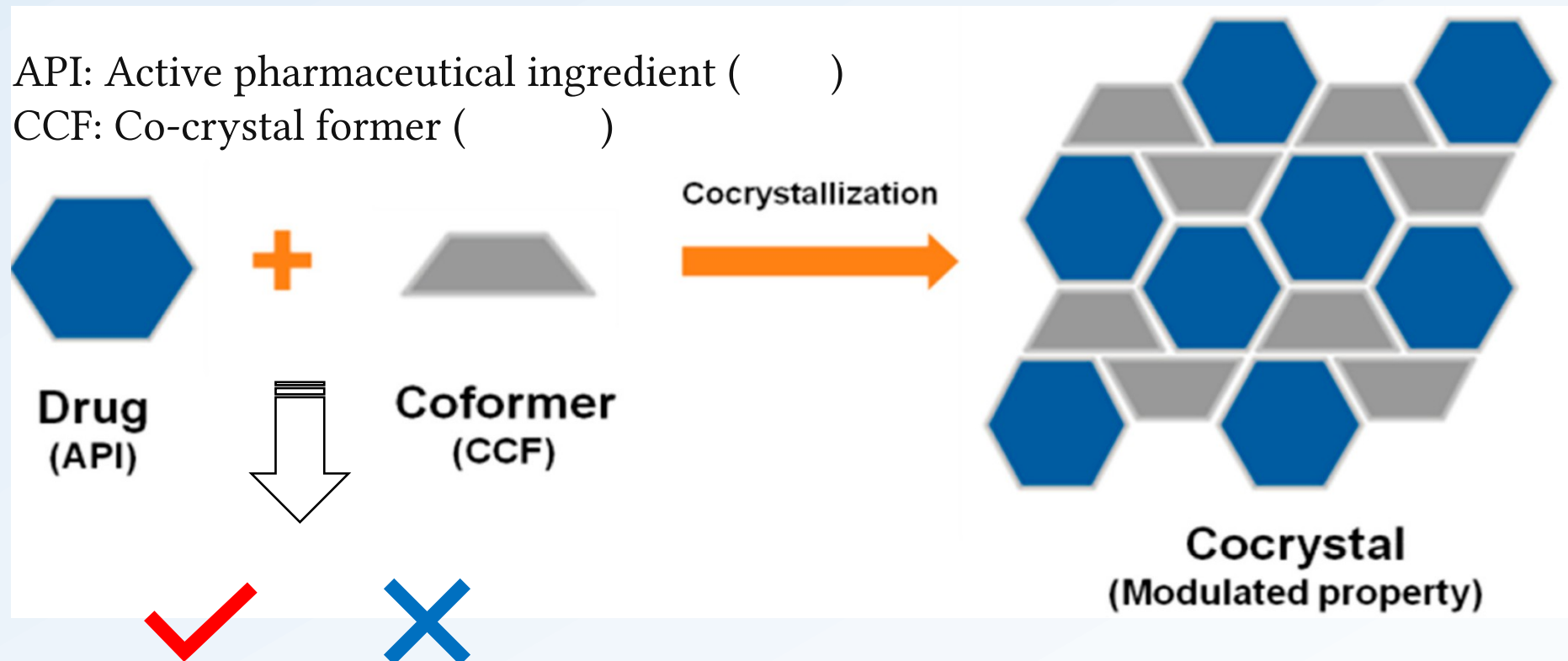


Nodes → Edges → Nodes



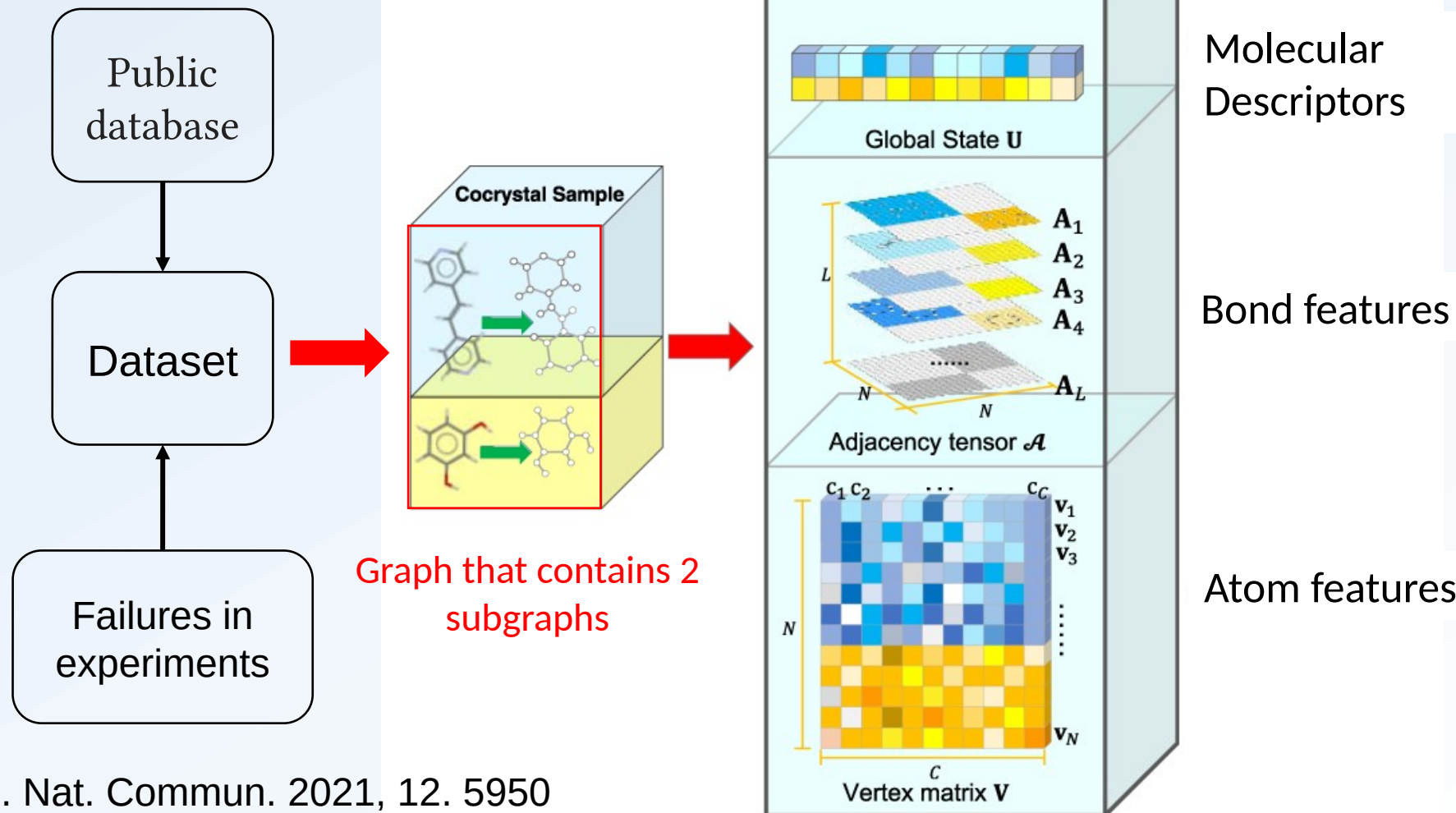
Fully interactive

◆ Co-crystal materials



Cocrystals are assembled by noncovalent forces in definite stoichiometric ratio
How to effectively choose coformer has been a challenging task

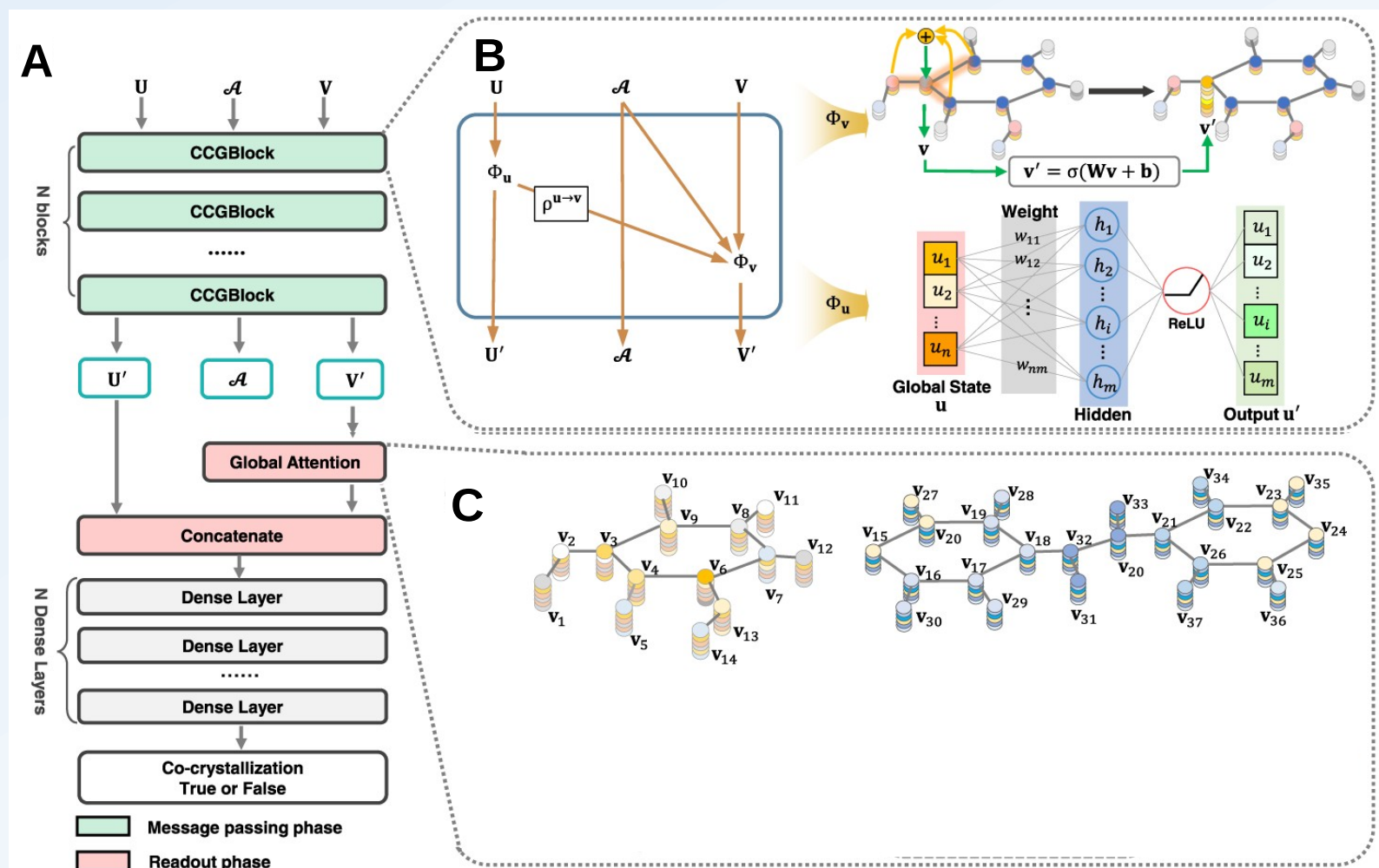
◆ Predict stability of cocrystal



Xuemei Pu et. al. Nat. Commun. 2021, 12. 5950

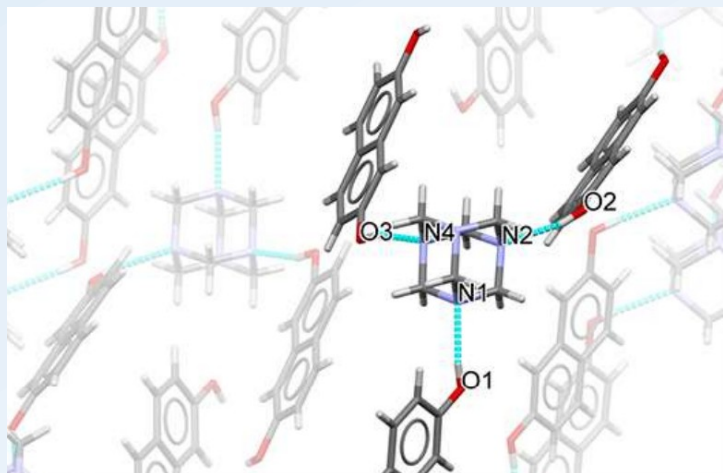
2026/9/11

◆ Predict stability of cocrystal



The more similar the environment of the atoms, the easier it is to form bonds

◆ Performance



Atoms in red are key points of prediction, which are actually consistent with chemical theory

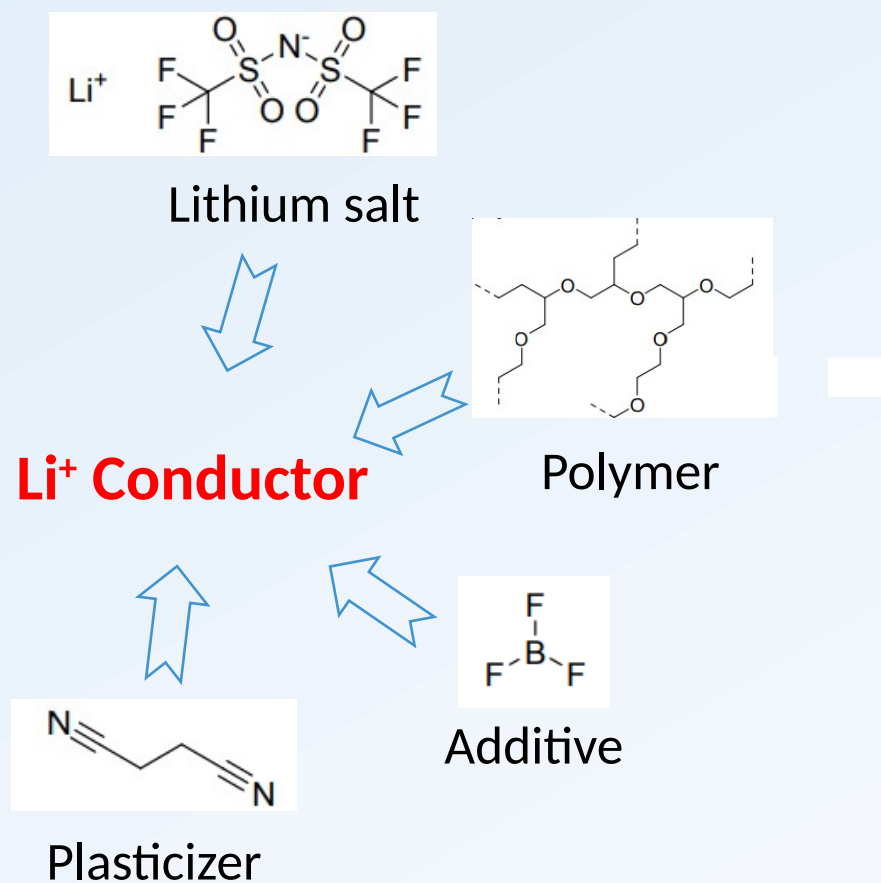
Visualization of attention weight

Can GNNs find things that are still hidden from humans?

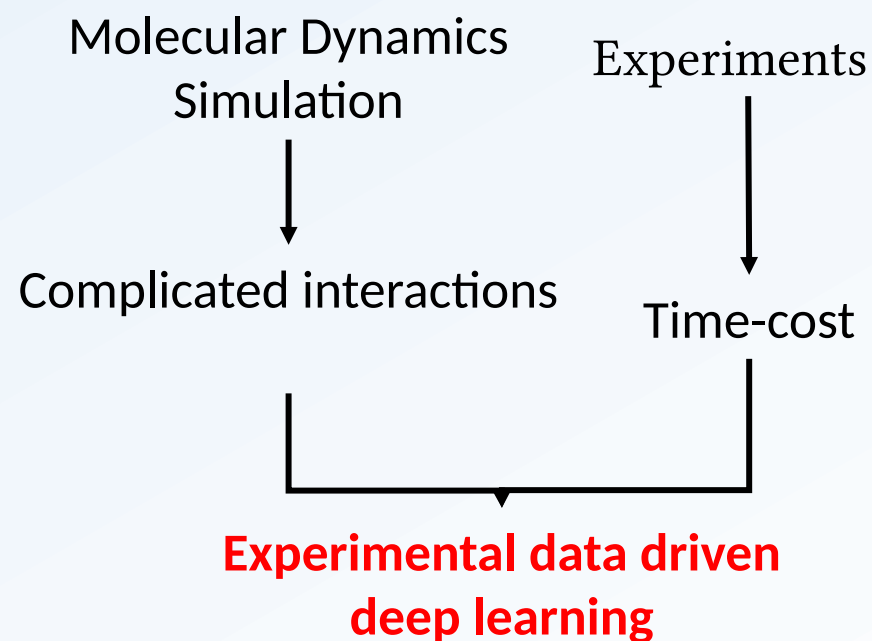
TNR: true negative rate BACC: balanced accuracy

Model	TPR (%)	TNR (%)	BACC (%)
DNN-des	99.55 (± 0.19)	89.11 (± 2.42)	94.33 (± 1.25)
DNN-FP	98.57 (± 0.46)	86.48 (± 4.86)	92.52 (± 2.37)
Graph-CNN	98.94 (± 0.39)	87.20 (± 3.33)	93.07 (± 1.60)
GCN	98.98 (± 0.43)	87.64 (± 3.47)	93.31 (± 1.76)
CCGNet	99.82 (± 0.14)	97.26 (± 1.61)	98.54 (± 0.79)

◆ Li⁺-conducting polymers

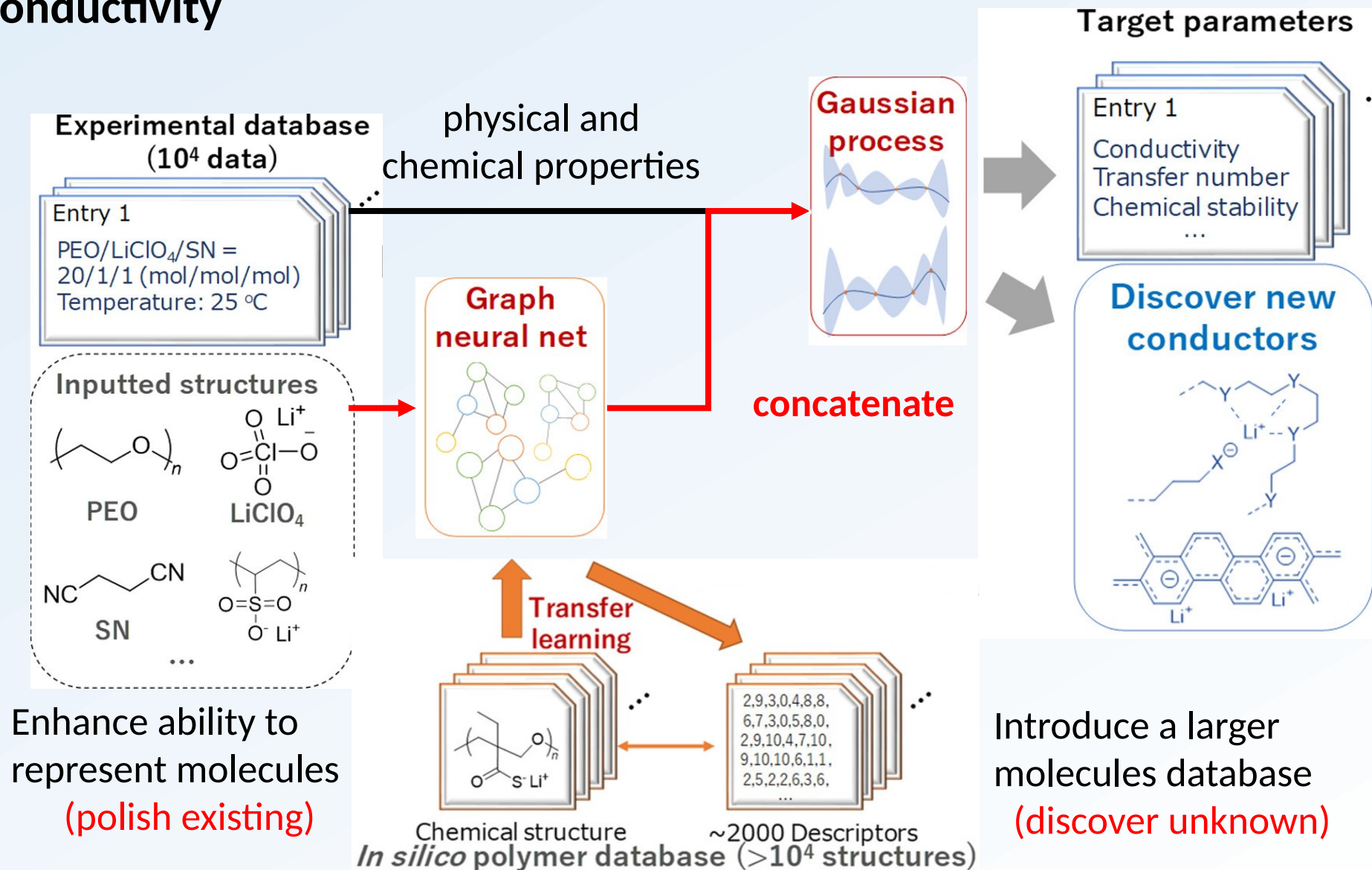


Methods to get conductivity

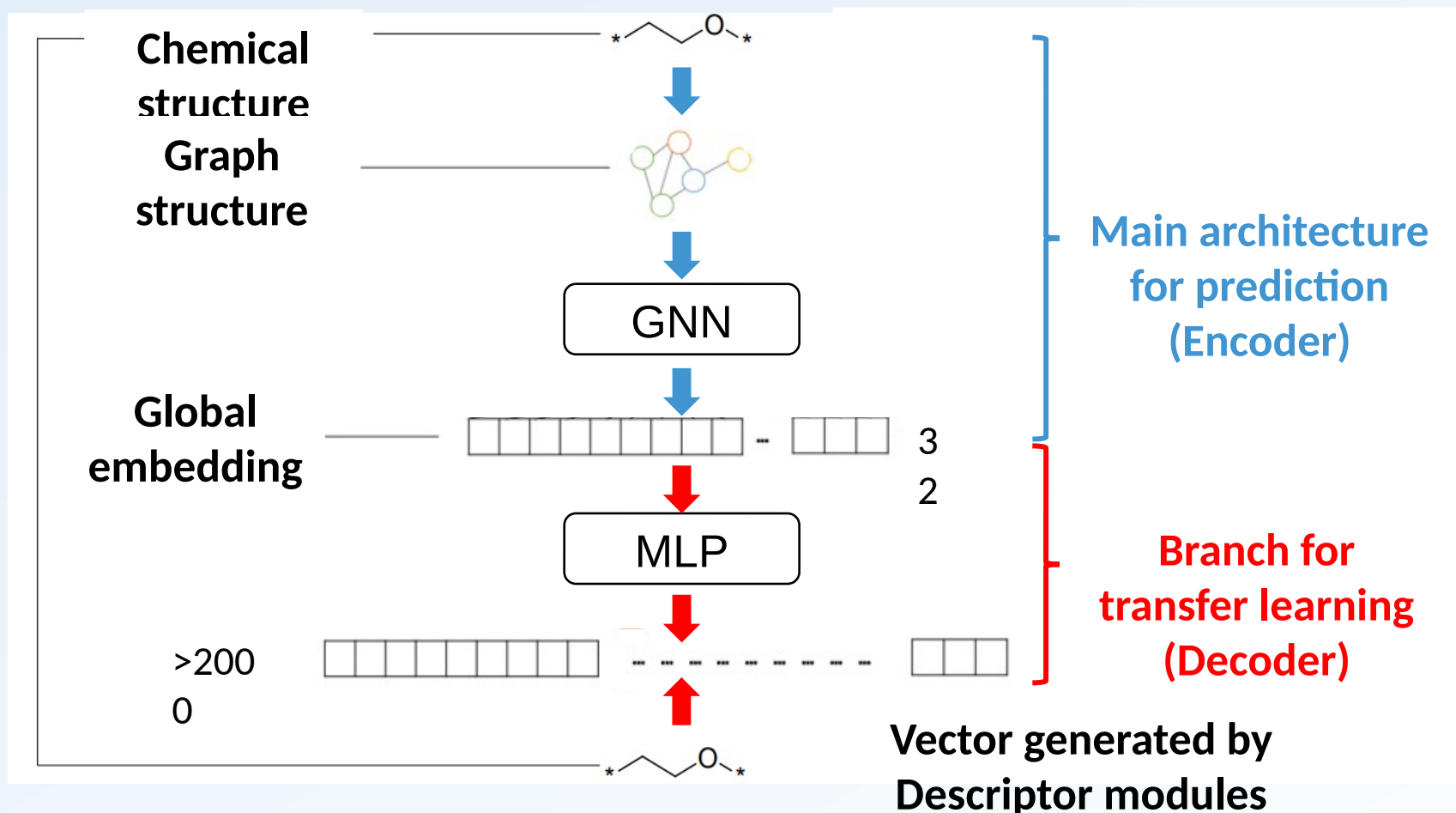


Kenichi Oyaizu et. al. *J. Am. Chem. Soc.* 2020, 142, 7, 3301–3305

◆ Predict Conductivity

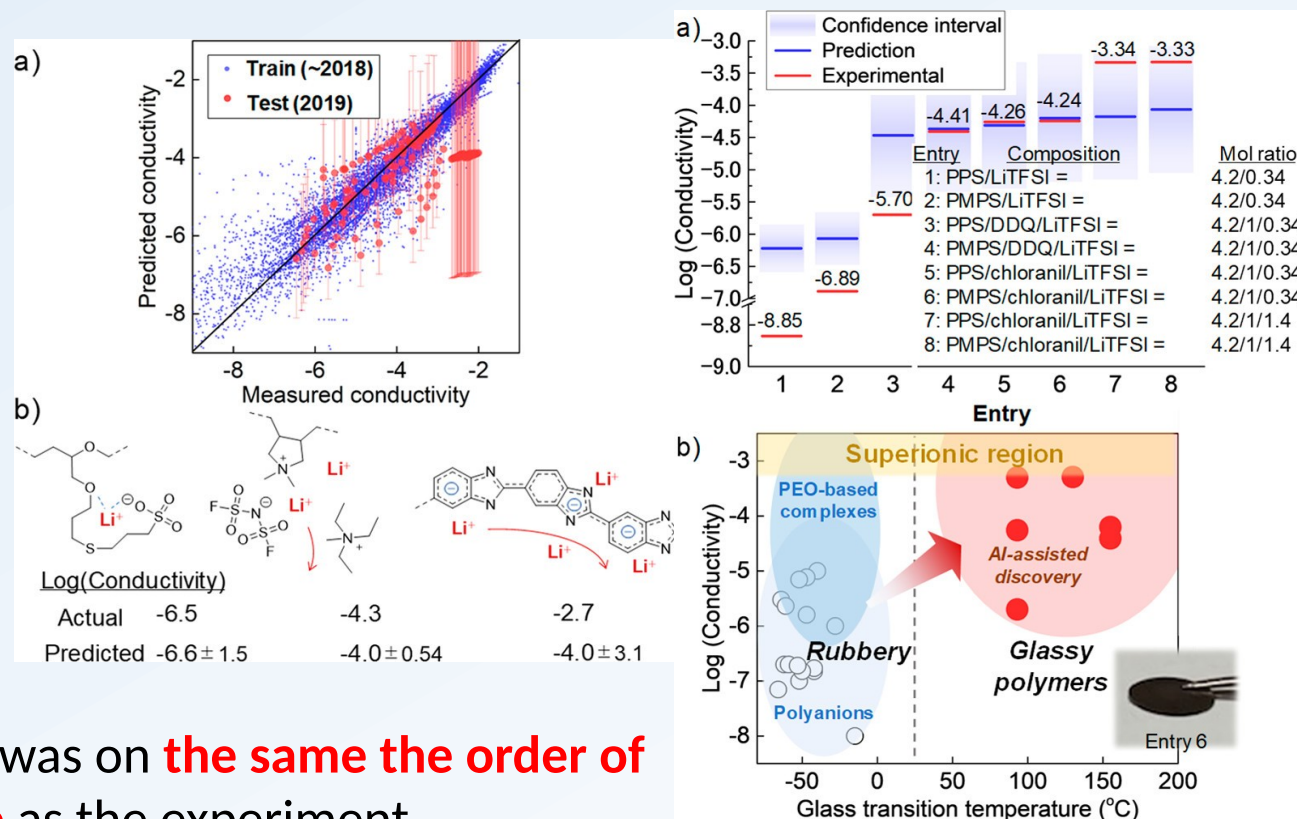


◆ GNN transfer learning



Squeeze 2000-D information into 32-D vector by self-supervised learning

◆ Materials Discovery

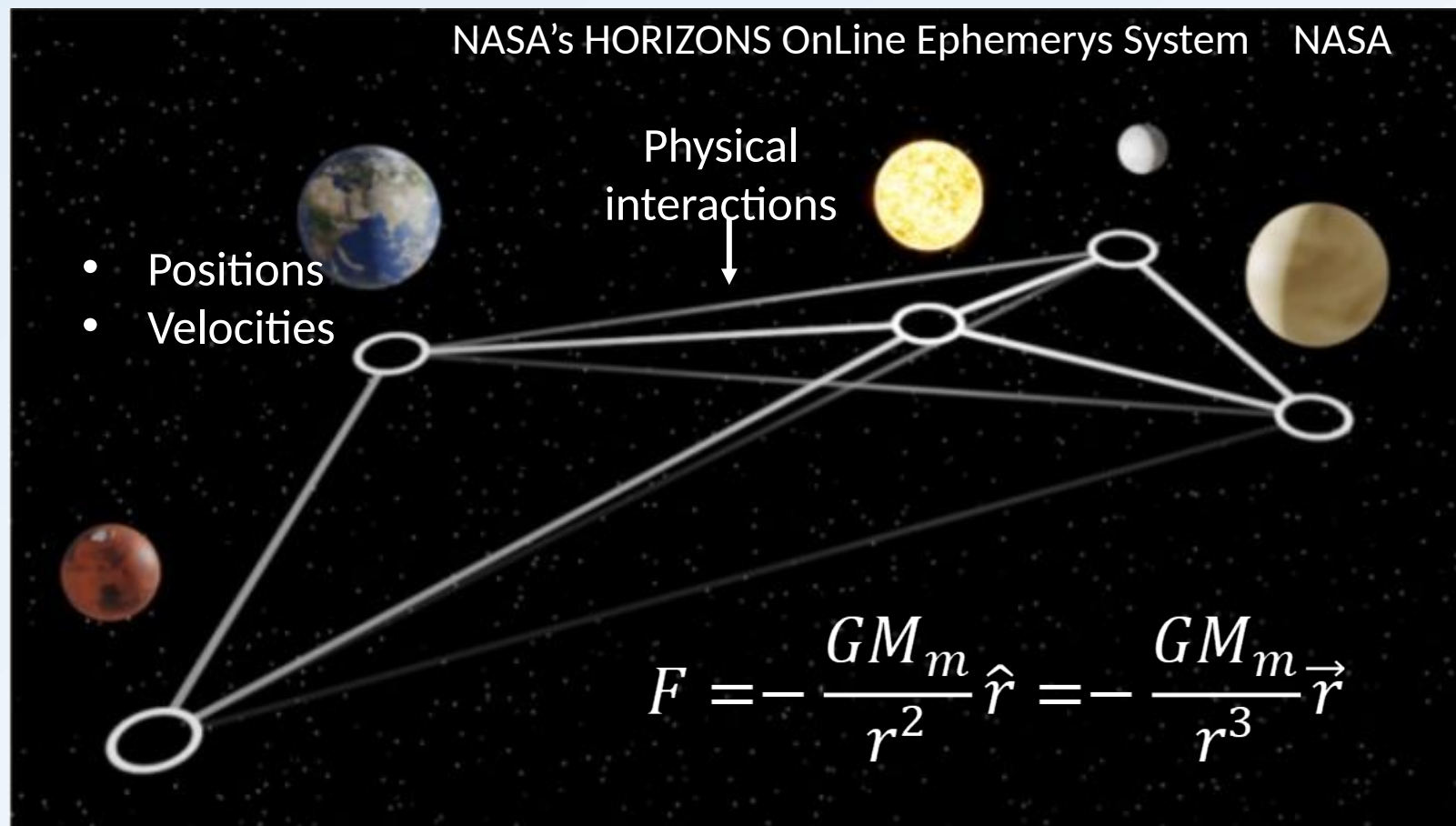


prediction was on **the same the order of magnitude** as the experiment

Unlike the pursuit of low-temperature superconductivity, AI finds that glassy polymers in reasonable proportions still have good conductivity

Can GNN be more powerful?

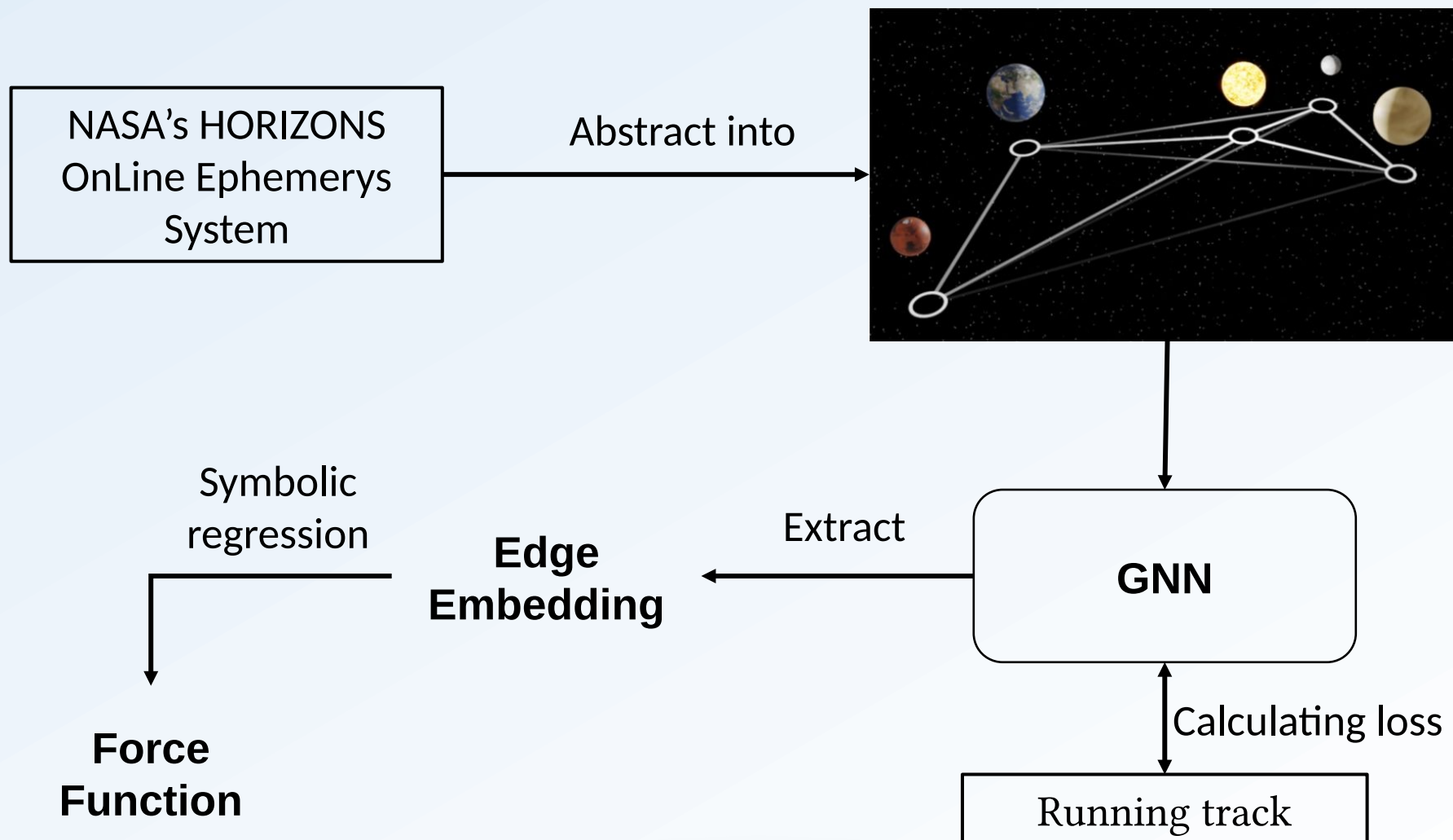
◆ Discover the law of universal gravitation



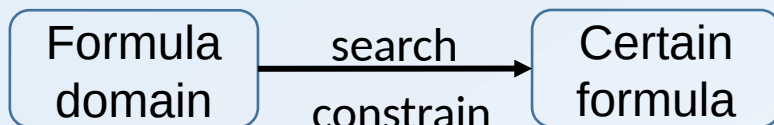
Challenges: no mass information of planets

$$F = f(\text{position}, \text{velocity})$$

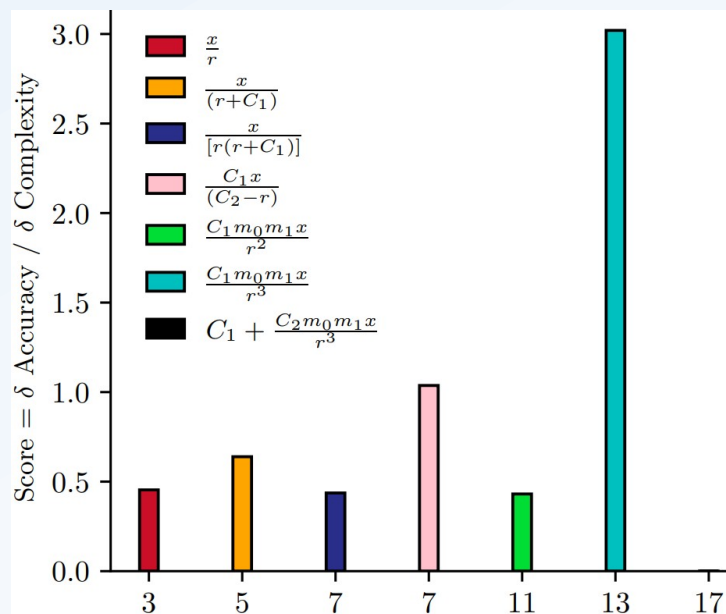
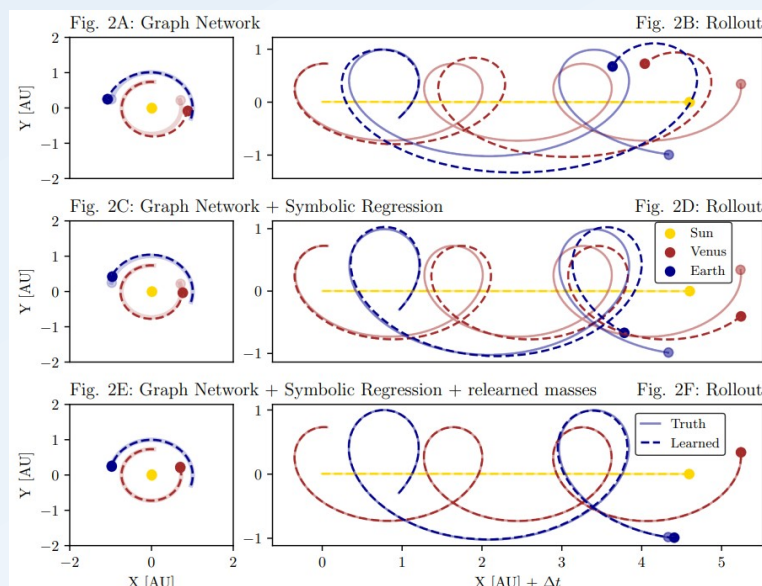
◆ Discover the law of universal gravitation



◆ Discover the law of universal gravitation



1. Explicit mapping function
2. Enhance generalization ability

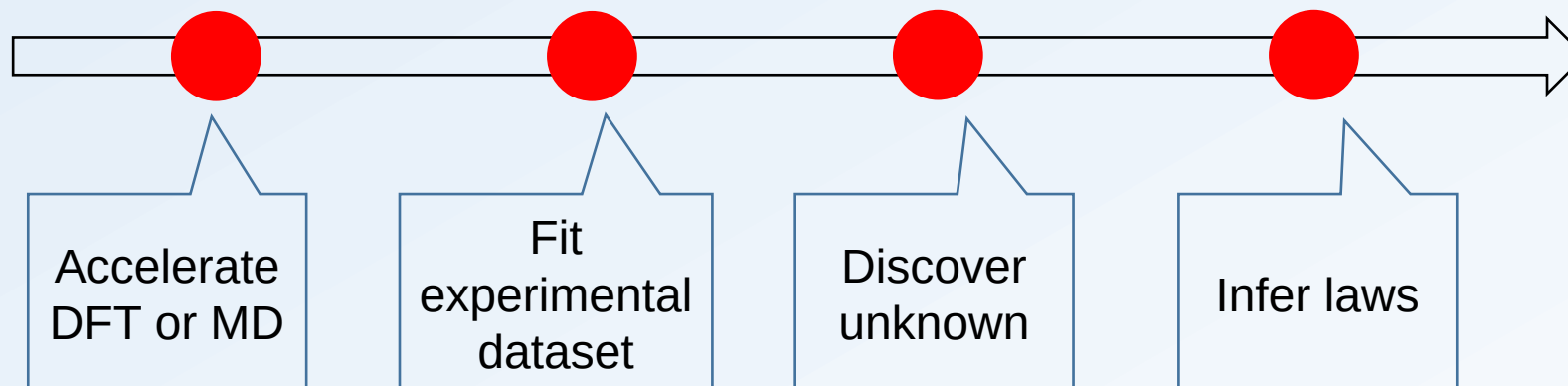


$$f = \frac{C_1 m_0 m_1 x}{r^3} \xrightarrow[\substack{\text{when } C_1 = G \\ x = r}]{\text{}} F = -\frac{G m_0 m_1}{r^3} \vec{r}$$

◆ Discover the law of universal gravitation

Videos before and after symbolic regression

Summary



- Graph has a strong ability to represent molecules
- Knowledge becomes more important for AI
- Experiments and algorithms complement each other
 - Experiments are the source of data for algorithms
 - Algorithms make experimentation and discovery more efficient

The era of data-driven science is coming.....

🚩 DeepSeek-V3.2 正式版发布，强化 Agent 能力，融入思考推理，在网页端、APP 和 API 全面上线，[点击查看详情](#)。

deepseek

探索未至之境

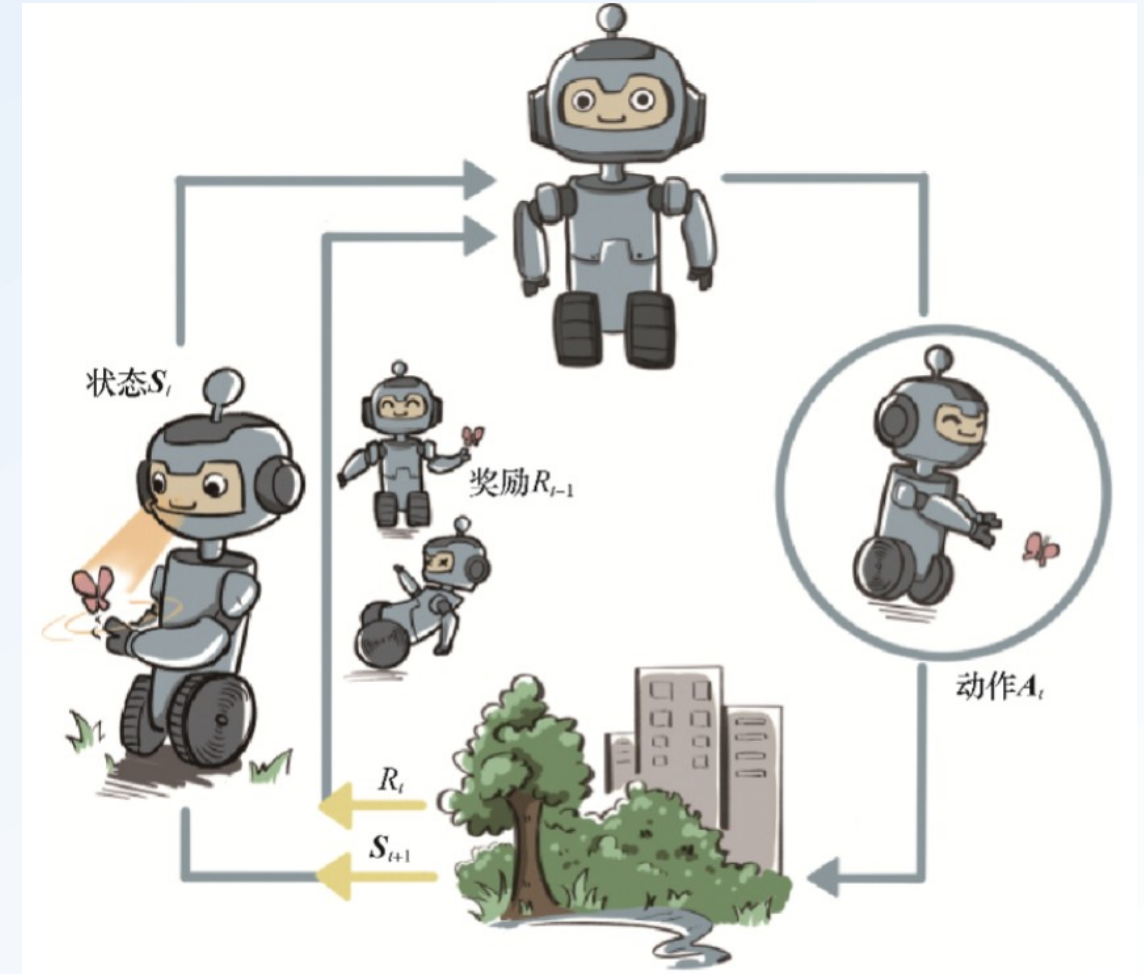
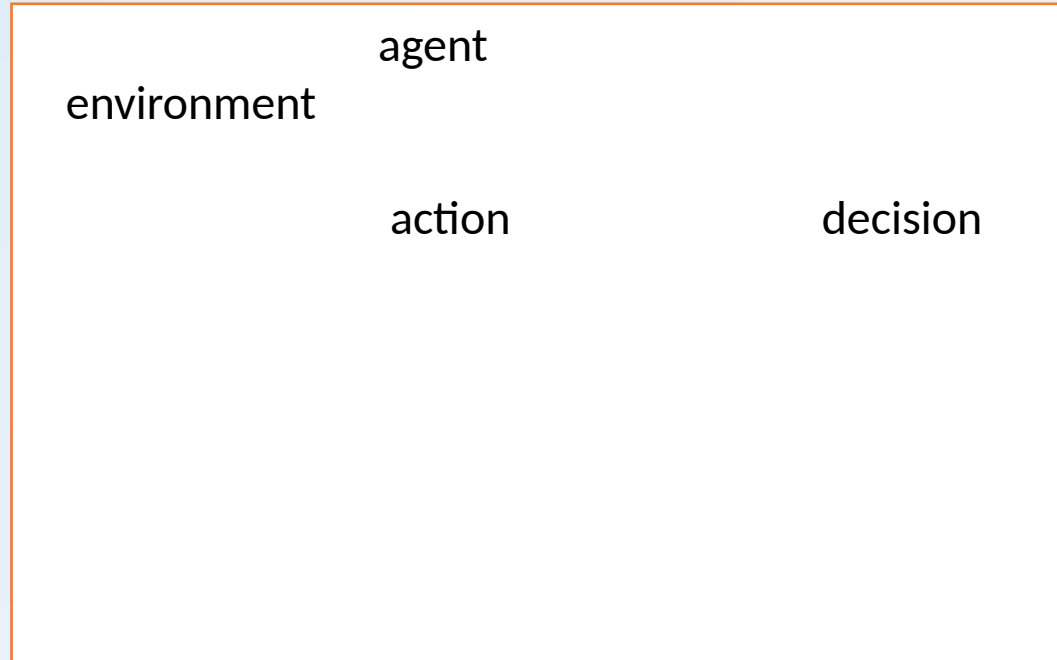
开始对话

与 DeepSeek 免费对话
体验全新旗舰模型

API 开放平台

调用 DeepSeek 最新模型
快速集成、流畅体验

Reinforcement learning



1.

Agent

" " " "

Agent

AI

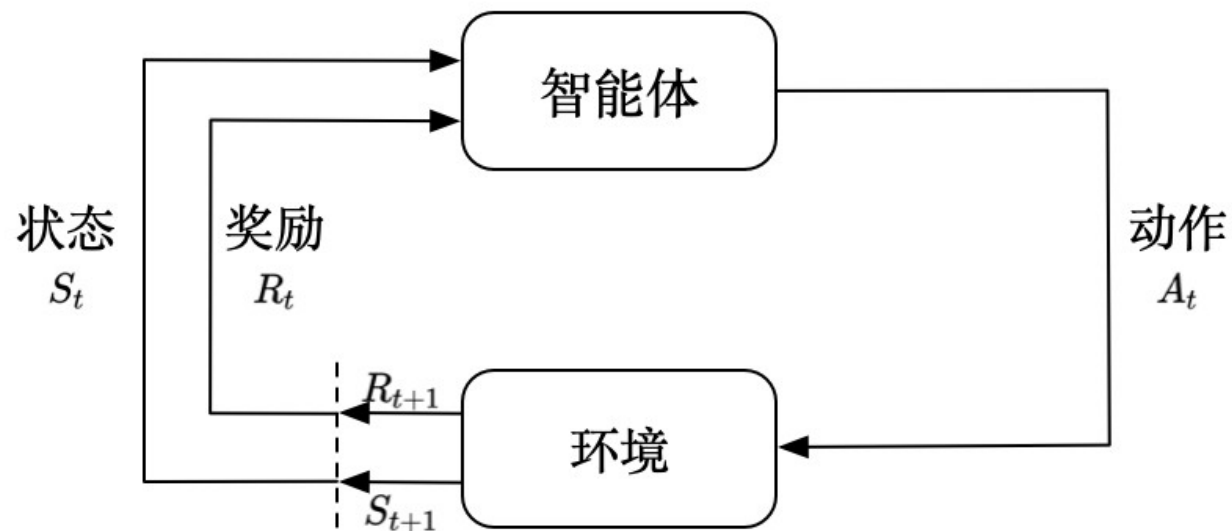
Environment

Agent

Agent

Agent

Agent



2. State

Agent

token

3. Action

Agent

" " " " " " " "

4. Reward

Reward

Agent

Agent

5. Return

Return

Agent t $r_{t+1}, r_{t+2}, r_{t+3}, \dots$ t

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

γ gamma $0 \leq \gamma \leq 1$

$\gamma = 1$ Agent $\gamma = 0$ Agent

Markov property

$$P(S_{t+1} | S_t) = P(S_{t+1} | S_1, \dots, S_t)$$

马尔可夫过程（Markov process）指具有马尔可夫性质的随机过程，也被称为马尔可夫链（Markov chain）。我们通常用元组 $\langle \mathcal{S}, \mathcal{P} \rangle$ 描述一个马尔可夫过程，其中 $\mathcal{S}$ 是有限数量的状态集合， $\mathcal{P}$ 是状态转移矩阵（state transition matrix）。假设一共有个 n 状态，此时 $\mathcal{S} = \{s_1, s_2, s_3 \dots s_n\}$ 。状态转移矩阵 $\mathcal{P}$ 定义了所有状态对之间的转移概率，即

$$\mathcal{P} = \begin{bmatrix} P(s_1|s_1) & \dots & P(s_n|s_1) \\ \vdots & \ddots & \vdots \\ P(s_1|s_n) & \dots & P(s_n|s_n) \end{bmatrix}$$

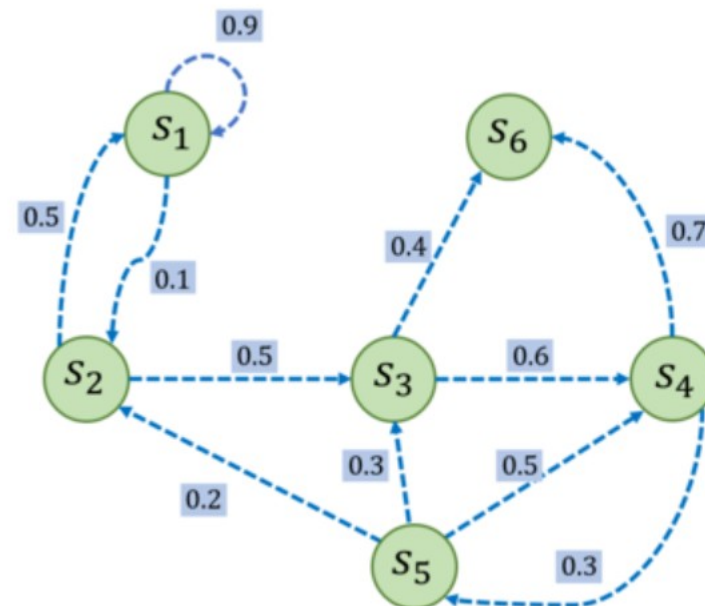


图3-1 马尔可夫过程的一个简单例子

$$\mathcal{P} = \begin{bmatrix} 0.9 & 0.1 & 0 & 0 & 0 & 0 \\ 0.5 & 0 & 0.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.6 & 0 & 0.4 \\ 0 & 0 & 0 & 0 & 0.3 & 0.7 \\ 0 & 0.2 & 0.3 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

在马尔可夫过程的基础上加入奖励函数 r 和折扣因子 γ ，就可以得到马尔可夫奖励过程（Markov reward process）。奖励函数 R 是一个期望，表示当我们到达某一个状态的时候，可以获得多大的奖励。一个马尔可夫奖励过程由 $\langle \mathcal{S}, \mathcal{P}, r, \gamma \rangle$ 构成，各个组成元素的含义如下所示。

- $\mathcal{S}$ 是有限状态的集合；
- $\mathcal{P}$ 是状态转移矩阵；
- r 是奖励函数；
- γ 是折扣因子（discount factor），的取值范围为 $[0,1)$ 。引入折扣因子的理由为远期利益具有一定不确定性，有时我们更希望能够尽快获得一些奖励，所以我们需要对远期利益打一些折扣。接近 1 的更关注长期的累计奖励，接近 0 的更考虑短期奖励。

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

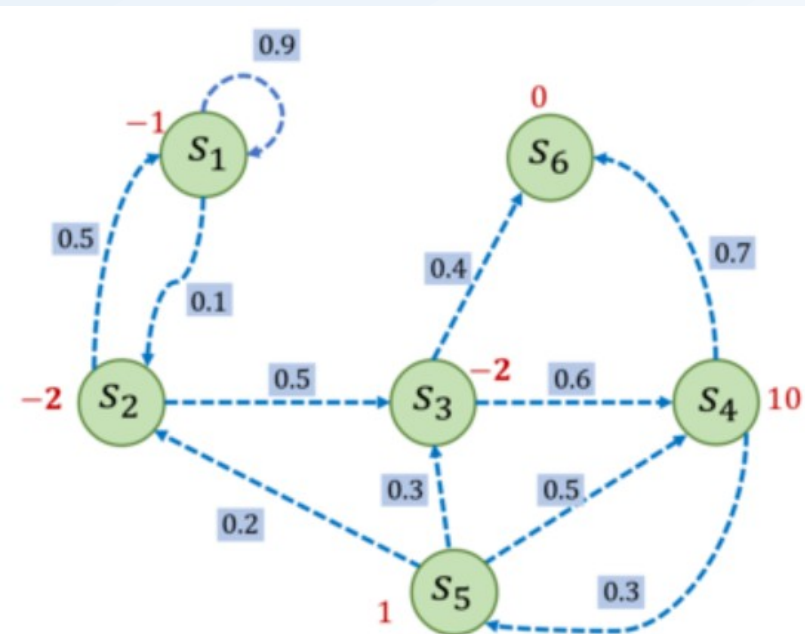


图3-2 马尔可夫奖励过程的一个简单例子

function value

value

$$V(s) = \mathbb{E}[G_t | S_t = s]$$

$$V(s) = R(s) + \gamma \sum_{s' \in S} p(s' | s) V(s')$$

$$\begin{aligned} V(s) &= \mathbb{E}[G_t | s_t = s] \\ &= \mathbb{E}[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots | s_t = s] \\ &= \mathbb{E}[r_{t+1} | s_t = s] + \gamma \mathbb{E}[r_{t+2} + \gamma r_{t+3} + \gamma^2 r_{t+4} + \dots | s_t = s] \\ &= R(s) + \gamma \mathbb{E}[G_{t+1} | s_t = s] \\ &= R(s) + \gamma \mathbb{E}[V(s_{t+1}) | s_t = s] \\ &= R(s) + \gamma \sum_{s' \in S} p(s' | s) V(s') \end{aligned}$$

$$\mathbb{E}[X | Z] = \mathbb{E}[\mathbb{E}[X | Y, Z] | Z]$$

$$\begin{aligned} \mathbb{E}[G_{t+1} | S_t = s_t] &= \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1} = s_{t+1}, S_t = s_t] | S_t = s_t] \\ &= \mathbb{E}[\mathbb{E}[G_{t+1} | S_{t+1} = s_{t+1}] | S_t = s_t] \\ &= \mathbb{E}[V(s_{t+1}) | s_t = s] \end{aligned}$$

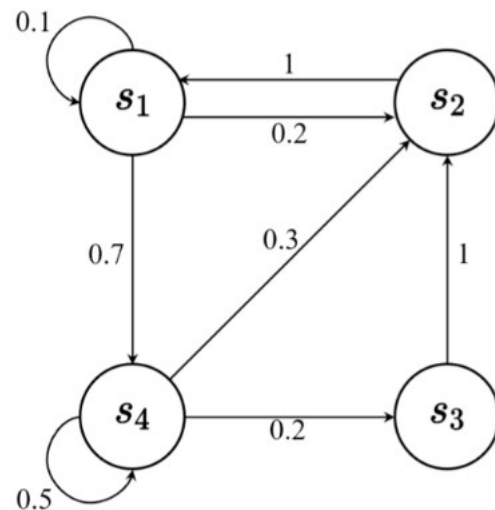
这就是大名鼎鼎的——贝尔曼方程（**Bellman equation**）：

$$V(s) = \underbrace{R(s)}_{\text{即时奖励}} + \underbrace{\gamma \sum_{s' \in S} p(s' | s) V(s')}_{\text{未来奖励的折扣总和}}$$

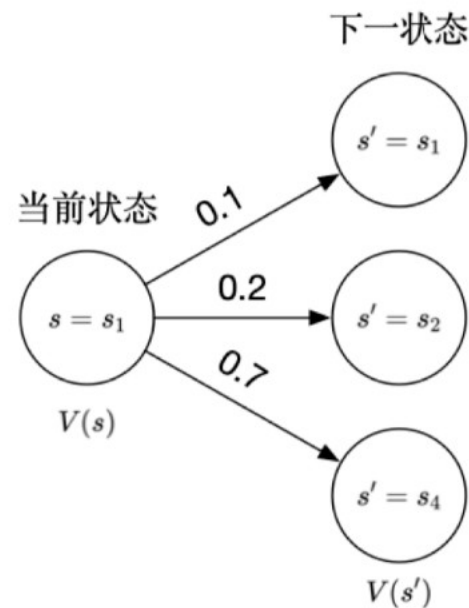
其中，

- s' 可以看成未来的所有状态，
- $p(s' | s)$ 是指从当前状态转移到未来状态的概率。
- $V(s')$ 代表的是未来某一个状态的价值。我们从当前状态开始，有一定的概率去到未来的所有状态，所以我们要把 $p(s' | s)$ 写上去。我们得到了未来状态后，乘一个 γ ，这样就可以把未来的奖励打折扣。
- $\gamma \sum_{s' \in S} p(s' | s) V(s')$ 可以看成未来奖励的折扣总和（discounted sum of future reward）。

贝尔曼方程定义了当前状态与未来状态之间的关系。未来奖励的折扣总和加上即时奖励，就组成了贝尔曼方程。



(a) 马尔可夫链



(b) 状态转移示例

s1

3



$$\begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix} = \begin{pmatrix} R(s_1) \\ R(s_2) \\ \vdots \\ R(s_N) \end{pmatrix} + \gamma \begin{pmatrix} p(s_1 | s_1) & p(s_2 | s_1) & \dots & p(s_N | s_1) \\ p(s_1 | s_2) & p(s_2 | s_2) & \dots & p(s_N | s_2) \\ \vdots & \vdots & \ddots & \vdots \\ p(s_1 | s_N) & p(s_2 | s_N) & \dots & p(s_N | s_N) \end{pmatrix} \begin{pmatrix} V(s_1) \\ V(s_2) \\ \vdots \\ V(s_N) \end{pmatrix}$$

我们当前的状态是向量 $[V(s_1), V(s_2), \dots, V(s_N)]^T$ 。每一行来看，向量 V 乘状态转移矩阵里面的某一行，再加上它当前可以得到的奖励，就会得到它当前的价值。

当我们把贝尔曼方程写成矩阵形式后，可以直接求解：

$$\begin{aligned} V &= R + \gamma P V \\ IV &= R + \gamma P V \\ (I - \gamma P)V &= R \\ V &= (I - \gamma P)^{-1} R \end{aligned}$$

analytic solution

$$V = (I - \gamma P)^{-1} R$$

前面的马尔可夫过程和马尔可夫奖励过程都是自发改变的随机过程；而如果有一个外界的“刺激”来共同改变这个随机过程，就有了马尔可夫决策过程（Markov decision process, MDP）。我们将这个来自外界的刺激称为智能体（agent）的动作，在马尔可夫奖励过程（MRP）的基础上加入动作，就得到了马尔可夫决策过程（MDP）。马尔可夫决策过程由元组 $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma \rangle$ 构成，其中：

- $\mathcal{S}$ 是状态的集合；
- $\mathcal{A}$ 是动作的集合；
- r 是折扣因子；
- γ 是奖励函数，此时奖励可以同时取决于状态和动作，在奖励函数只取决于状态时，则退化为；
- $\mathcal{P}$ 是状态转移函数，表示在状态执行动作之后到达状态的概率。

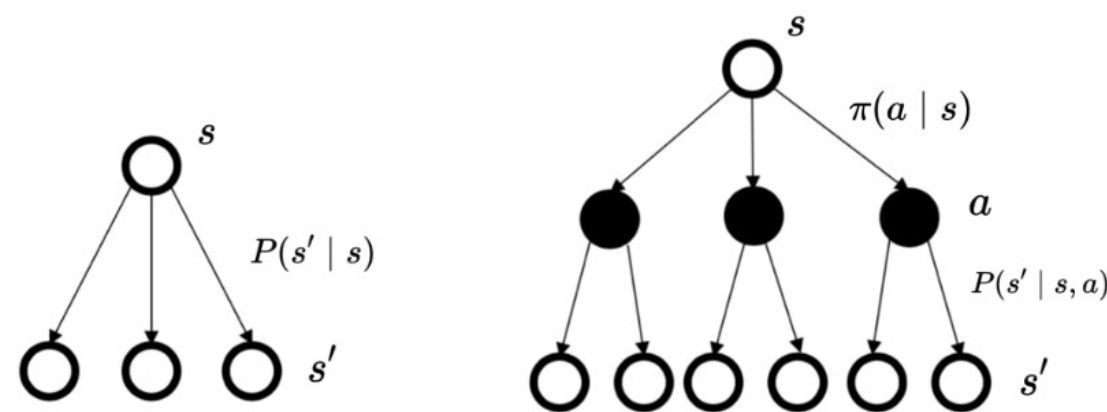
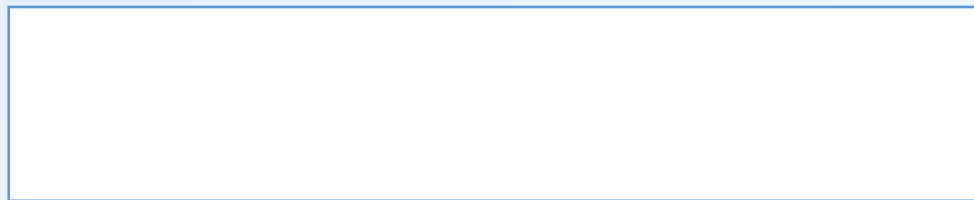
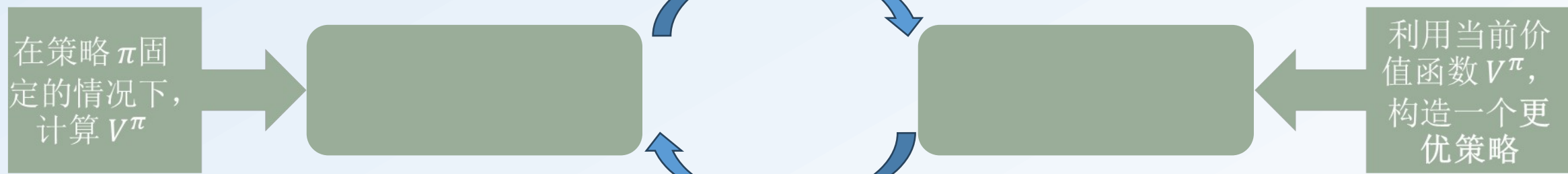


图 2.9 马尔可夫决策过程与马尔可夫过程/马尔可夫奖励过程的状态转移的对比

“ ” “ ”

$\pi^0 \xrightarrow{\text{策略评估}} V^{\pi^0} \xrightarrow{\text{策略提升}} \pi^1 \xrightarrow{\text{策略评估}} V^{\pi^1} \xrightarrow{\text{策略提升}} \pi^2 \xrightarrow{\text{策略评估}} \dots \xrightarrow{\text{策略提升}} \pi^*$

→ → →



$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

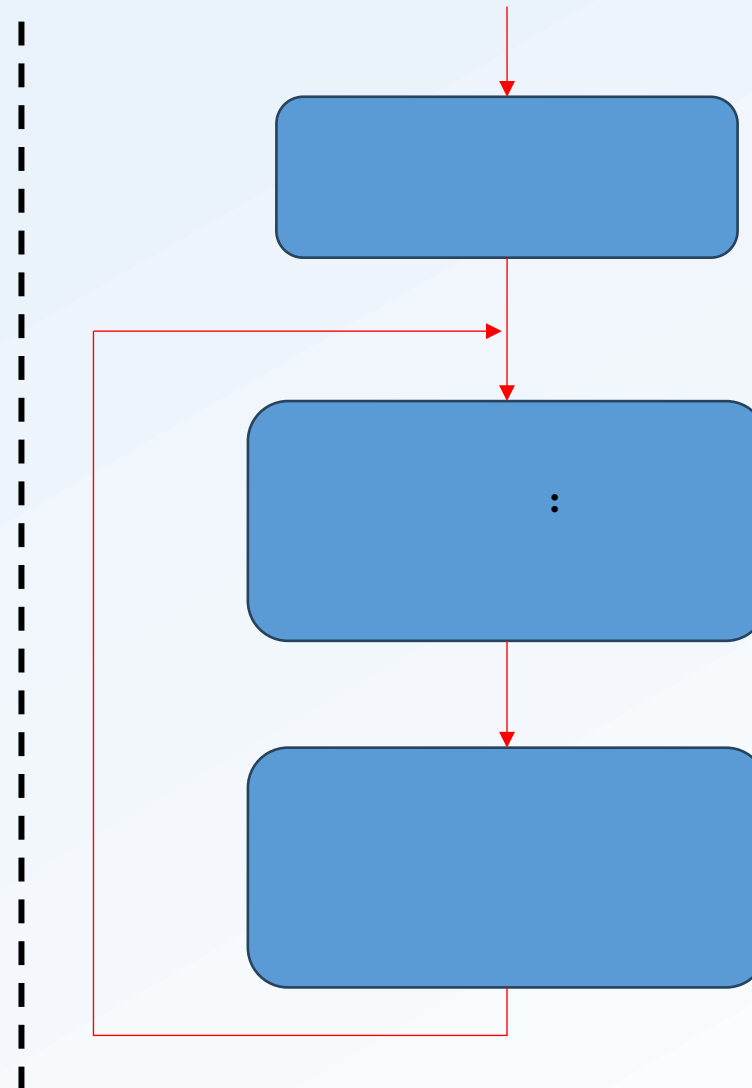
$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')]$$

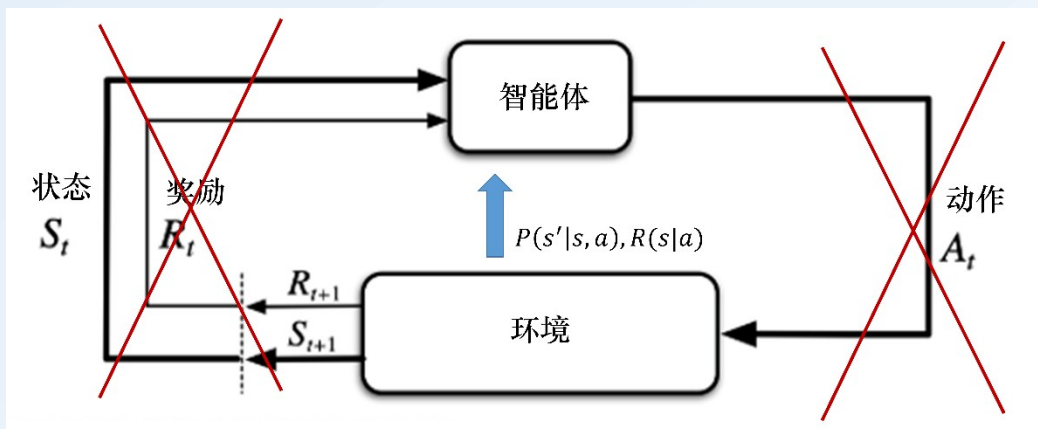
$$Q^{\pi}(s, a) = \sum P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

$$\pi'(s) = \arg \max_a Q^{\pi}(s, a)$$

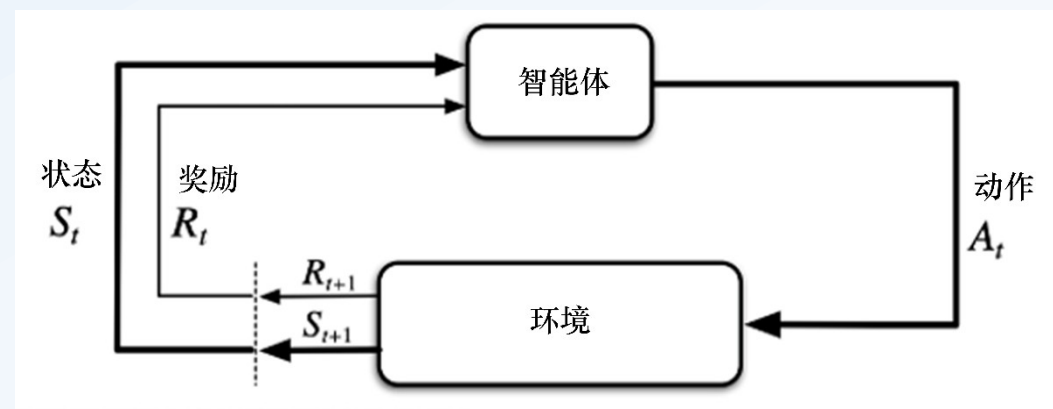
这里能保证新策略 π' 能够比旧策略 π 更好或者至少与其一样好。当策略提升之后得到的策略和之前的策略一样时, 说明策略迭代达到了收敛, 此时 π' 就是最优策略。

- 随机初始化策略 $\pi(s)$ 和价值函数 $V(s)$
- **while** $\Delta > \theta$ **do**: (策略评估循环)
- $\Delta \leftarrow 0$
- 对于每一个状态 $s \in \mathcal{S}$:
- $v \leftarrow V(s)$
- $V(s) \leftarrow r(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V(s')$
- $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
- **end while**
- $\pi_{\text{old}} \leftarrow \pi$
- 对于每一个状态 $s \in \mathcal{S}$:
- $\pi(s) \leftarrow \arg \max_a r(s, a) + \gamma \sum_{s'} P(s'|s, a) V(s')$
- 若 $\pi_{\text{old}} = \pi$, 则停止算法并返回 V 和 π ; 否则转到策略评估循环





1



2



——Sarsa

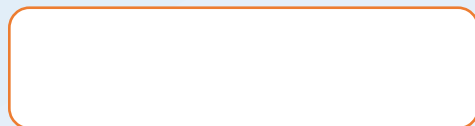
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \underbrace{\gamma Q(s_{t+1}, a_{t+1})}_{\text{TD}} - \underbrace{Q(s_t, a_t)}_{\text{Q}}]$$

Q
Q

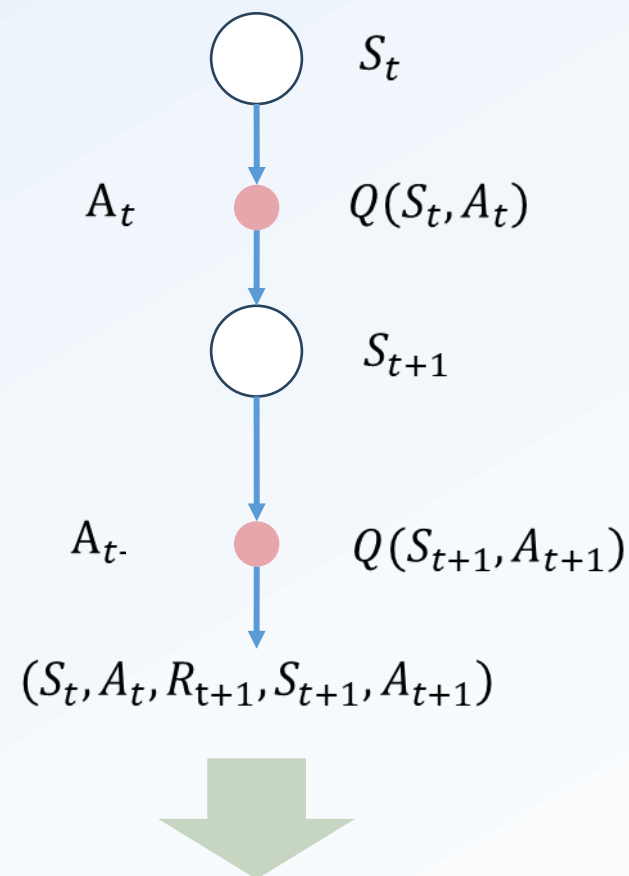
总得来说就是用下一步的Q值 $Q(s_{t+1}, a_{t+1})$ 来更新这一步的Q值 $Q(s_t, a_t)$ 。

采用软更新的方法，就是每次我们只更新一点点。

Sarsa 直接估计Q表格，得到Q表格后，我们就可以更新策略。



- 初始化 $Q(s, a)$
- **for** 序列 $e = 1 \rightarrow E$ **do**:
- 得到初始状态 s
- 用 ϵ -greedy 策略根据 Q 选择当前状态 s 下的动作 a
- **for** 时间步 $t = 1 \rightarrow T$ **do**:
- 得到环境反馈的 r, s'
- 用 ϵ -greedy 策略根据 Q 选择当前状态 s' 下的动作 a'
- $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
- $s \leftarrow s', a \leftarrow a'$
- **end for**
- **end for**





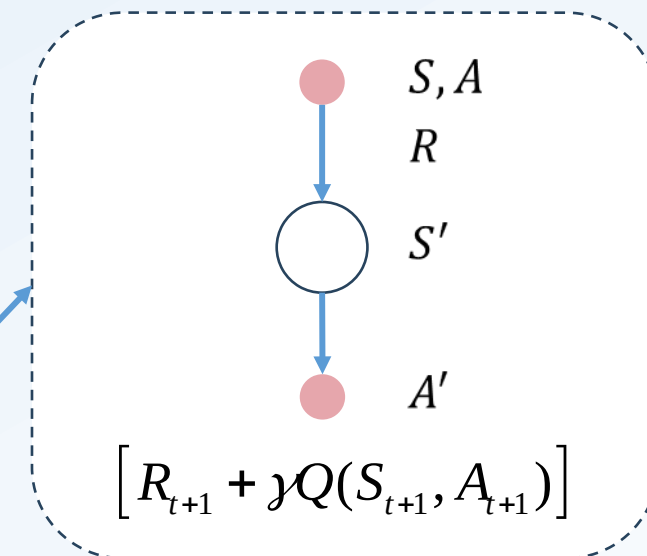
Sarsa learning target Sarsa Q-learning Q-learning TD

Q-learning

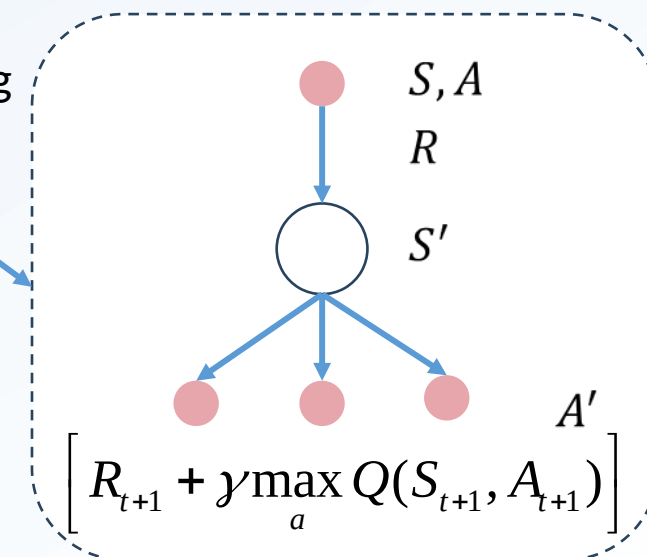
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

Q学习不使用实际执行的下一个动作 A_{t+1} 的Q值，而是使用在下一个状态 S_{t+1} 中**最大化**Q函数估算值的动作 a' 的Q值。这隐含地假设从状态 S_{t+1} 开始将采取最优（贪婪）动作，从而有效地学习最优策略 π^* ，同时可能遵循一个不同的、更具探索性的行为策略（如 ϵ -贪婪策略）。

Sarsa



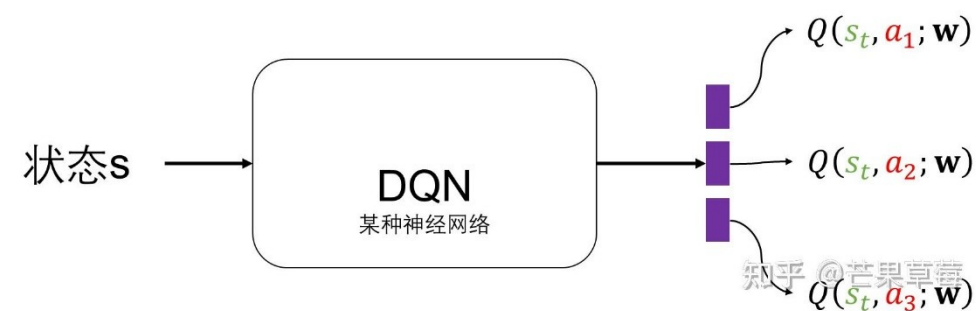
Q-Learning



Q-learning

DQN

DQN: 基本原理



Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *nature* 518.7540 (2015): 529-533.

强化学习的目标就是要最大化交互过程中累积的折扣奖励。

强化学习的目标函数就是最大化轨迹回报的期望：

$$\bar{R}_\theta = \sum_{\tau} R(\tau) p_\theta(\tau) = \mathbb{E}_{\tau \sim p_\theta(\tau)} [R(\tau)]$$

其中 $\tau = (s_0, a_0, s_1, a_1 \dots)$ 是一条轨迹。

一条轨迹对应的累积奖励为：
$$R(\tau) = \sum_{t=0}^T \gamma^t r_t$$

$$\max_{\theta} J(\theta)$$

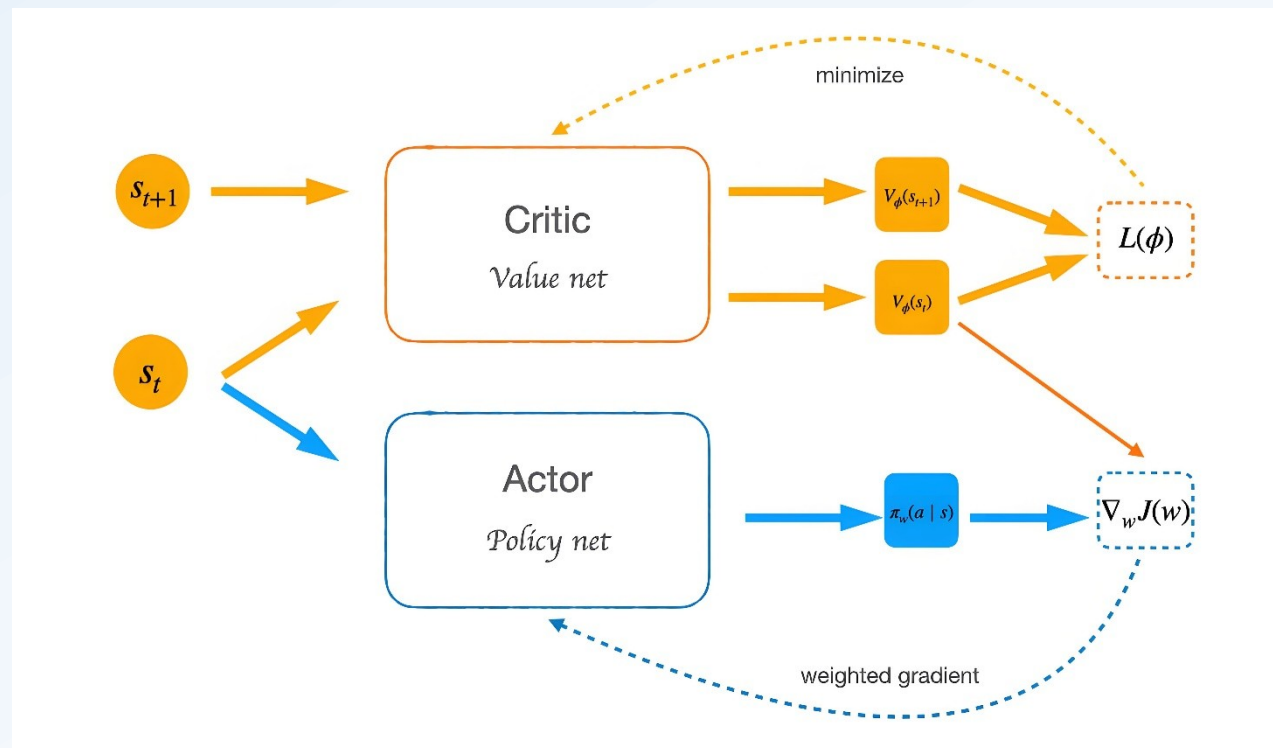
Actor-Critic



Actor Critic算法的核心是“分工合作”

它有两个网络，同时结合了策略梯度方法和值函数估计：

- **Actor**: 负责“执行动作”，也就是学习策略 $\pi_{\theta}(a|s)$ ，和我们之前讲的策略梯度的策略一样，输出动作的概率。
- **Critic**（评论家）：负责“评估动作”，也就是学习价值函数 $V_{\phi}(s)$ ，评估当前状态 s 的价值，给 **Actor** 的动作打分。



Input:

学习率 α_{actor} , α_{critic} , 折扣因子 γ

Initialize:

初始化策略网络参数 θ (Actor)

初始化价值网络参数 ϕ (Critic)

for episode = 1, 2, ..., N do

初始化环境, 获得初始状态 s

while s 不是终止状态 do

1. 根据策略选择动作

根据当前策略 $\pi_{\theta}(a|s)$ 采样动作 a , 记录 $\log_{\text{prob}}$

$= \log_{\pi_{\theta}}(a | s)$

2. 与环境交互

执行动作 a , 获得奖励 r 和下一状态 s'

3. 计算 TD 误差

评估下一个状态的价值: $V(s') = \text{Critic}(s'; \phi)$

评估当前状态的价值: $V(s) = \text{Critic}(s; \phi)$

计算时序差分误差 $\delta = r + \gamma * V(s') - V(s)$

4. 更新 Critic (最小化 TD 误差)

$L_{\text{critic}} = \delta^2$

$\phi \leftarrow \phi - \alpha_{\text{critic}} * \nabla_{\phi} L_{\text{critic}}$

5. 更新 Actor (策略梯度)

$\theta \leftarrow \theta + \alpha_{\text{actor}} * (\nabla_{\theta} \log_{\text{prob}}) * \delta$

6. 状态转移

$s \leftarrow s'$

end while

end for

① TD误差的作用 $\delta = r + \gamma V(s') - V(s)$ 它在这里同时承担两个角色: 用于更新 Critic (让价值更准确) 作为 Advantage 的估计, 指导 Actor 更新

② Actor 更新的本质 $\theta \leftarrow \theta + \alpha * \nabla \log \pi(a|s) * \delta$ 可以这样讲给学生:

$\delta > 0 \rightarrow$ 这个动作比预期好 $\rightarrow$ 概率 $\uparrow$

$\delta < 0 \rightarrow$ 这个动作比预期差 $\rightarrow$ 概率 $\downarrow$

③ 和 REINFORCE 的关键区别 REINFORCE: 用 G_t (完整回报) Actor-Critic: 用 δ (一步 TD 误差)  本质: $G_t \rightarrow$ 高方差 (Monte Carlo) $\delta \rightarrow$ 低方差 (Bootstrapping)

PPO Proximal Policy Optimization

“ ”



Actor

Critic

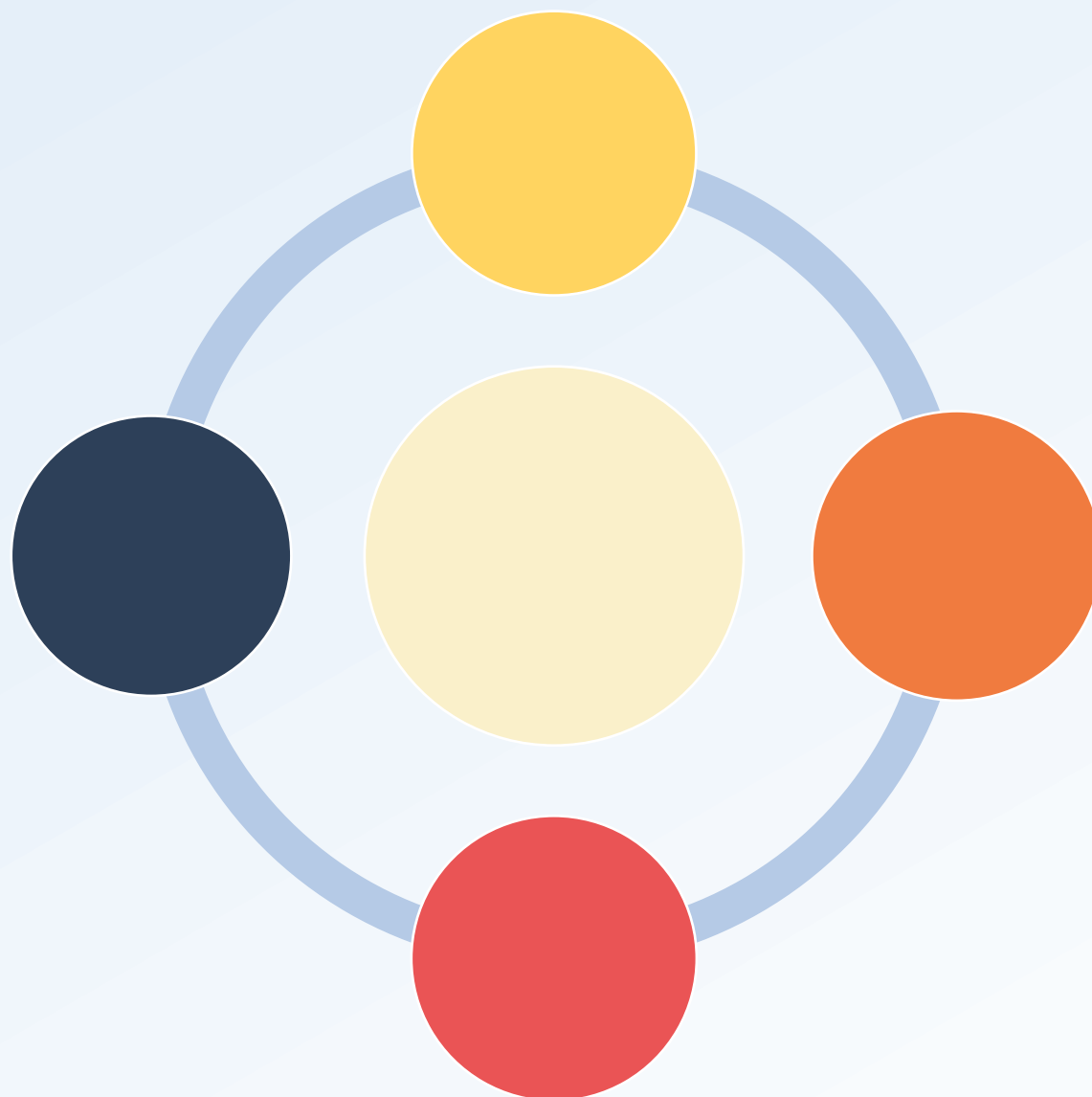
PPO

“ ”



“ ”

Schulman, John, et al. "Proximal policy optimization algorithms." *arXiv preprint arXiv:1707.06347* (2017).



1.

-
-
-

DoE

Deep Q-learning for optimal scouting runs

DQN / /

Deep RL for chromatography control

deep RL
flow rate
column control policy

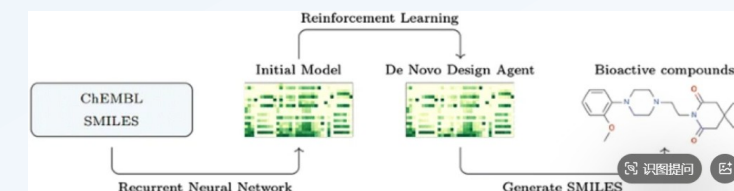
Kensert, Alexander, et al. "Deep Q-learning for the selection of optimal isocratic scouting runs in liquid chromatography." *Journal of Chromatography A* 1638 (2021): 461900.
Andersson, David, et al. "Adaptable control policies for variable liquid chromatography columns using deep reinforcement learning." *Scientific Reports* 13.1 (2023): 11270.

2.

REINVENT: RL for de novo drug design

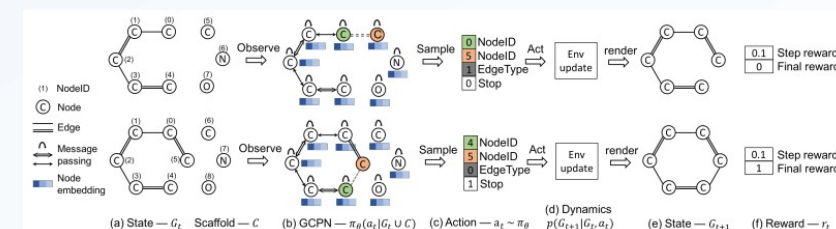
RNN SMILES RL
reward ADMET

policy gradient
prior model + RL fine-tune



MolDQN: Molecular Optimization Using Deep RL

SMILES



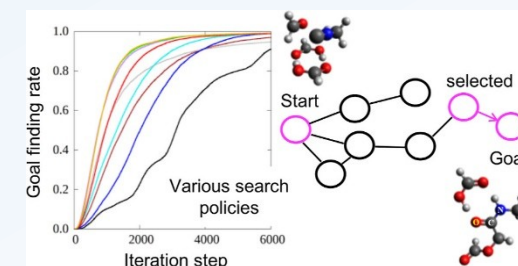
Olivecrona, Marcus, et al. "Molecular de-novo design through deep reinforcement learning." *Journal of cheminformatics* 9.1 (2017): 48.

You, Jiaxuan, et al. "Graph convolutional policy network for goal-directed molecular graph generation." *Advances in neural information processing systems* 31 (2018).

3.

Comparative Analysis of Reinforcement Learning Algorithms for Finding Reaction Pathways

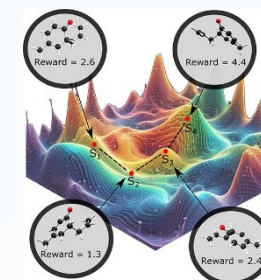
RL
state
action
reward
RL



Reinforcement Learning for Traversing Chemical Structure Space

RL agent “ ”

RL Nudged Elastic Band NEB

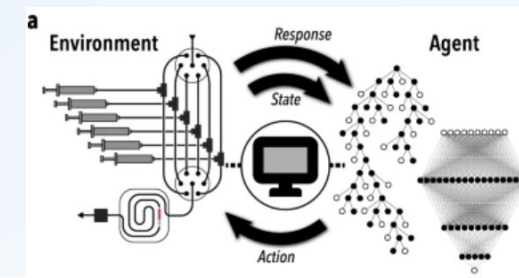


Matsumura, Yoshihiro, Koji Tabata, and Tamiki Komatsuzaki. "Comparative Analysis of Reinforcement Learning Algorithms for Finding Reaction Pathways: Insights from a Large Benchmark Data Set." *Journal of Chemical Theory and Computation* 21.7 (2025): 3523-3535. Barrett, Rhyann, and Julia Westermayr. "Reinforcement learning for traversing chemical structure space: Optimizing transition states and minimum energy paths of molecules." *The Journal of Physical Chemistry Letters* 15.1 (2024): 349-356.

4.

ChemGymRL: An Interactive Framework for Reinforcement Learning for Digital Chemistry

RL

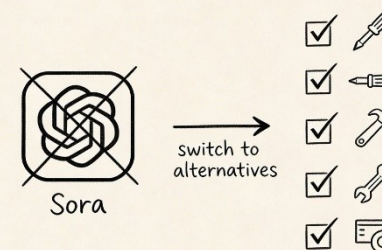


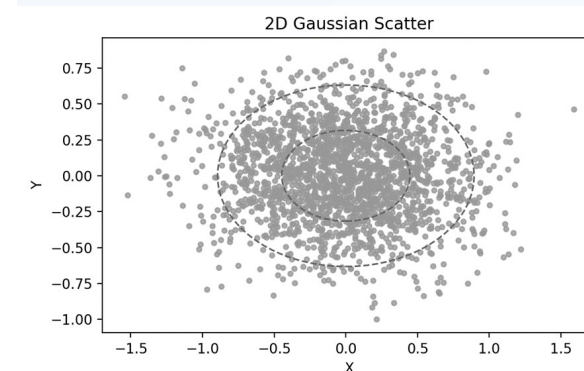
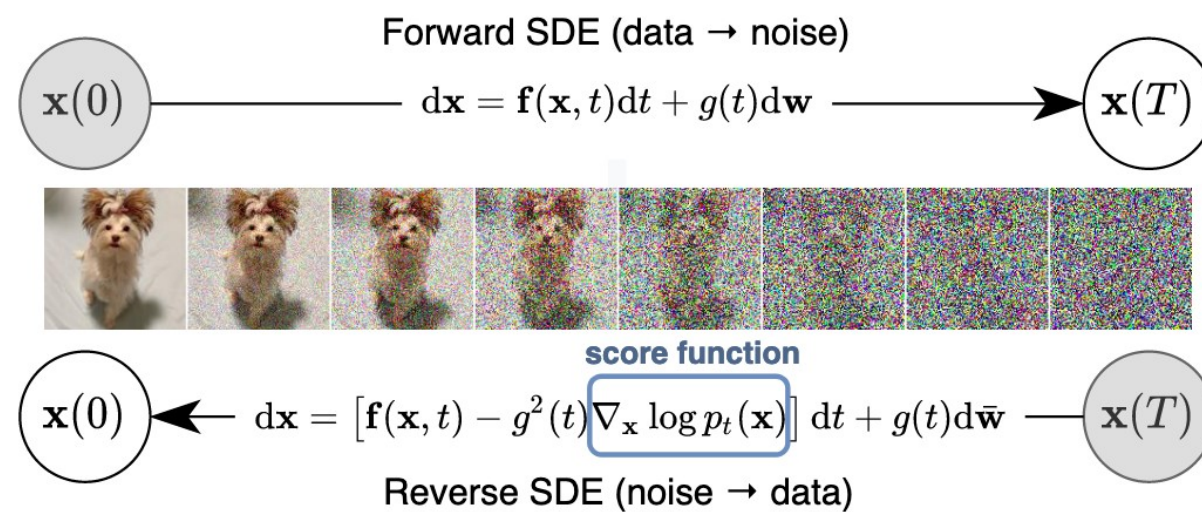
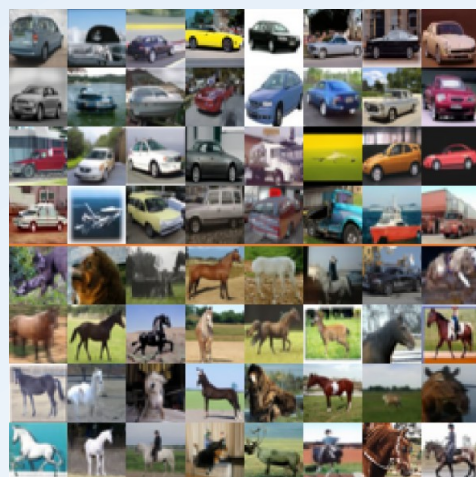
AlphaFlow: autonomous discovery and optimization of multi-step chemistry

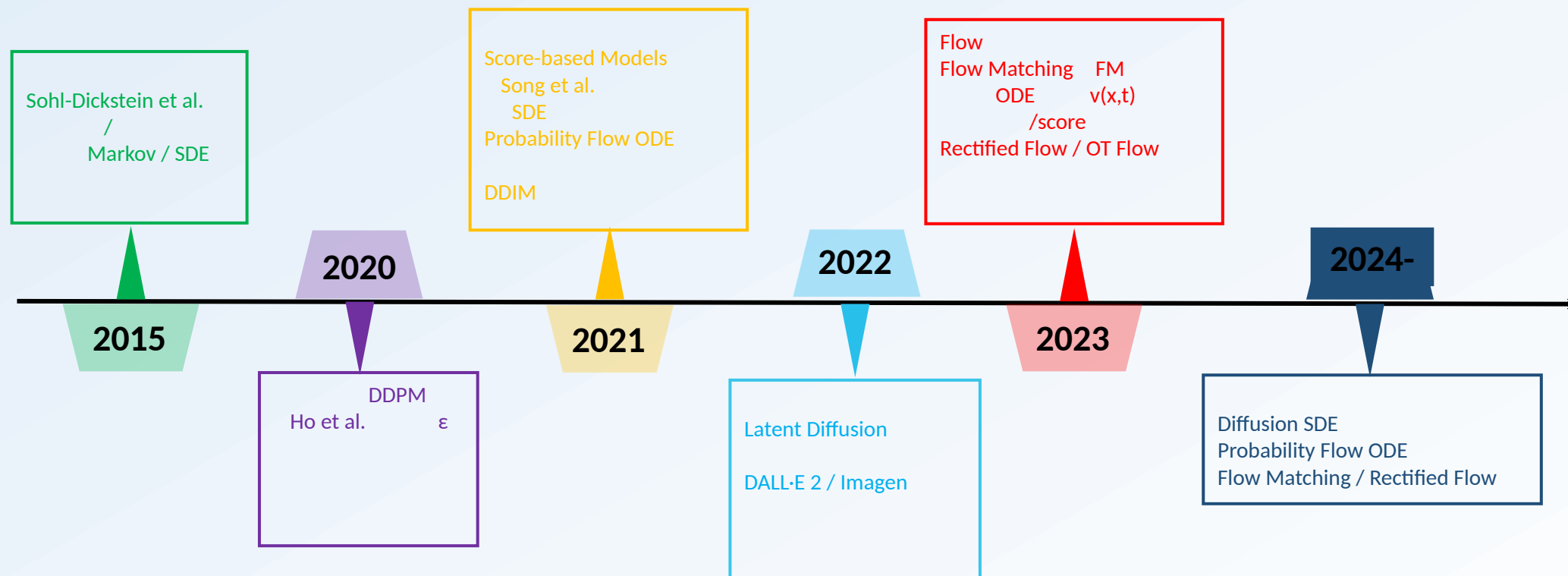
- RL +
- RL “ ”

Volk, Amanda A., et al. "AlphaFlow: autonomous discovery and optimization of multi-step chemistry using a self-driven fluidic lab guided by reinforcement learning." *Nature Communications* 14.1 (2023): 1403.
 Beeler, Chris, et al. "Chemgymrl: An interactive framework for reinforcement learning for digital chemistry." *arXiv preprint arXiv:2305.14177* (2023).

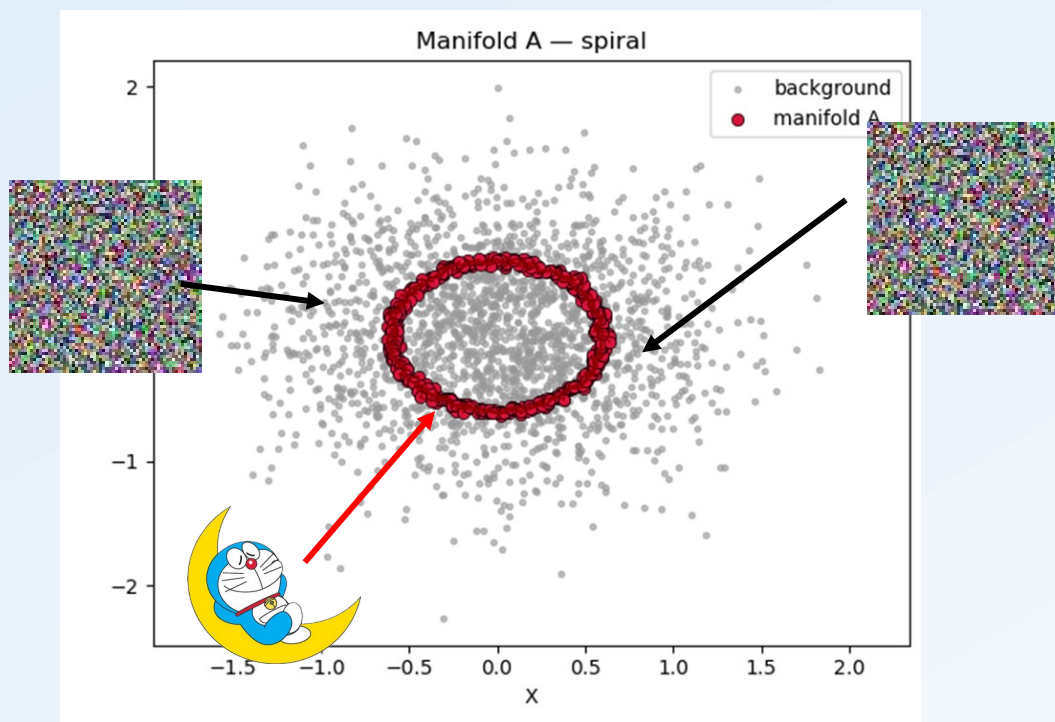
(AI Generated Content)





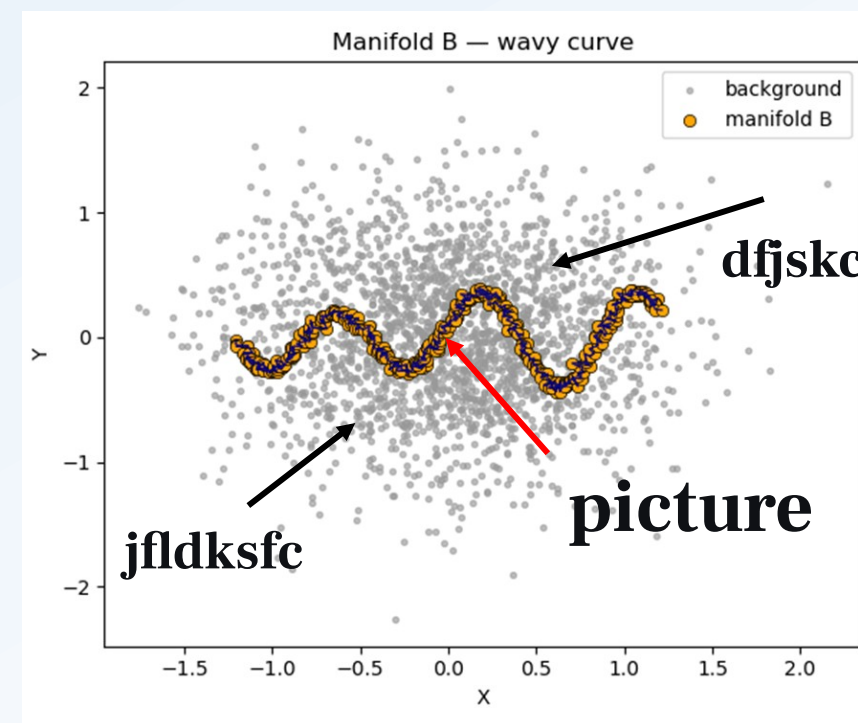


“ ” “ ”

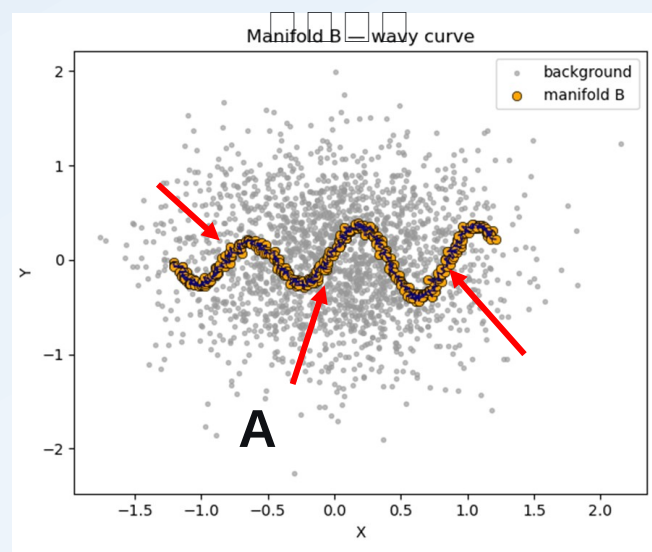
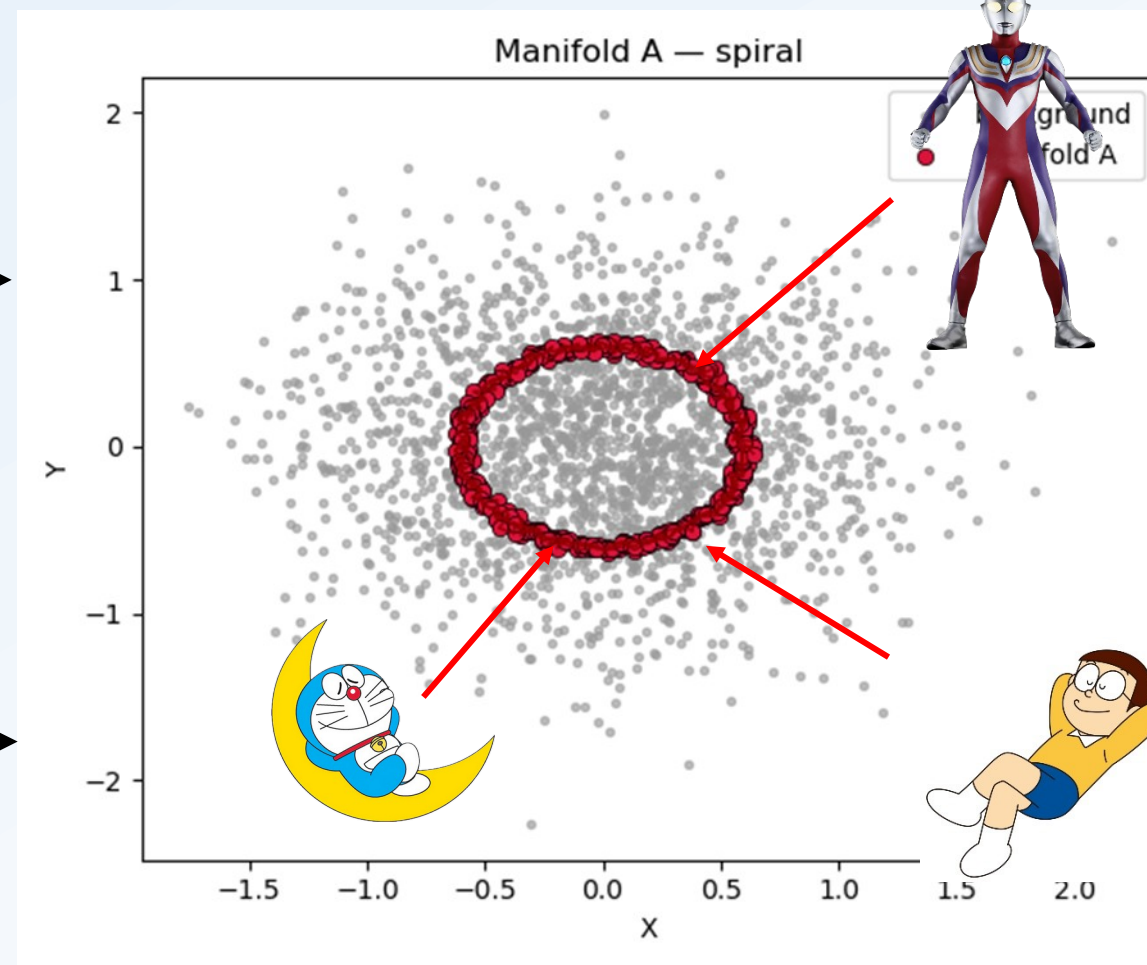
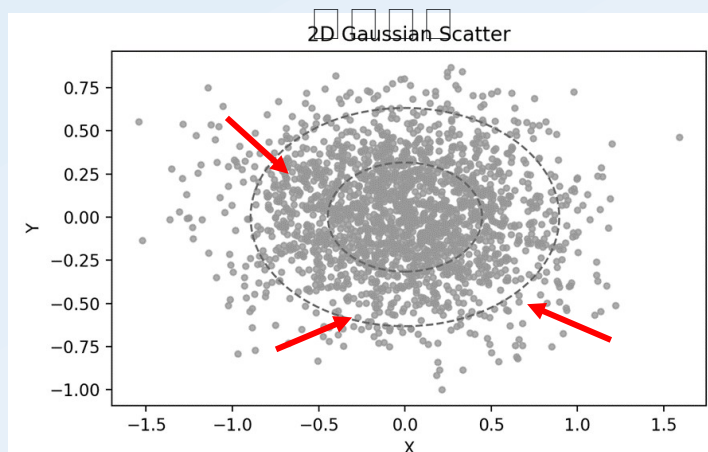


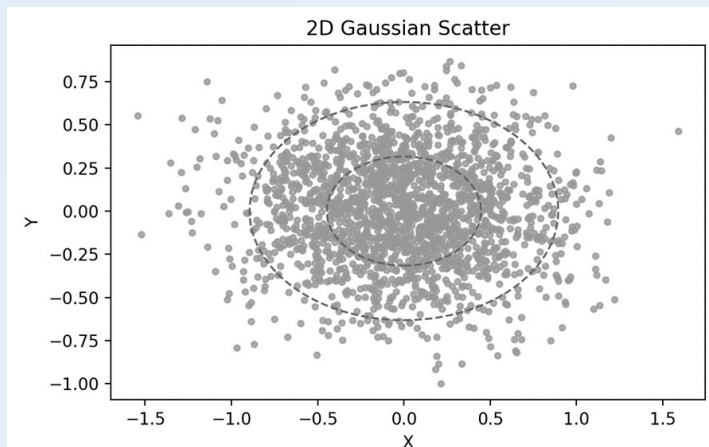
RGB

“ ”



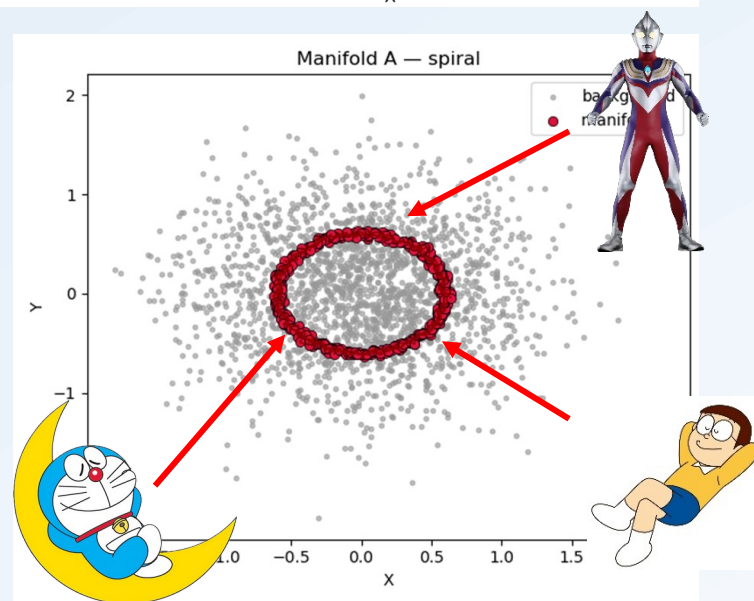
“





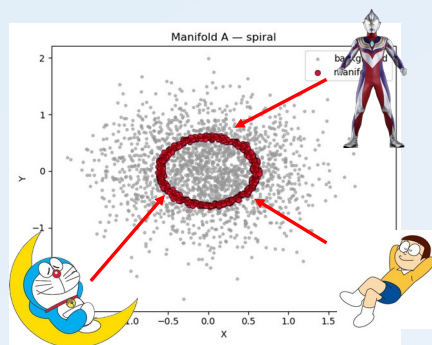
□ □ □ □ □ □

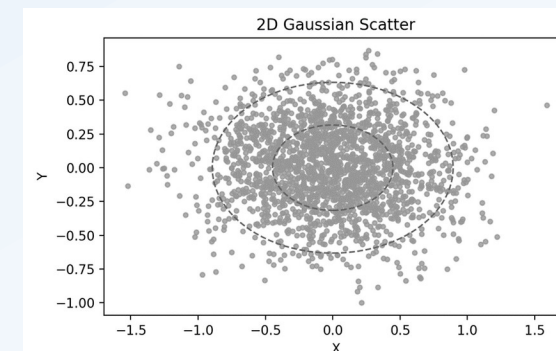
$$p(x) = \frac{1}{2\pi\sqrt{|\Sigma|}} \exp\left[-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right]$$



□ □ □ □ □ □

$$p(x) \rightarrow p_{data}(x)$$


 $p_{data}(x)$

 $p_t(x)$

 $p_{gau}(x)$

$$p_{data}(x)$$

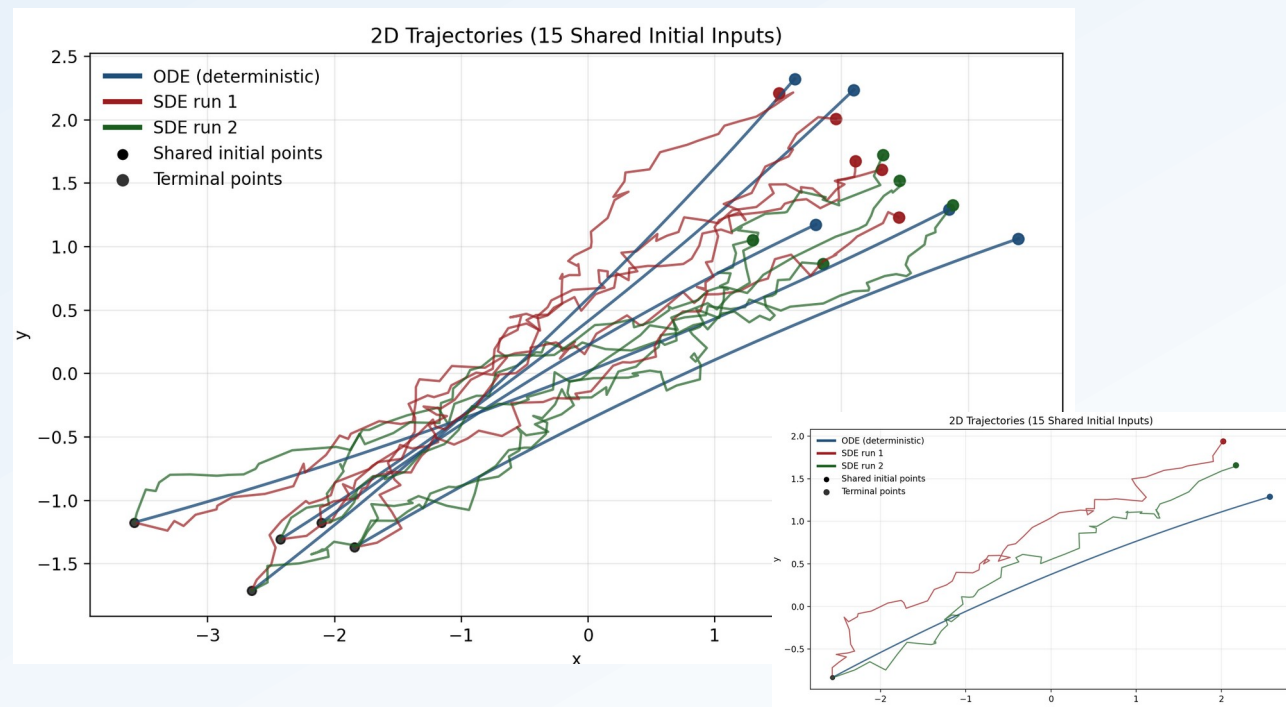
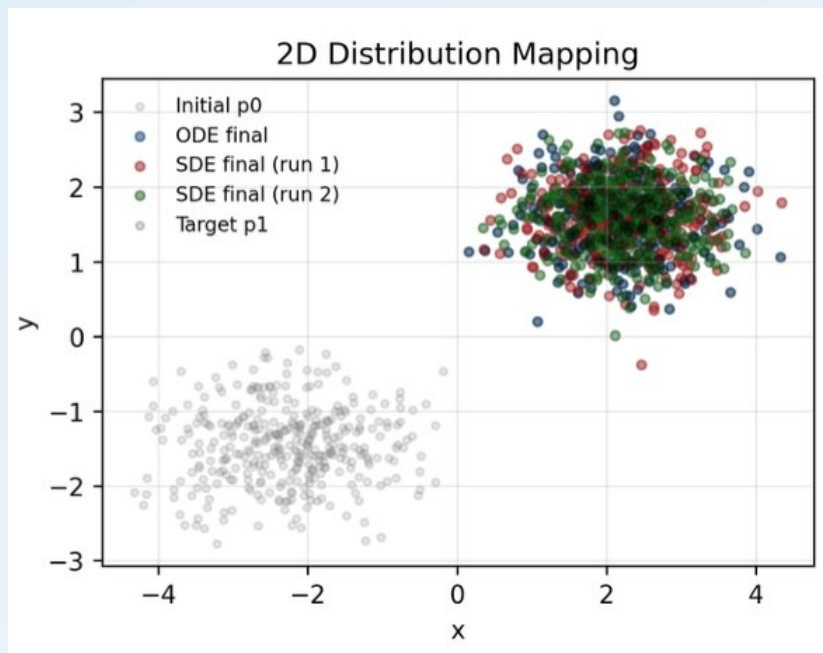
$$p_{gau}(x)$$

$$p_t(x), t \in [0, T]$$

$$dx = f(x, t)dt$$

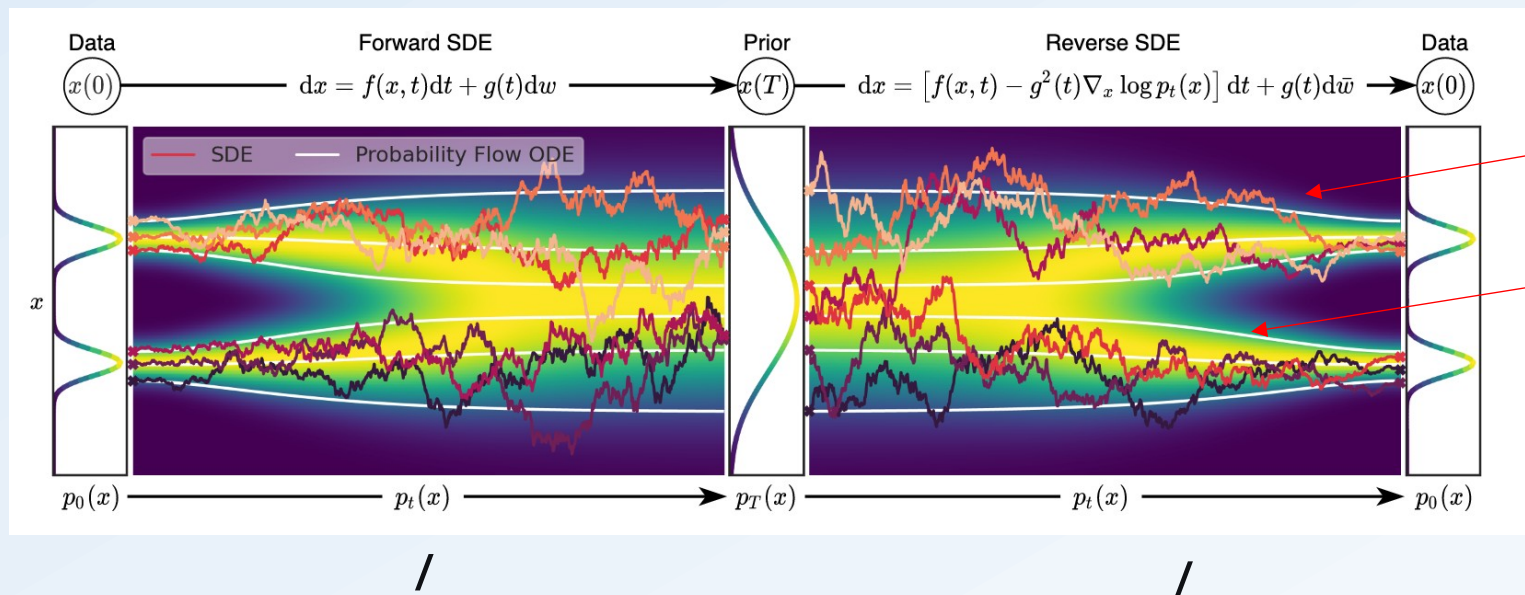
$$dx = f(x, t)dt + g(t)d\omega$$

x : 表示随机变量



$$dx = f(x, t)dt$$

$$dx = f(x, t)dt + g(t)d\omega$$



$$d\mathbf{x} = f(\mathbf{x}, t)dt + g(t)d\omega$$

$$d\mathbf{x} = [f(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\omega$$

$$d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{1}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt \quad d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{3}{2} g^2(t) \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

(SDE)

DDPM- (2020)

Denoising Diffusion
Probabilistic Models

NCSN- (2019)

Noise Conditional Score
Network

SDE- (2021)

Score-Based Generative
Model

EDM- (2022)

Elucidating the Design Space
of Diffusion-Based
Generative Models

(ODE)

EDM- (2022)

Elucidating the Design Space
of Diffusion-Based
Generative Models

DDIM- (2022)

Denoising Diffusion Implicit
Models

FM- (2022)

Flow Matching for
generative modeling

RF- (2022)

Rectified Flow

CM- (2023)

Consistency Models

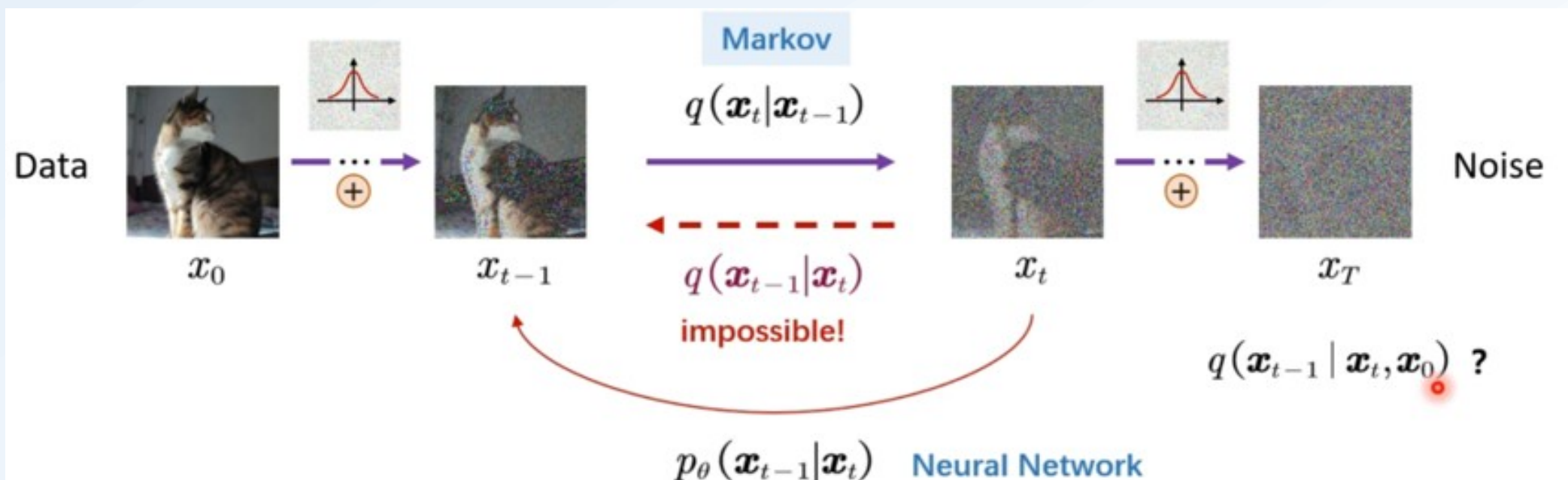
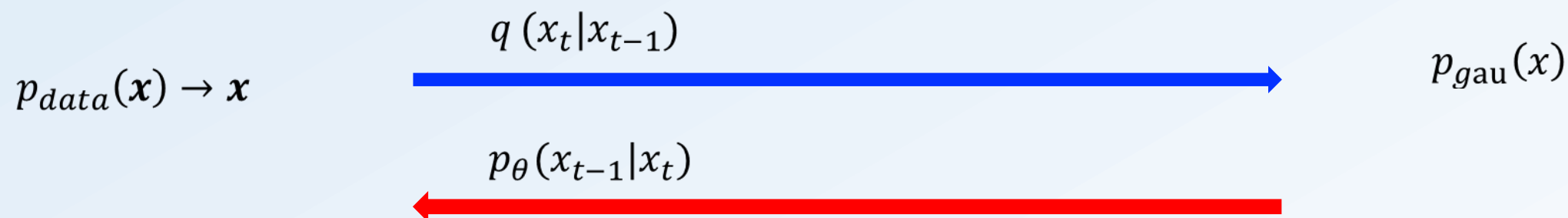
sCM- (2023)

Simplifying stabilizing &
scaling continuous time
consistency models

JiT- (2026)

Back to Basics: Let Denoising
Generative Models Denoise

DDPM



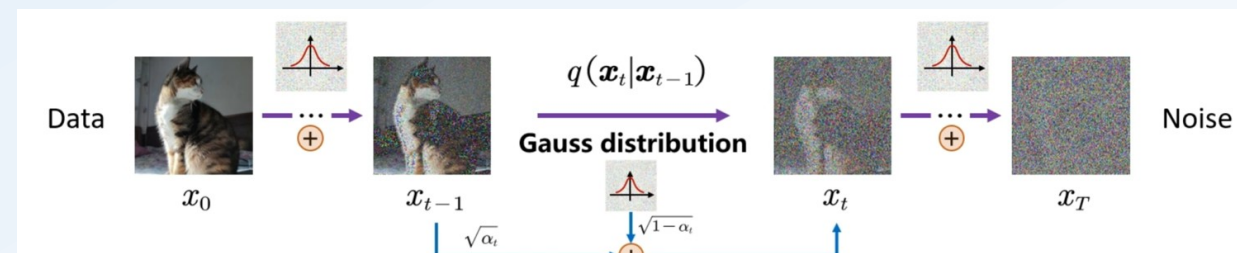
$q(x_t|x_{t-1})$: 表示以 x_{t-1} 为条件的时候, x_t 的分布。

$$q(x_t|x_{t-1}) = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t \quad \epsilon_t: \text{标准高斯分布}$$

$$q(x_1|x_0) = \sqrt{\alpha_1}x_0 + \sqrt{1-\alpha_1}\epsilon_1 \quad x_0: \text{初始图像}$$

α_t : 超参数

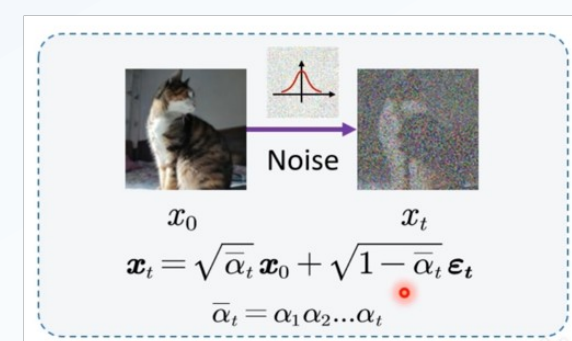
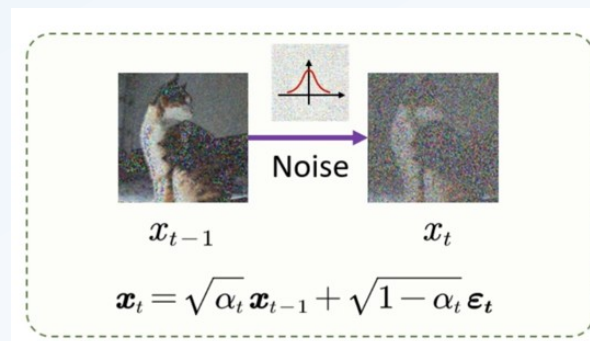
虽然我们无法得到图像分布 $p_{data}(x)$, 但是我们可以指定其演化的随机过程



$$\text{gaussian} = \sqrt{\alpha_t} \text{signal} + \sqrt{1-\alpha_t} \text{noise}$$

$$q(x_t|x_{t-1}) = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t$$

$$\begin{aligned} q(x_{t+1}|x_{t-1}) &= \sqrt{\alpha_{t+1}}(\sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t) + \sqrt{1-\alpha_{t+1}}\epsilon_{t+1} \\ &= \sqrt{\alpha_{t+1}\alpha_t}x_{t-1} + \sqrt{1-\alpha_t\alpha_{t+1}}\epsilon \end{aligned}$$



$$P_{t+1} \sim N(\mu_{t+1}, \sigma_{t+1}^2), P_t \sim N(\mu_t, \sigma_t^2) \rightarrow P = P_{t+1} + P_t = N(\mu_{t+1} + \mu_t, \sigma_t^2 + \sigma_{t+1}^2)$$

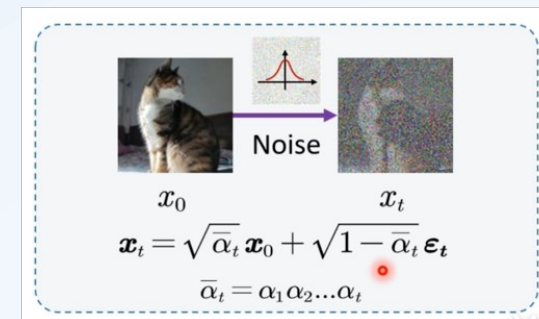
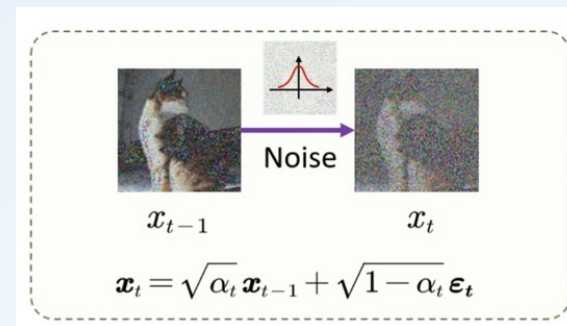
$q(\mathbf{x}_t | \mathbf{x}_{t-1})$: 表示以 $\mathbf{x}_{t-1}$ 为条件的时候, $\mathbf{x}_t$ 的分布。

$$\begin{aligned} q(\mathbf{x}_{t+1} | \mathbf{x}_{t-1}) &= \sqrt{\alpha_{t+1}}(\sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t) + \sqrt{1 - \alpha_{t+1}} \boldsymbol{\epsilon}_{t+1} \\ &= \sqrt{\alpha_{t+1} \alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t \alpha_{t+1}} \boldsymbol{\epsilon} \end{aligned}$$



$$\begin{aligned} q(\mathbf{x}_t | \mathbf{x}_0) &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_t \\ \bar{\alpha}_t &= \alpha_1 \alpha_2 \cdots \alpha_t \end{aligned}$$

“ ”



$$\begin{aligned} q(\mathbf{x}_t | \mathbf{x}_0) &= \sqrt{1 - \beta_1} \mathbf{x}_0 + \sqrt{\beta_1} \boldsymbol{\epsilon}_1 \\ \beta_1, \beta_2, \dots, \beta_T &= \sqrt{1 - \beta_2} \mathbf{x}_0 + \sqrt{\beta_2} \boldsymbol{\epsilon}_2 \\ &\vdots \\ &= \sqrt{1 - \beta_t} \mathbf{x}_0 + \sqrt{\beta_t} \boldsymbol{\epsilon}_t \end{aligned}$$

III

$$\mathbf{x}_t = \sqrt{1 - \beta_1} \cdots \sqrt{1 - \beta_t} \mathbf{x}_0 + \sqrt{1 - (1 - \beta_1) \cdots (1 - \beta_t)} \boldsymbol{\epsilon}_t$$

知乎 @哈根达斯

$q(\mathbf{x}_{t-1}|\mathbf{x}_t)$: 表示以 $\mathbf{x}_t$ 为条件的时候, $\mathbf{x}_{t-1}$ 的分布。

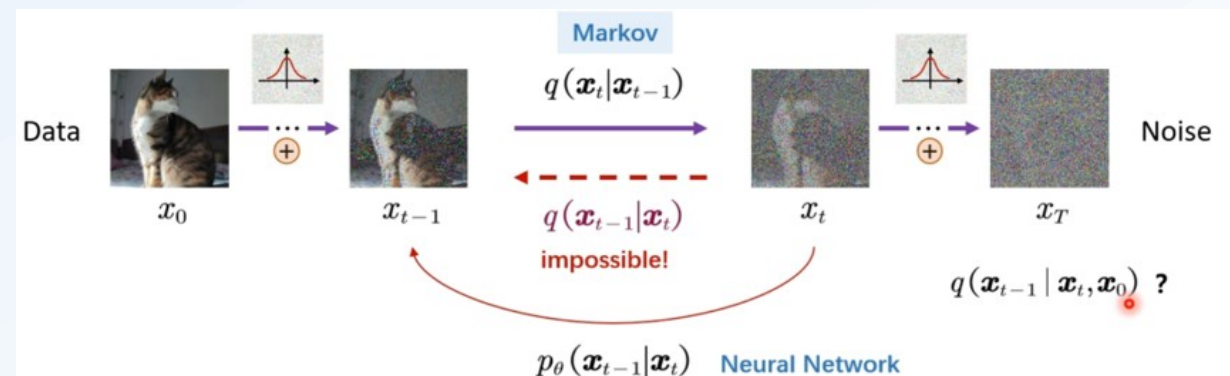
根据 $\mathbf{x}_0$ 表示图片, $\mathbf{x}_t$ 表示噪声图, 反向过程 $q(\mathbf{x}_0|\mathbf{x}_t)$ 本质上就是从噪声分布生成图片分布的过程, 模型要学习的就是这个反向生成的过程。

$$Loss = -E \left[\sum_{t>1} q(\mathbf{x}_{t-1}|\mathbf{x}_t) \log p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) \right]$$

困难的是我们无法的得到 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 分布的解析形式

因此我们需要找到一个具有解析形式的分布来代替 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$

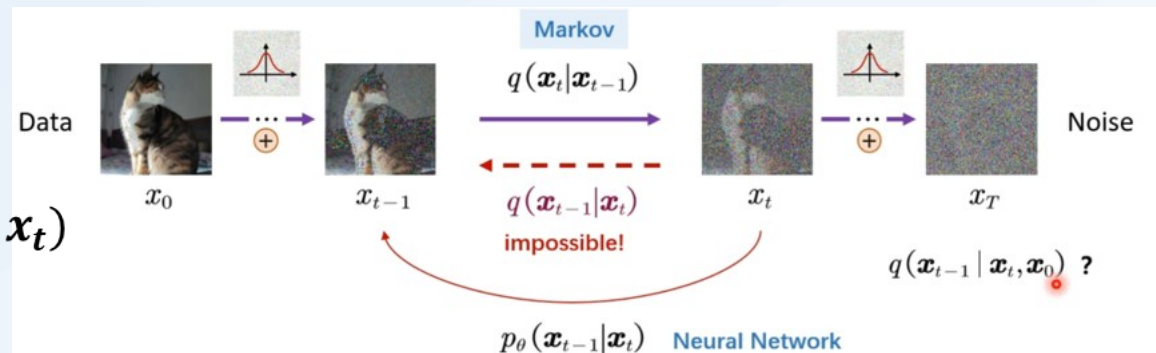
$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) \longrightarrow q(\mathbf{x}_{t-1}|\mathbf{x}_0, \mathbf{x}_t)$$



困难的是我们无法得到 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ 分布的解析形式

因此我们需要找到一个具有解析形式的分布来代替 $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) \longrightarrow q(\mathbf{x}_{t-1}|\mathbf{x}_0, \mathbf{x}_t)$$



$$\begin{aligned} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= \frac{q(\mathbf{x}_t, \mathbf{x}_{t-1}, \mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)} = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}, \mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)} \\ &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)q(\mathbf{x}_0)}{q(\mathbf{x}_t, \mathbf{x}_0)} \\ &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)q(\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)q(\mathbf{x}_0)} \longrightarrow \\ &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\ &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \end{aligned}$$

$$\begin{aligned} q(\mathbf{x}_t|\mathbf{x}_{t-1}) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1 - \alpha_t)\mathbf{I}) \\ q(\mathbf{x}_{t-1}|\mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbf{I}) \\ q(\mathbf{x}_t|\mathbf{x}_0) &= \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \end{aligned}$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = N(\mathbf{x}_{t-1}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

$$\begin{aligned} \boldsymbol{\mu} &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\mathbf{x}_0}{1 - \bar{\alpha}_t} \\ \boldsymbol{\Sigma} &= \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{I} \end{aligned}$$

此时用 $q(x_{t-1}|x_0, x_t)$ 代替对应的分布，重写损失函数

$$\begin{aligned} Loss &\rightarrow -E \left[\sum_{t>1} q(x_{t-1}|x_0, x_t) \log p_\theta(x_{t-1}|x_t) \right] \\ &= D_{KL}(q(x_{t-1}|x_0, x_t) \| p_\theta(x_t|x_{t-1})) \end{aligned}$$



$$\theta : \arg \min_{\theta} D_{KL}(q(x_{t-1}|x_t, x_0) \| p_\theta(x_{t-1}|x_t)) \rightarrow \arg \min_{\theta} \frac{1}{2\Sigma_{real}^{-1}} [(\mu_\theta - \mu_{real})^T (\mu_\theta - \mu_{real})]$$

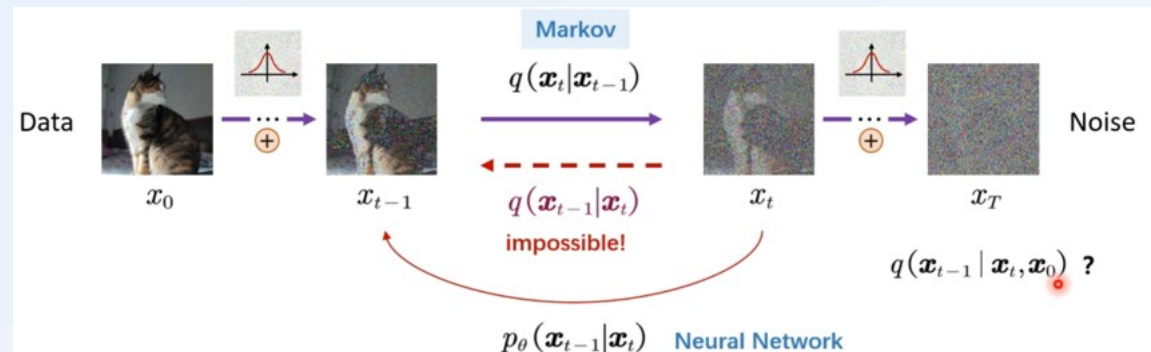
$$\theta : \arg \min_{\theta} \frac{1}{2\Sigma_{real}^{-1}} \|\mu_\theta - \mu_{real}\|_2^2$$

使用重参数化技巧，重写 $q(x_{t-1}|x_0, x_t) = N(x_{t-1}; \mu, \Sigma)$

$$\mu = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{1 - \bar{\alpha}_t} = \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right]$$

$$\mu_\theta = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right)$$

$$\begin{aligned} \theta : \arg \min_{\theta} \frac{1}{2\Sigma_{real}^{-1}} \|\mu_\theta - \mu_{real}\|_2^2 \\ &= \arg \min_{\theta} \frac{1}{2\Sigma_{real}^{-1}} \left\| \frac{1 - \alpha_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} [\epsilon_t - \epsilon_\theta(x_t, t)] \right\|_2^2 \\ &= \arg \min_{\theta} \frac{1}{2\Sigma_{real}^{-1}} \frac{(1 - \alpha_t)^2}{\alpha_t(1 - \bar{\alpha}_t)} \|\epsilon_t - \epsilon_\theta(x_t, t)\|_2^2 \end{aligned}$$



$$p_\theta(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta, \Sigma)$$

$$\begin{aligned} &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t + \sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0 - \frac{\sqrt{\alpha_t}(1 - \alpha_t)\sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\alpha_t}}}{1 - \bar{\alpha}_t} \\ &= \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})x_t}{1 - \bar{\alpha}_t} + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} - \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} \\ &= \frac{\alpha_t(1 - \bar{\alpha}_{t-1})x_t}{\sqrt{\alpha_t}(1 - \bar{\alpha}_t)} + \frac{\sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)x_0}{\sqrt{\alpha_t}\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_t)} - \frac{\sqrt{\alpha_t}\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)\sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\alpha_t}\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_t)} \\ &= \frac{1}{\sqrt{\alpha_t}} \left[\frac{(\alpha_t - \bar{\alpha}_t)x_t}{(1 - \bar{\alpha}_t)} + \frac{(1 - \alpha_t)x_0}{(1 - \bar{\alpha}_t)} - \frac{(1 - \alpha_t)\sqrt{1 - \bar{\alpha}_t}\epsilon}{(1 - \bar{\alpha}_t)} \right] \\ &= \frac{1}{\sqrt{\alpha_t}} \left[x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right] \end{aligned}$$

$$q(x_t|x_0) = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon_t \quad \bar{\alpha}_t = \alpha_1\alpha_2 \cdots \alpha_t$$

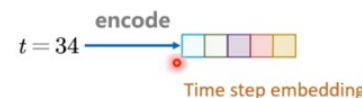
- 随机从图片数据集中采集部分的样本 x_0
- 通过样本 x_0 ，随机化高斯分布得到 x_t 和 ϵ_t 的数值

$$\theta : \arg \min_{\theta} \| \epsilon_t - \epsilon_{\theta}(x_t, t) \|_2^2$$

- 使用一个神经网络模型（Unet）根据 x_t 和 t 预测 ϵ_t 的数值，写为 $\epsilon_{\theta}(x_t, t)$

-

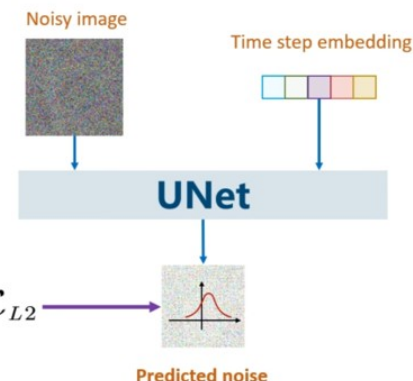
1. Randomly select a time step and encode it.



2. Add noise to image.



3. Train the UNet.



Algorithm 2 DDPM Training Algorithm

Require: Dataset $\mathcal{D}$, model parameters θ , learning rate η , noise schedule $\{\beta_t\}_{t=1}^T$, number of timesteps T , batch size M

- 1: Initialize θ randomly
- 2: **repeat**
- 3: Sample minibatch $\{x_0^{(i)}\}_{i=1}^M \sim \mathcal{D}$
- 4: **for** each sample $x_0^{(i)}$ **do**
- 5: Sample $t \sim \text{Uniform}(\{1, 2, \dots, T\})$
- 6: Sample $\epsilon^{(i)} \sim \mathcal{N}(0, \mathbf{I})$
- 7: $\alpha_t \leftarrow \prod_{s=1}^t (1 - \beta_s)$
- 8: $x_t^{(i)} \leftarrow \sqrt{\alpha_t}x_0^{(i)} + \sqrt{1 - \alpha_t}\epsilon^{(i)}$
- 9: $\hat{\epsilon}_{\theta}^{(i)} \leftarrow \epsilon_{\theta}(x_t^{(i)}, t)$
- 10: $\mathcal{L}^{(i)} \leftarrow \|\epsilon^{(i)} - \hat{\epsilon}_{\theta}^{(i)}\|_2^2$
- 11: **end for**
- 12: $\mathcal{L}(\theta) \leftarrow \frac{1}{M} \sum_{i=1}^M \mathcal{L}^{(i)}$
- 13: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$
- 14: **until** convergence
- 15: **return** trained parameters θ

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}, \boldsymbol{\Sigma})$$

$$\boldsymbol{\mu}_{\theta} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) \quad \boldsymbol{\Sigma} = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{I}$$

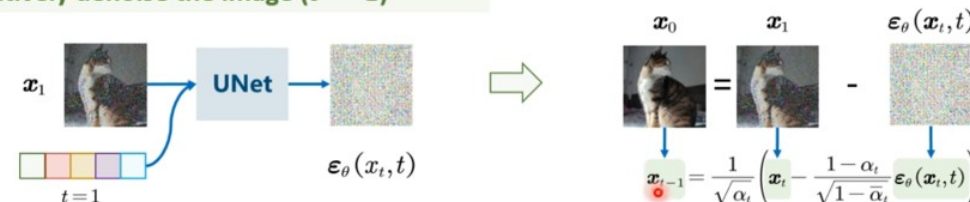
- 从高斯分布中随机采样多个样本作为 $\mathbf{x}_t$
- 将时间 t 和 $\mathbf{x}_t$ 输入到模型中，模型会输出一个 $\epsilon_{\theta}(\mathbf{x}_t, t)$
- 根据分布 $p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}, \boldsymbol{\Sigma})$ ，可以得到 $\mathbf{x}_{t-1}$ 的分布形式，在这个分布上采样就可以得到 $\mathbf{x}_t - \mathbf{x}_{t-1}$ 对
- 循环这个过程，直到得到 $\mathbf{x}_0$

1. Iteratively denoise the image ($T = 1000$)



2. Iteratively denoise the image ($T = 999..2$)

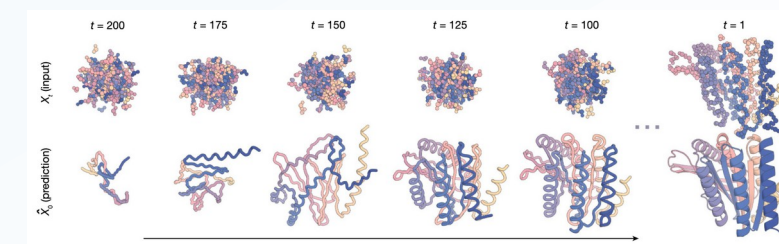
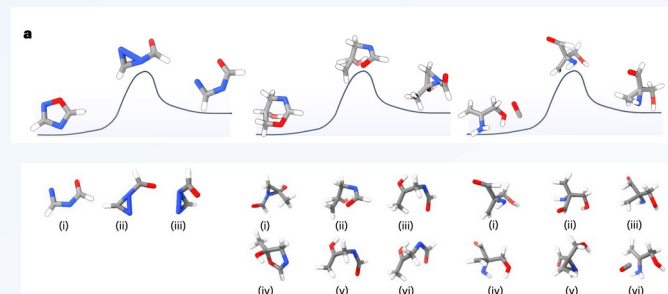
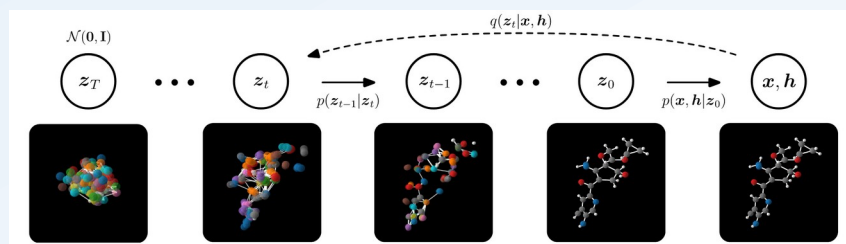
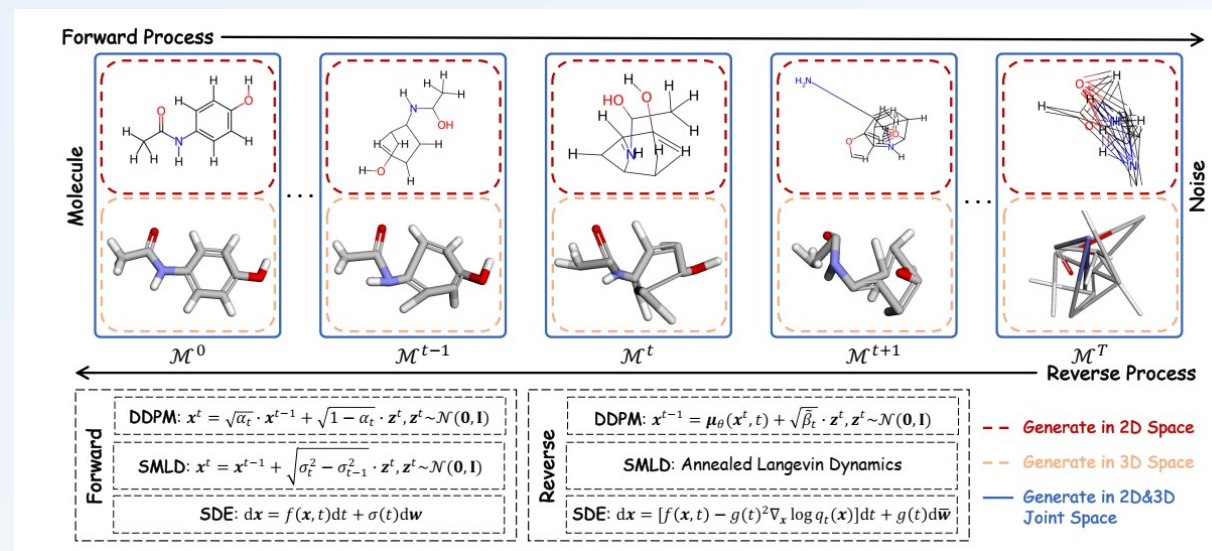
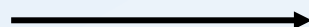
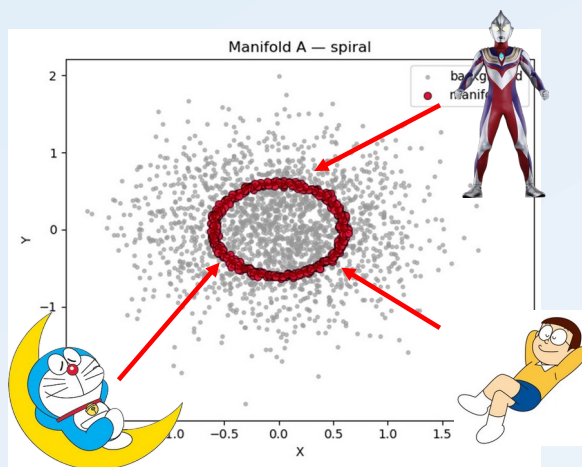
3. Iteratively denoise the image ($T = 1$)



Algorithm 3 DDPM Sampling Algorithm

Require: Trained model ϵ_{θ} , number of timesteps T , noise schedule $\{\beta_t\}_{t=1}^T$, image dimensions

- 1: Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
- 2: **for** $t = T$ **to** 1 **do**
- 3: $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ if $t > 1$, else $\mathbf{z} \leftarrow 0$
- 4: $\alpha_t \leftarrow \prod_{s=1}^t (1 - \beta_s)$
- 5: $\alpha_{t-1} \leftarrow \prod_{s=1}^{t-1} (1 - \beta_s)$
- 6: $\hat{\epsilon} \leftarrow \epsilon_{\theta}(\mathbf{x}_t, t)$
- 7: $\mathbf{x}_{t-1} \leftarrow \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \hat{\epsilon} \right) + \sqrt{\beta_t} \mathbf{z}$
- 8: **end for**
- 9: **return** generated sample $\mathbf{x}_0$



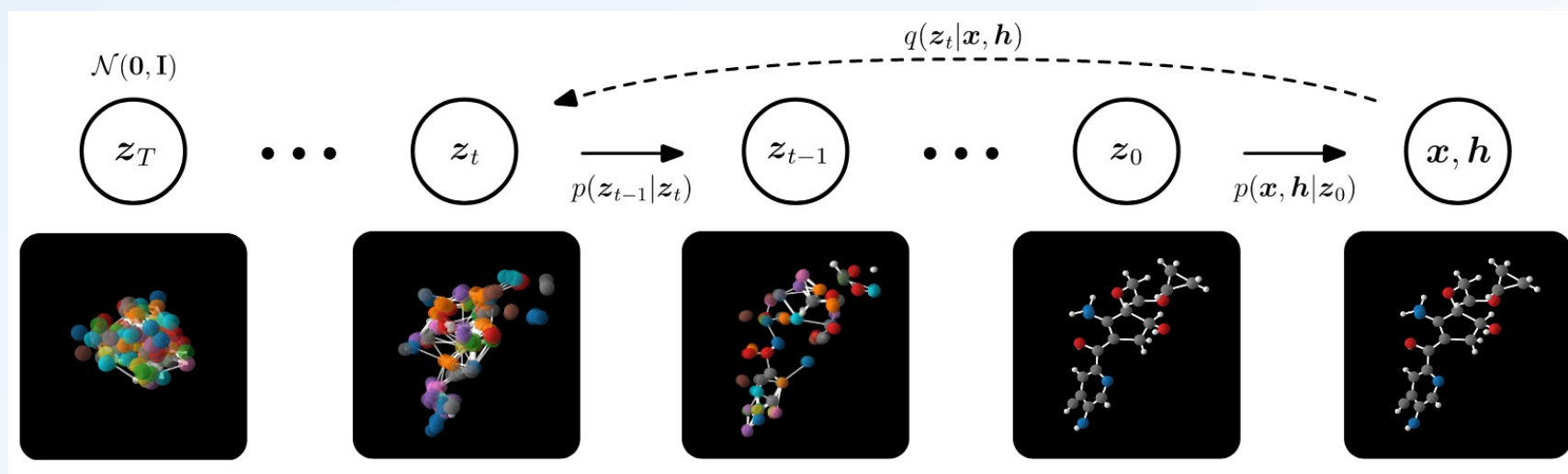
$\mathbf{x}$: 分子坐标 $\mathbf{h}$: 分子性质

$\mathbf{z}_t$: 分子变量

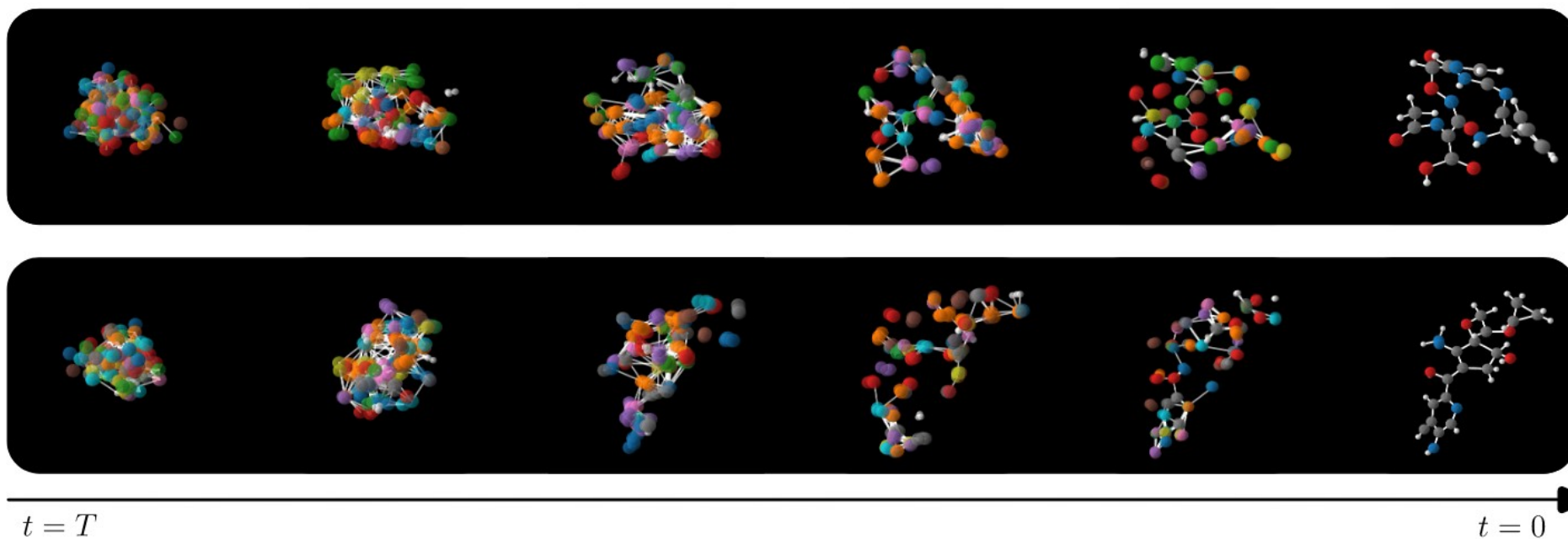
$$q(\mathbf{z}_t | \mathbf{x}, \mathbf{h}) = N_{xh}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}[\mathbf{x}, \mathbf{h}], (1 - \bar{\alpha}_t)\mathbf{I})$$

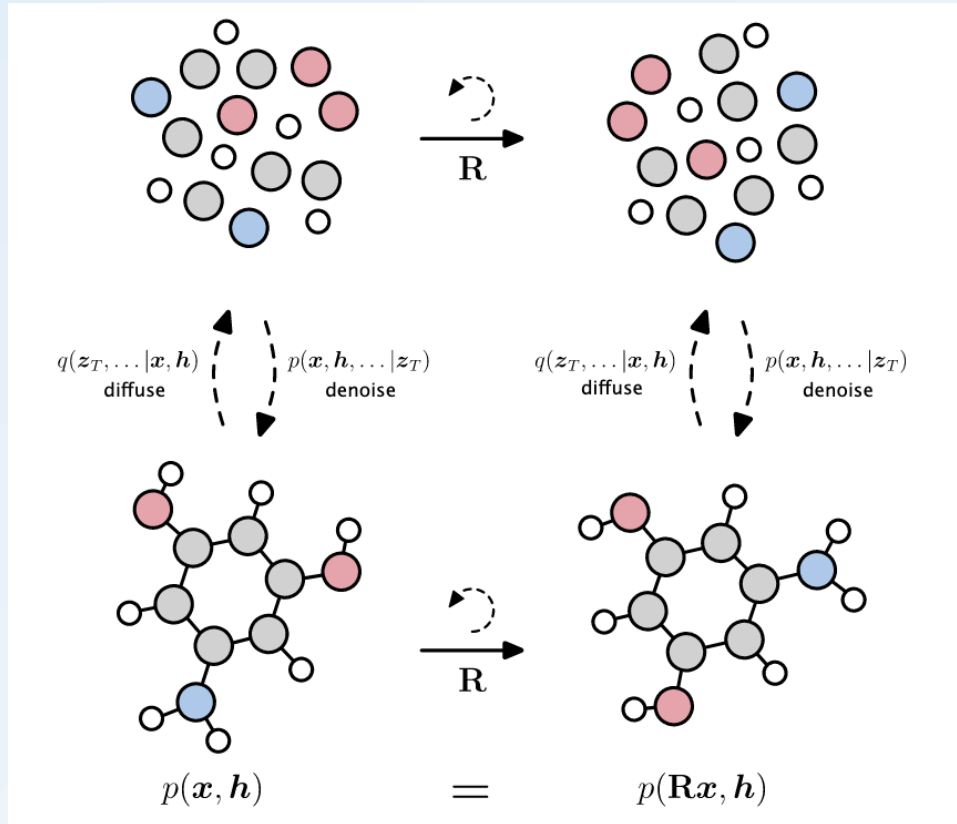
$$q(\mathbf{z}_s | \mathbf{z}_t) = N_{xh}(\mathbf{z}_s | \boldsymbol{\mu}_{t \rightarrow s}([\mathbf{x}, \mathbf{h}], \mathbf{z}_t), \sigma_{t \rightarrow s}^2 \mathbf{I})$$

$$q(\mathbf{z}_s | \mathbf{z}_t) \rightarrow q(\mathbf{z}_s | \mathbf{z}_t, \mathbf{y})$$



$$\begin{aligned}
 q(\mathbf{z}_t | \mathbf{x}, \mathbf{h}) &= N_{xh}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}[\mathbf{x}, \mathbf{h}], (1 - \bar{\alpha}_t)\mathbf{I}) \\
 &= N_x(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{x}, (1 - \bar{\alpha}_t)\mathbf{I}) \cdot N_h(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{h}, (1 - \bar{\alpha}_t)\mathbf{I}) \\
 &= N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{x}, (1 - \bar{\alpha}_t)\mathbf{I}) \cdot N(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{h}, (1 - \bar{\alpha}_t)\mathbf{I})
 \end{aligned}$$





$$q(\mathbf{z}_t | \mathbf{x}, \mathbf{h}) = N_{xh}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}[\mathbf{x}, \mathbf{h}], (1 - \bar{\alpha}_t)\mathbf{I})$$

$$q(\mathbf{z}_t | \mathbf{T}\mathbf{x}, \mathbf{h}) = N_{xh}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}[\mathbf{T}\mathbf{x}, \mathbf{h}], (1 - \bar{\alpha}_t)\mathbf{I}) = q(\mathbf{z}_t | \mathbf{T}\mathbf{x}, \mathbf{h})$$

$$N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{x}, (1 - \bar{\alpha}_t)\mathbf{I}) = N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}(\mathbf{x} - \bar{\mathbf{x}}), (1 - \bar{\alpha}_t)\mathbf{I})$$

$$\begin{aligned} N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{T}\mathbf{x}, (1 - \bar{\alpha}_t)\mathbf{I}) &= N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}(\mathbf{T}\mathbf{x} - \mathbf{T}\bar{\mathbf{x}}), (1 - \bar{\alpha}_t)\mathbf{I}) \\ &= N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}(\mathbf{x} - \bar{\mathbf{x}}), (1 - \bar{\alpha}_t)\mathbf{I}) \\ &= N_{x-mass}(\mathbf{z}_t | \sqrt{\bar{\alpha}_t}\mathbf{x}, (1 - \bar{\alpha}_t)\mathbf{I}) \end{aligned}$$

$$q(\mathbf{R}\mathbf{z}_t | \mathbf{R}\mathbf{x}, \mathbf{h}) = N_{xh}(\mathbf{R}\mathbf{z}_t | \sqrt{\bar{\alpha}_t}[\mathbf{R}\mathbf{x}, \mathbf{h}], (1 - \bar{\alpha}_t)\mathbf{I}) = q(\mathbf{z}_t | \mathbf{x}, \mathbf{h})$$

SE(3)

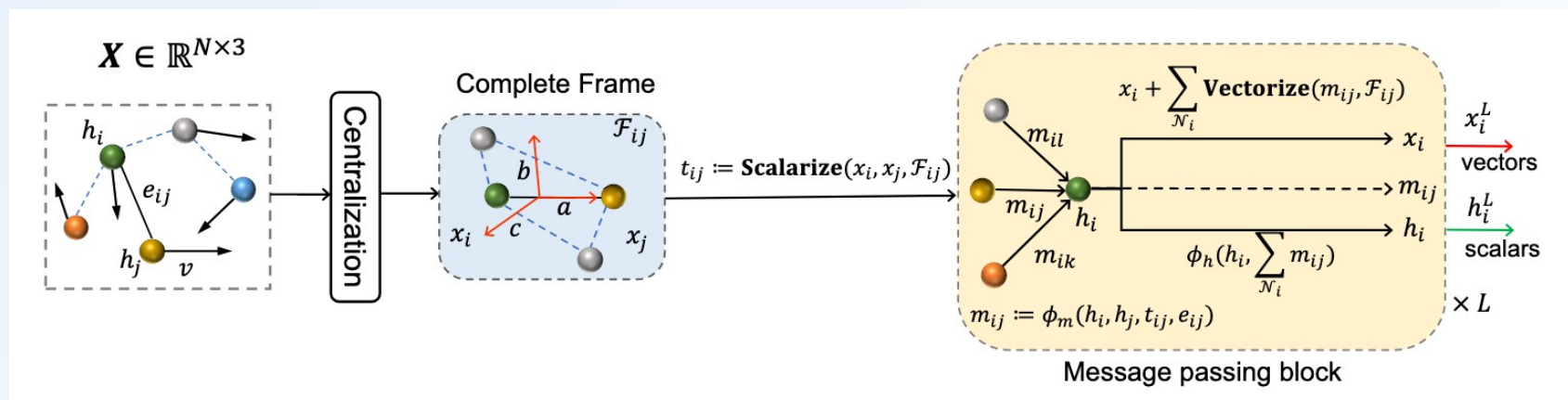
$$\hat{\mathbf{e}}_t^{(x)}, \hat{\mathbf{e}}_t^{(h)} = \text{EGCN} \left(\mathbf{z}_t^{(x)}, [\mathbf{z}_t^{(h)}, \frac{t}{T}] \right) - [\mathbf{z}_t^{(x)}, \mathbf{0}]$$

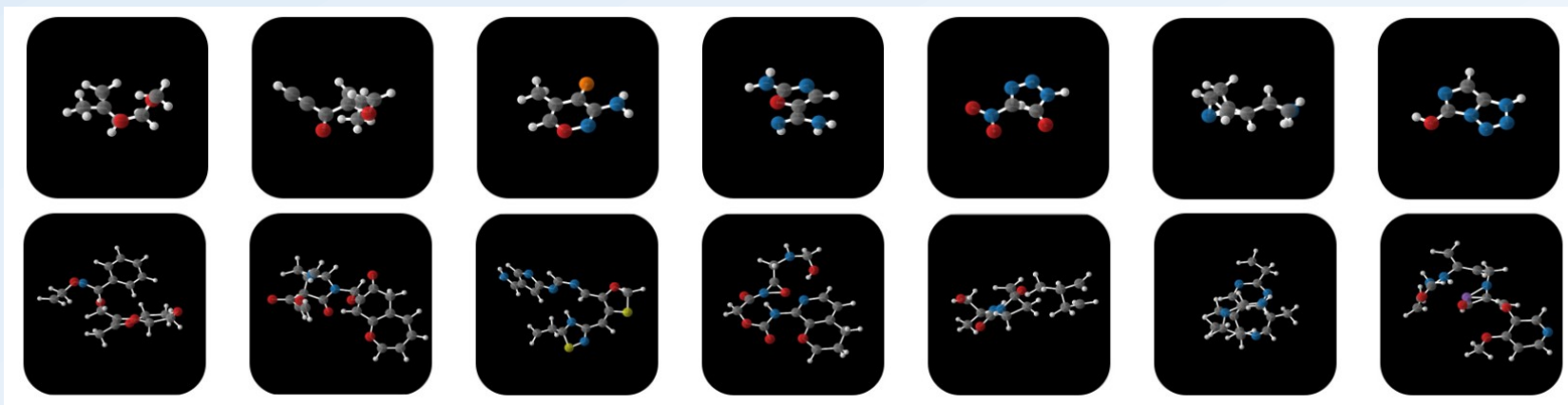
$$[\mathbf{x}^{l+1}, \mathbf{h}^{l+1}] = \text{EGCL}[\mathbf{x}^l, \mathbf{h}^l]$$

$$\mathbf{m}_{ij} = \phi_e(\mathbf{h}_i^l, \mathbf{h}_j^l, \mathbf{d}_{ij}^2, a_{ij})$$

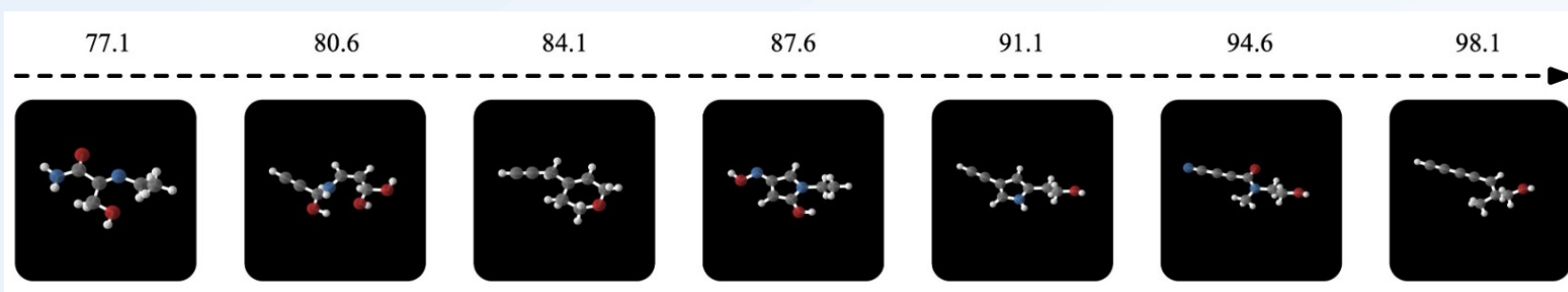
$$\mathbf{h}_i^{l+1} = \phi_h \left(\mathbf{h}_i^l, \sum_{j \neq i} \tilde{e}_{ij} \mathbf{m}_{ij} \right)$$

$$\mathbf{x}_i^{l+1} = \mathbf{x}_i^l + \sum_{j \neq i} \frac{(\mathbf{x}_i^l - \mathbf{x}_j^l)}{d_{ij} + 1} \phi_x(\mathbf{h}_i^l, \mathbf{h}_j^l, \mathbf{d}_{ij}^2, a_{ij})$$



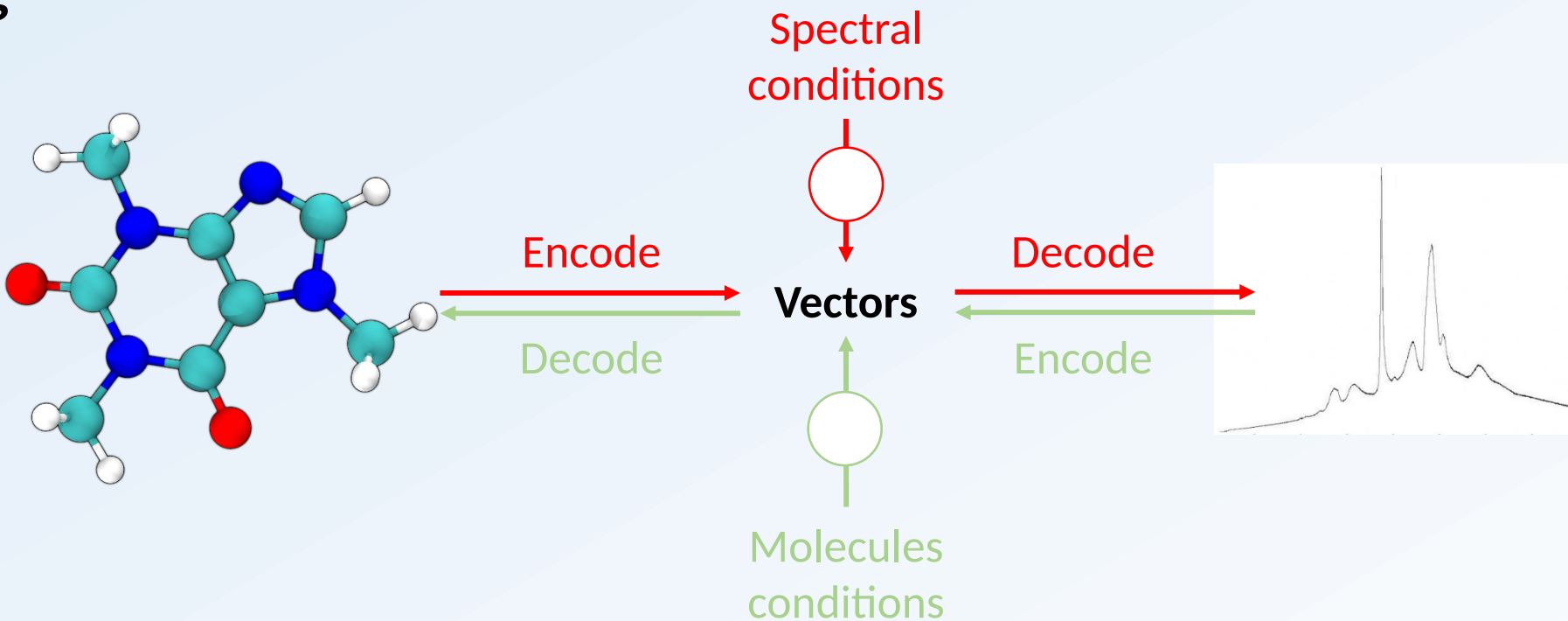


# Metrics	NLL	Atom stable (%)	Mol stable (%)
E-NF	-59.7	85.0	4.9
G-Schnet	N.A	95.7	68.1
GDM	-94.7	97.0	63.2
GDM-aug	-92.5	97.6	71.6
EDM (ours)	-110.7$\pm$1.5	98.7$\pm$0.1	82.0$\pm$0.4
Data		99.0	95.2



Task	α	$\Delta\epsilon$	ϵ_{HOMO}	ϵ_{LUMO}	μ	C_v
Units	Bohr ³	meV	meV	meV	D	$\frac{\text{cal}}{\text{mol K}}$
Naive (U-bound)	9.01	1470	645	1457	1.616	6.857
#Atoms	3.86	866	426	813	1.053	1.971
EDM	2.76	655	356	584	1.111	1.101
QM9 (L-bound)	0.10	64	39	36	0.043	0.040

◆ Thinking



Can the transformation of structure and spectrum be as simple as switching between Chinese and English?

Contents

1. Why neural network and deep learning?

- Learning map (where we are now)
- Tasks
- Basic knowledge and tips

2. Classic architectures

- CNN
- RNN
- GNN
- Transformer
- Reinforcement learning
- Diffusion

3. Replication code

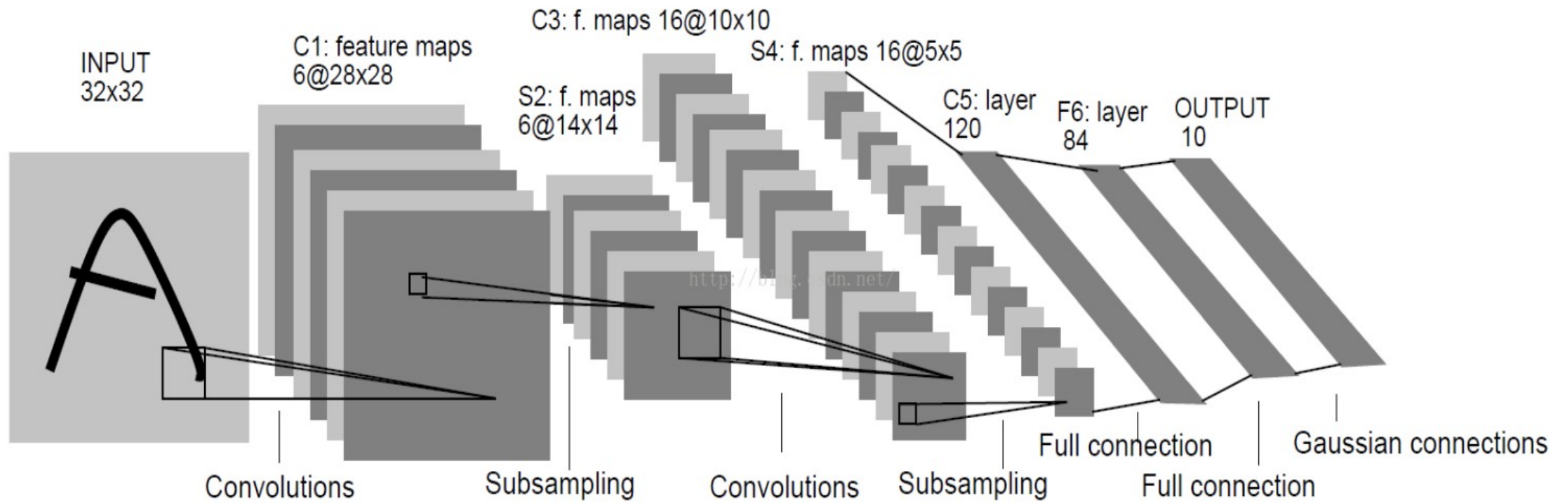
<https://pytorch.org/>

[Get Started](#)[Ecosystem](#) ▾[Edge](#) ▾[Blog](#)[Tutorials](#)[Docs](#) ▾[Resources](#) ▾[GitHub](#)

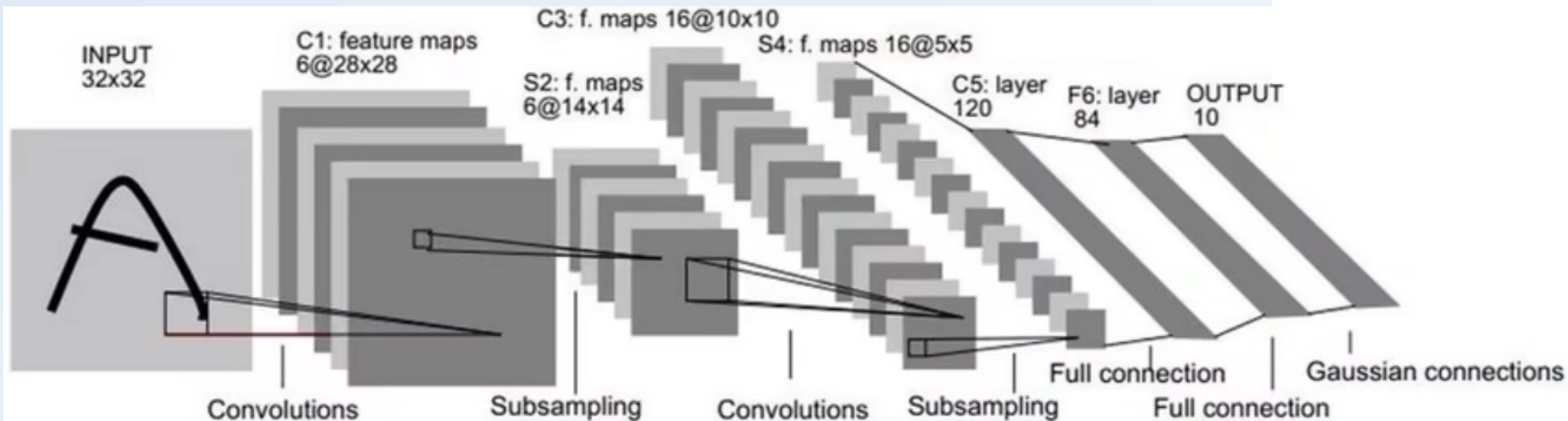
NOTE: Latest PyTorch requires Python 3.8 or later. For more details, see Python section below.

PyTorch Build	Stable (2.2.2)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	ROCm 5.7	CPU
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118</pre>			

[Previous versions of PyTorch](#) >[Amazon Web Services](#) >[Google Cloud Platform](#) >[Microsoft Azure](#) >



<https://www.cs.ryerson.ca/~aharley/vis/conv/>



	输入	卷积、池化、神经元	输出	训练参数
输入层*			32*32	0
卷积层1	32*32	6个 5*5卷积核 步长为1	6*28*28 (32-5+0) /1+1	1* (5*5) *6 +6=156
池化层1	6*28*28	2*2 步长为2	6*14*14	0
卷积层2	6*14*14	16个 5*5卷积核 步长为1	16*10*10 (14-5+0) /1+1	6* (5*5) *16 +16=2416
池化层2	16*10*10	2*2 步长为2	16*5*5	0
全连接层1	16*5*5	120个 5*5卷积核 步长为1	120*1*1 (5-5+0) /1+1	16* (5*5) *120+120=48120
全连接层2	120		84	120*84+84=10164
输出层	84		10	84*10+10=850

Thanks for your... ☐

Attention

